

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



HCMUTE

ĐỒ ÁN TỐT NGHIỆP
XÂY DỰNG BỘ XỬ LÝ RV32I PIPELINE 5 TẦNG
TÍCH HỢP CƠ CHẾ DỰ ĐOÁN NHÁNH
NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

GVHD: TS. PHẠM VĂN KHOA
SVTH: HUỲNH ĐÀO NGỌC KHẢI
MSSV: 22161268
SVTH: NGÔ QUANG THÀNH
MSSV: 22161317

Tp. Hồ Chí Minh, tháng 6 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



HCMUTE

ĐỒ ÁN TỐT NGHIỆP
XÂY DỰNG BỘ XỬ LÝ RV32I PIPELINE 5 TẦNG
TÍCH HỢP CƠ CHẾ DỰ ĐOÁN NHÁNH
NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

GVHD: TS. PHẠM VĂN KHOA
SVTH: HUỲNH ĐÀO NGỌC KHẢI
MSSV: 22161268
SVTH: NGÔ QUANG THÀNH
MSSV: 22161317

Tp. Hồ Chí Minh, tháng 6 năm 2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Xây dựng hệ xử lý RV32I Pipeline 5 tầng tích hợp cơ chế dự đoán nhánh

Sinh viên: + Huỳnh Đào Ngọc Khải MSSV: 22161268
+ Ngô Quang Thành MSSV: 22161317
Hướng dẫn: TS. Phạm Văn Khoa

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Hợp lý

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết, có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Sử dụng Verilog thiết kế FPGA xác minh thiết kế

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá/ kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Đạt

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Tốt

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Xuất sắc

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo, tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

8. Đánh giá về sự trùng lặp của đề tài

Cần không định đề tài có trùng lặp hay không? Nếu có đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Đạt - Cho bảo vệ đề tài

Tp. HCM, ngày 15 tháng 06 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)

Phạm Văn Khoa

* Giảng viên hướng dẫn có thể ghi tiếp vào mặt sau

PHIẾU GIAO NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên: Ngô Quang Thành MSSV: 22161317

Họ và tên: Huỳnh Đào Ngọc Khải MSSV: 22161268

2. Thông tin đề tài

Tên đề tài: Xây dựng bộ xử lý RV32I pipeline 5 tầng tích hợp cơ chế dự đoán nhánh

Mục đích đề tài: Thiết kế và xây dựng bộ vi xử lý RISC-V RV32I 32-bit sử dụng kiến trúc pipeline 5 tầng, tích hợp cơ chế xử lý hazard và dự đoán nhánh nhằm nâng cao hiệu suất thực thi chương trình

3. Nội dung thực hiện

- Nghiên cứu kiến trúc tập lệnh RISC-V RV32I và mô hình pipeline 5 tầng.
- Thiết kế và xây dựng bộ xử lý RV32I pipeline 5 tầng bằng ngôn ngữ mô tả phân cứng SystemVerilog.
- Thiết kế và tích hợp cơ chế dự đoán nhánh động
- Thực hiện mô phỏng, kiểm chứng chức năng và đánh giá hiệu năng của bộ xử lý.

4. Lời cam đoan của sinh viên

Chúng tôi cam đoan đồ án tốt nghiệp này là kết quả nghiên cứu và thực hiện của nhóm dưới sự hướng dẫn của giảng viên hướng dẫn. Các kết quả, số liệu và nội dung trình bày trong đồ án là trung thực, được trích dẫn đầy đủ theo quy định và chưa từng được sử dụng để bảo vệ bất kỳ học vị nào trước đây. Chúng tôi hoàn toàn chịu trách nhiệm về nội dung cam đoan này.

TP.HCM, ngày 19 tháng 6 năm 2026

Sinh viên thực hiện:

Ngô Quang Thành

Huỳnh Đào Ngọc Khải

LỜI CẢM ƠN

Trước hết, nhóm xin gửi lời cảm ơn chân thành và sâu sắc đến thầy Phạm Văn Khoa, người đã tận tình hướng dẫn, hỗ trợ và định hướng chuyên môn cho chúng em trong suốt quá trình thực hiện đề tài. Sự tận tâm và kiến thức quý báu của thầy là nguồn động lực lớn giúp nhóm hoàn thành đồ án một cách hiệu quả và bài bản.

Nhóm cũng xin chân thành cảm ơn quý thầy cô trong khoa đã truyền đạt kiến thức trong suốt thời gian học tập, tạo nền tảng vững chắc cho việc nghiên cứu và thực hiện đề tài này.

Cuối cùng, chúng em xin cảm ơn gia đình và bạn bè đã luôn động viên, tạo điều kiện thuận lợi để nhóm có thể tập trung hoàn thành tốt nhiệm vụ được giao.

Mặc dù đã cố gắng hết sức, nhưng do giới hạn về thời gian và kinh nghiệm, bài báo cáo không tránh khỏi những thiếu sót. Nhóm rất mong nhận được những góp ý quý báu từ thầy cô để hoàn thiện hơn trong các nghiên cứu sau này.

MỤC LỤC

DANH MỤC ẢNH.....	VII
DANH MỤC BẢNG.....	XI
DANH MỤC CÁC TỪ VIẾT TẮT.....	XII
CHƯƠNG 1: GIỚI THIỆU.....	1
1.1 Lý do chọn đề tài.....	1
1.2 Mục tiêu đề tài.....	2
1.3 Phạm vi thực hiện.....	3
1.4 Đối tượng nghiên cứu.....	3
1.5 Phương pháp nghiên cứu.....	4
1.6 Bố cục đề tài.....	4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	6
2.1 Giới thiệu về kiến trúc, tập lệnh RISC-V.....	6
2.1.1 Giới thiệu RISC-V.....	6
2.1.2 Ưu điểm và nhược điểm của RISC-V.....	7
2.1.3 Tập lệnh RV32I.....	9
2.1.3.1 Tập thanh ghi.....	9
2.1.3.2 Định dạng lệnh.....	10
2.2 Kỹ thuật Pipeline.....	17
2.2.1 Giới thiệu.....	17
2.2.2 Hoạt động của pipeline 5 tầng.....	17
2.2.3 Xung đột trong pipeline.....	19
2.3 Cơ chế dự đoán nhánh.....	21
2.3.1 Cơ chế dự đoán nhánh tĩnh.....	21
2.3.2 Cơ chế dự đoán nhánh động.....	22
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG.....	23
3.1 Tổng quan về hệ thống.....	23
3.1.1 Yêu cầu của hệ thống.....	23
3.1.2. Mục tiêu của hệ thống.....	24
3.2 RISC-V 5 tầng tích hợp cơ chế dự đoán nhánh.....	24

3.2.1 Hạn chế của kiến trúc RISC-V Pipeline truyền thống	24
3.2.2 Kiến trúc RISC-V cải tiến với dự đoán nhánh động	25
3.2.3 Sơ đồ khối tổng thể	25
3.3 Thiết kế chi tiết các khối RISC-V Pipeline	26
3.4 Thiết kế khối dự đoán nhánh	35
3.4.1 Ý tưởng thiết kế	35
3.4.2 Kiến trúc tổng thể của khối dự đoán nhánh	36
3.4.3 Branch Target Buffer (BTB)	37
3.4.4 Pattern History Table (PHT)	38
3.5 Phương pháp đánh giá	41
CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ	42
4.1 Kết quả mô phỏng từng khối	42
4.1.1 Khối Program Counter	42
4.1.2 Khối Instruction Memory	43
4.1.3 Khối Register File	45
4.1.4 Khối Arithmetic Logic Unit (ALU)	46
4.1.5 Khối Data Memory	48
4.1.6 Khối Control Logic	49
4.1.7 Khối Pipeline Register	51
4.1.8 Khối Hazard Unit	52
4.2 Kết quả mô phỏng toàn hệ thống	58
4.2.1 RV32I Pipeline 5 tầng cơ bản	66
4.2.2 Pipeline 5 tầng có tích hợp cơ chế dự đoán nhánh	76
4.3 Đánh giá hiệu năng pipeline và pipeline có tích hợp dự đoán nhánh ..	84
4.3.1 Chương trình kiểm thử	84
4.3.2 Kiểm chứng chức năng	87
4.3.2.1 Chương trình tìm số nguyên tố	88
4.3.2.2 Thuật toán Bubble Sort	90
4.3.2.3 Thuật toán Quick Sort	92

4.3.3 Đánh giá hiệu suất	95
4.3.3.1 Đánh giá phần cứng	95
4.3.3.2 Đánh giá chương trình tìm số nguyên tố.....	97
4.3.3.3 Đánh giá thuật toán Bubble Sort	98
4.3.3.4 Đánh giá thuật toán Quick Sort	99
4.3.4 Nhận xét và đánh giá.....	100
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	102
5.1 Kết luận	102
5.2 Hướng phát triển	102
TÀI LIỆU THAM KHẢO	103

DANH MỤC ẢNH

Hình 2.1: Định dạng lệnh RV32I	11
Hình 2.2: Định dạng lệnh R-type	13
Hình 2.3: Định dạng lệnh I-type	13
Hình 2.4: Định dạng mã lệnh I-type cho các phép toán immediate trong RISC-V	14
Hình 2.5: Định dạng mã lệnh Load và Store trong RISC-V	14
Hình 2.6: Sơ đồ đường ống 5 giai đoạn.	18
Hình 2.7: Xung đột cấu trúc khi dùng chung bộ nhớ.....	20
Hình 2.8: Minh họa xung đột dữ liệu.....	20
Hình 2.9: Địa chỉ PC tại EX.....	21
Hình 3.1: Sơ đồ tổng quan pipeline RISC-V 5 tầng với dự đoán nhánh [2].....	25
Hình 3.2: Sơ đồ Program Counter.....	26
Hình 3.3: Instruction memory	27
Hình 3.4: Register file.....	27
Hình 3.5: Data memory.....	28
Hình 3.6: Cấu trúc ALU.....	29
Hình 3.7: Control logic.	30
Hình 3.8: pipeline register.....	31
Hình 3.9: Hazard detection	32
Hình 3.10: Forwarding control logic.....	33
Hình 3.11: Branch Unit.....	34
Hình 3.12: Kiến trúc khối dự đoán nhánh.....	36
Hình 3.13: Branch Target Buffer.	37
Hình 3.14: Pattern History Table.	39
Hình 3.15: Máy trạng thái bộ đếm bão hòa 2-bit của PHT.	41
Hình 4.1: Dạng sóng mô tả chức năng cập nhật địa chỉ mới của khối PC.....	42
Hình 4.2: Chương trình được nạp vào Instruction Memory	43
Hình 4.3: Dạng sóng mô tả chức năng nạp dữ liệu của khối IMEM	44

Hình 4.4: Dạng sóng mô tả chức năng ghi và đọc dữ liệu của khối Register File	45
Hình 4.5: Dạng sóng mô tả các hoạt động chính của khối xử lý logic số học- ALU	46
Hình 4.6: Dạng sóng mô tả chức năng ghi và đọc dữ liệu của khối Data Memory	48
Hình 4.7A: Dạng sóng mô tả chức năng giải mã lệnh khối Control Logic 0 – 7ns	50
Hình 4.7B: Dạng sóng mô tả chức năng giải mã lệnh khối Control Logic 7 – 17ns	50
Hình 4.8: Dạng sóng mô tả chức năng truyền dữ liệu tuần tự của khối Pipeline Register.....	51
Hình 4.9: Dạng sóng mô tả chức năng lấy dữ liệu có chọn lọc của khối Forwarding Unit.....	53
Hình 4.10A: Dạng sóng mô tả chức năng phát hiện stall của khối hazard dectection từ 0 – 60ns.....	55
Hình 4.10B: Dạng sóng mô tả chức năng phát hiện stall của khối Hazard Dectection từ 60 – 150ns.....	55
Hình 4.11A: Dạng sóng mô tả chức năng rẽ nhánh của khối branch Unit từ 0 – 9ns	57
Hình 4.11B: Dạng sóng mô tả chức năng rẽ nhánh của khối branch Unit từ 9 – 20ns	57
Hình 4.12: Mã giả (pseudo-code) của chương trình kiểm thử toàn hệ thống	60
Hình 4.13: Lưu đồ của chương trình kiểm thử toàn hệ thống.....	61
Hình 4.14A: Dạng sóng hoạt động toàn hệ thống từ 0 – 105ns.....	66
Hình 4.14B: Dạng sóng hoạt động toàn hệ thống từ 105 – 295ns	67
Hình 4.14C: Dạng sóng hoạt động toàn hệ thống từ 255 – 445ns	69
Hình 4.14D: Dạng sóng hoạt động toàn hệ thống từ 445 - 595ns.....	74
Hình 4.14E: Dạng sóng hoạt động toàn hệ thống từ 1665 - 1865ns	75

Hình 4.15A: Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 0 - 105ns	76
Hình 4.15B: Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 175 - 295ns	77
Hình 4.15C: Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 285 - 455ns	78
Hình 4.15D: Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 445 - 615ns	79
Hình 4.15E: Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 595 - 735ns	80
Hình 4.15F: Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 705 - 875ns	81
Hình 4.15G: Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 1525 - 1615ns	82
Hình 4.15H: Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 1615 - 1745ns	83
Hình 4.16: Lưu đồ của chương trình tìm số nguyên tố	85
Hình 4.17: Lưu đồ của chương trình thuật toán Bubble Sort	86
Hình 4.18: Lưu đồ của chương trình thuật toán quick sort	87
Hình 4.19: Mô tả dạng sóng hoạt động của chương trình tìm số nguyên tố	88
Hình 4.20: Dữ liệu bộ nhớ đầu vào và đầu ra của chương trình tìm số nguyên tố	89
Hình 4.21: Mô tả dạng sóng hoạt động của thuật toán Bubble Sort	90
Hình 4.22: Dữ liệu bộ nhớ trước khi sắp xếp theo thuật toán Bubble Sort	92
Hình 4.23: Dữ liệu bộ nhớ sau khi sắp xếp theo thuật toán Bubble Sort	92
Hình 4.24: Mô tả dạng sóng hoạt động của thuật toán Quick Sort	93
Hình 4.25: Dữ liệu bộ nhớ trước khi sắp xếp theo thuật toán Quick Sort	94
Hình 4.26: Dữ liệu bộ nhớ sau khi sắp xếp theo thuật toán Quick Sort	94
Hình 4.27: So sánh độ chiếm dụng Logic element và Register giữa hai kiến trúc	95

Hình 4.28: So sánh độ chiếm dụng Memory Bit giữa hai kiến trúc	95
Hình 4.29: So sánh tần số hoạt động cực đại giữa hai kiến trúc	96
Hình 4.30: So sánh công suất tiêu thụ giữa hai kiến trúc.....	96

DANH MỤC BẢNG

Bảng 2.1: Chức năng các thanh ghi của RV32I.....	10
Bảng 2.2: Tổng quan định dạng lệnh trong RV32I.....	11
Bảng 2.3: Các trường của RV32I.....	12
Bảng 2.4: Danh sách lệnh S-Type.....	15
Bảng 2.5: Danh sách lệnh B-Type	15
Bảng 2.6: Danh sách lệnh U-Type	16
Bảng 2.7: Lệnh J-Type và ứng dụng.....	17
Bảng 3.1: Tín hiệu của module Program Counter	26
Bảng 3.2: Tín hiệu của module imem	27
Bảng 3.3: Tín hiệu của module register file.....	28
Bảng 3.4: Tín hiệu của module Data Memory.....	29
Bảng 3.5: Tín hiệu module ALU.....	30
Bảng 3.6: Tín hiệu module Control Logic	30
Bảng 3.7: Các tín hiệu đi qua từng thanh ghi Pipeline	32
Bảng 3.8: Tín hiệu module Hazard Detection.....	33
Bảng 3.9: Tín hiệu module Forwarding control logic.....	34
Bảng 3.10: Mô tả tín hiệu Branch Unit.....	35
Bảng 3.11: Tín hiệu module BTB	38
Bảng 3.12: Tín hiệu module PHT	39
Bảng 4.1: Mã Assembly dùng mô phỏng cho chương trình kiểm thử toàn hệ thống	61
Bảng 4.2: Đo lường độ hiệu quả của chương trình tìm số nguyên tố chưa được tối ưu.....	97
Bảng 4.3: Đo lường độ hiệu quả của chương trình tìm số nguyên tố được tối ưu	98
Bảng 4.4: Đo lường độ hiệu quả của chương trình thuật toán Bubble Sort.....	98
Bảng 4.5: Đo lường độ hiệu quả của chương trình thuật toán Quick Sort.....	99

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
ABI	Application Binary Interface
ALU	Arithmetic Logic Unit
BPU	Branch Prediction Unit
BTB	Branch Target Buffer
CPU	Central Processing Unit
CPI	Cycles Per Instruction
DMEM	Data Memory
EX	Execute
FPGA	Field Programmable Gate Array
ID	Instruction Decode
IF	Instruction Fetch
IMEM	Instruction Memory
IPC	Instructions Per Cycle
ISA	Instruction Set Architecture
IoT	Internet of Thing
MEM	Memory Access
PC	Program Counter
PHT	Pattern History Table
RAM	Random Access Memory
RISC-V	Reduced Instruction Set Computer V
RTL	Register Transfer Level
RV32I	RISC-V 32-bit Integer Base Instruction Set
SRAM	Static Random Access Memory
WB	Write Back

CHƯƠNG 1: GIỚI THIỆU

1.1 Lý do chọn đề tài

Trong bối cảnh công nghệ số và ngành công nghiệp bán dẫn phát triển mạnh mẽ, nhu cầu về các hệ thống xử lý có hiệu năng cao, tiêu thụ năng lượng thấp và khả năng tùy biến linh hoạt ngày càng trở nên quan trọng. Các bộ vi xử lý và vi điều khiển hiện nay được ứng dụng rộng rãi trong nhiều lĩnh vực như hệ thống nhúng, thiết bị IoT, điện tử tiêu dùng, ô tô thông minh và tự động hóa công nghiệp. Những dòng vi điều khiển phổ biến như ARM Cortex-M đều yêu cầu khả năng xử lý nhanh và ổn định để đáp ứng các tác vụ thời gian thực.

Trong các hệ thống này, kiến trúc pipeline là một kỹ thuật quan trọng nhằm tăng tốc độ xử lý bằng cách cho phép nhiều lệnh được thực thi song song ở các giai đoạn khác nhau [1]. Mô hình pipeline 5 tầng (IF–ID–EX–MEM–WB) được sử dụng phổ biến trong nhiều thiết kế CPU nhờ khả năng cân bằng giữa hiệu năng và độ phức tạp phần cứng [1, 11, 14]. Tuy nhiên, pipeline thường gặp phải các hazard, đặc biệt là control hazard do các lệnh rẽ nhánh gây ra [1, 12]. Khi gặp lệnh rẽ nhánh, CPU chưa thể xác định ngay luồng thực thi tiếp theo, khiến pipeline phải stall hoặc flush, làm tăng số chu kỳ thực thi và giảm hiệu năng tổng thể [1, 12].

Đối với các hệ thống nhúng có tài nguyên phần cứng hạn chế, việc nâng cao hiệu suất đường ống xử lý là rất quan trọng nhằm đáp ứng yêu cầu thời gian thực. Vì vậy, các bộ xử lý hiện đại thường sử dụng cơ chế dự đoán nhánh để xác định trước hướng thực thi của các lệnh rẽ nhánh [2, 12]. Cơ chế này giúp giảm số lần tạm dừng và xóa lệnh sai, từ đó cải thiện CPI và nâng cao thông lượng xử lý của hệ thống [7, 13].

Bên cạnh đó, kiến trúc RISC-V đang trở thành xu hướng mới trong lĩnh vực thiết kế vi xử lý nhờ đặc tính mã nguồn mở, khả năng mở rộng linh hoạt và không phụ thuộc vào bản quyền thương mại [1, 11]. Điều này tạo điều kiện thuận lợi cho nghiên cứu học thuật cũng như phát triển các bộ xử lý tùy chỉnh phục vụ cho giáo dục, nghiên cứu và công nghiệp. Hiện nay, nhiều hệ thống nhúng và vi điều khiển thế hệ mới cũng đang bắt đầu áp dụng kiến trúc RISC-V để thay thế cho các kiến trúc truyền thống [1, 11].

Tuy nhiên, phần lớn các thiết kế RISC-V pipeline cơ bản trong môi trường học thuật hiện nay chủ yếu tập trung vào hoạt động pipeline và xử lý data hazard, trong khi cơ chế Branch Prediction vẫn chưa được nghiên cứu và đánh giá đầy đủ ở mức RTL [11, 13]. Việc nghiên cứu và triển khai bộ dự đoán nhánh trên bộ xử lý RISC-V RV32I pipeline 5 tầng sẽ giúp nâng cao hiệu năng xử lý, đồng thời giúp hiểu rõ hơn về các kỹ thuật tối ưu vi kiến trúc đang được áp dụng trong các CPU hiện đại [2, 7, 13].

Xuất phát từ những lý do trên, nhóm quyết định thực hiện đề tài “Xây dựng bộ xử lý RISC-V RV32I Pipeline 5 tầng tích hợp cơ chế dự đoán nhánh” nhằm nghiên cứu, thiết kế và đánh giá hiệu quả của cơ chế Branch Prediction trên bộ xử lý RISC-V RV32I pipeline 5 tầng ở mức RTL. Đề tài không chỉ mang ý nghĩa học thuật trong lĩnh vực kiến trúc máy tính mà còn có giá trị thực tiễn trong định hướng phát triển các hệ thống vi xử lý và vi điều khiển hiệu năng cao trong tương lai.

1.2 Mục tiêu đề tài

Mục tiêu chính của đề tài là thiết kế và xây dựng bộ xử lý RISC-V RV32I pipeline 5 tầng ở mức RTL, đồng thời tích hợp cơ chế dự đoán nhánh nhằm cải thiện hiệu năng xử lý và giảm ảnh hưởng của control hazard trong pipeline.

Các mục tiêu cụ thể của đề tài bao gồm:

- Thiết kế kiến trúc bộ xử lý RV32I pipeline 5-stage gồm các giai đoạn IF, ID, EX, MEM và WB.
- Xây dựng datapath và control unit cho bộ xử lý theo chuẩn tập lệnh RV32I.
- Thiết kế Forwarding Unit và Hazard Detection Unit để xử lý data hazard.
- Tích hợp cơ chế Branch Prediction động sử dụng bộ đếm bão hòa 2-bit kết hợp với Branch Target Buffer.
- Xây dựng cơ chế flush pipeline khi xảy ra dự đoán nhánh sai.
- Xây dựng testbench và thực hiện mô phỏng để kiểm chứng chức năng của hệ thống.

- Đánh giá hiệu năng thông qua các chỉ số như CPI và branch prediction accuracy.

Thông qua đề tài, nhóm mong muốn làm rõ hiệu quả của kỹ thuật Branch Prediction trong việc cải thiện hiệu năng bộ xử lý RISC-V pipeline, đồng thời tạo nền tảng cho các nghiên cứu mở rộng trong lĩnh vực thiết kế CPU và SoC.

1.3 Phạm vi thực hiện

Đề tài tập trung nghiên cứu và triển khai bộ xử lý RISC-V RV32I pipeline 5 tầng ở mức RTL bằng ngôn ngữ Verilog/SystemVerilog. Phạm vi thực hiện của đề tài bao gồm:

- Thiết kế pipeline RV32I 5-stage theo mô hình IF–ID–EX–MEM–WB.
- Thiết kế các module chức năng như Program Counter, Register File, ALU, Control Unit và Instruction Memory.
- Xử lý data hazard bằng cơ chế forwarding và stall.
- Tích hợp cơ chế Branch Prediction động sử dụng bộ đếm bão hòa 2-bit và Branch Target Buffer.
- Thực hiện mô phỏng và kiểm chứng hệ thống bằng các công cụ như Vivado và QuestaSim.
- Đánh giá hiệu năng hệ thống thông qua các chương trình kiểm thử có nhiều lệnh rẽ nhánh.

Đề tài không tập trung vào các kiến trúc nâng cao như superscalar, out-of-order execution hoặc các bộ dự đoán nhánh phức tạp như perceptron predictor hay TAGE predictor. Ngoài ra, việc tối ưu công suất tiêu thụ và triển khai ASIC cũng không nằm trong phạm vi nghiên cứu của đề tài.

1.4 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là bộ xử lý RISC-V RV32I pipeline 5 tầng và các kỹ thuật xử lý control hazard trong pipeline thông qua cơ chế dự đoán nhánh.

Cụ thể, đề tài tập trung nghiên cứu các nội dung sau:

- Kiến trúc tập lệnh RISC-V RV32I.
- Cấu trúc pipeline 5-stage của bộ xử lý.
- Các dạng hazard trong pipeline, đặc biệt là control hazard do branch instruction gây ra.
- Bộ dự đoán nhánh động sử dụng bộ đếm bão hòa 2-bit (PHT).
- BTB và cơ chế cập nhật predictor.
- Cơ chế flush pipeline khi xảy ra branch misprediction.
- Các chỉ số đánh giá hiệu năng như CPI và prediction accuracy.

Thông qua việc nghiên cứu các nội dung trên, đề tài hướng đến việc đánh giá khả năng cải thiện hiệu năng của bộ xử lý RISC-V pipeline khi áp dụng cơ chế Branch Prediction.

1.5 Phương pháp nghiên cứu

Đề tài được thực hiện dựa trên phương pháp nghiên cứu lý thuyết kết hợp với thiết kế và kiểm chứng ở mức RTL. Trước tiên, tiến hành nghiên cứu kiến trúc RISC-V RV32I, mô hình pipeline 5 tầng, các loại hazard và cơ chế dự đoán nhánh động sử dụng bộ đếm bão hòa 2-bit kết hợp Branch Target Buffer.

Trên cơ sở đó, bộ xử lý RV32I pipeline 5 tầng được thiết kế bằng Verilog/SystemVerilog, bao gồm các khối chức năng chính như Program Counter, Register File, ALU, Control Unit, Forwarding Unit và Hazard Detection Unit. Tiếp theo, cơ chế dự đoán nhánh được tích hợp vào pipeline cùng cơ chế flush khi xảy ra dự đoán sai.

Hệ thống được kiểm chứng thông qua mô phỏng bằng QuestaSim với các chương trình kiểm thử chứa nhiều lệnh nhánh. Cuối cùng, các chỉ số như CPI và độ chính xác dự đoán nhánh được thu thập và phân tích nhằm đánh giá hiệu quả của cơ chế dự đoán nhánh đối với hiệu năng bộ xử lý.

1.6 Bố cục đề tài

Nội dung của luận văn được chia thành 5 chương như sau:

Chương 1: Giới thiệu

Trình bày lý do chọn đề tài, tình hình nghiên cứu, mục tiêu đề tài, phạm vi thực hiện, đối tượng nghiên cứu, phương pháp nghiên cứu và bố cục của luận văn.

Chương 2: Cơ sở lý thuyết

Trình bày tổng quan về kiến trúc RISC-V, tập lệnh RV32I, kỹ thuật pipeline 5 tầng, các dạng hazard trong pipeline và cơ chế dự đoán nhánh sử dụng trong bộ xử lý.

Chương 3: Thiết kế hệ thống

Trình bày kiến trúc tổng thể của bộ xử lý RISC-V RV32I pipeline 5 tầng tích hợp cơ chế dự đoán nhánh. Đồng thời mô tả chi tiết các module chức năng như Program Counter, Instruction Memory, Register File, Data Memory, ALU, Control Unit, Pipeline Register, Hazard Detection Unit, Branch Target Buffer và Pattern History Table.

Chương 4: Kết quả và đánh giá

Trình bày kết quả mô phỏng dạng sóng của bộ xử lý pipeline 5 tầng và hệ thống có tích hợp cơ chế dự đoán nhánh. Đồng thời thực hiện đánh giá hiệu năng thông qua các chỉ số như CPI và branch prediction accuracy để so sánh hiệu quả hoạt động giữa các cấu hình hệ thống.

Chương 5: Kết luận và hướng phát triển

Tổng kết các kết quả đạt được của đề tài, đánh giá những ưu điểm và hạn chế của hệ thống, đồng thời đề xuất các hướng phát triển và cải tiến trong tương lai.\

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Giới thiệu về kiến trúc, tập lệnh RISC-V

2.1.1 Giới thiệu RISC-V

Các bộ xử lý RISC-V hiện đại thường áp dụng kiến trúc pipeline để cải thiện throughput của hệ thống. Trong pipeline 5 tầng gồm IF (Instruction Fetch), ID (Instruction Decode), EX (Execute), MEM (Memory Access) và WB (Write Back), nhiều lệnh có thể được thực thi song song tại các giai đoạn khác nhau nhằm tăng hiệu năng xử lý [11, 14]. Tuy nhiên, việc thực thi song song này cũng làm phát sinh các hazard như data hazard và control hazard, đặc biệt đối với các lệnh branch [1, 11].

Để giảm ảnh hưởng của control hazard, nhiều nghiên cứu đã đề xuất sử dụng các cơ chế dự đoán nhánh động (Dynamic Branch Prediction) nhằm dự đoán hướng thực thi của các lệnh rẽ nhánh trước khi kết quả thực tế được xác định [7], [12]. Thay vì chờ đến giai đoạn thực thi để xác định nhánh có được thực hiện hay không, bộ dự đoán nhánh cho phép bộ xử lý đưa ra quyết định sớm về địa chỉ lệnh kế tiếp, từ đó giảm số chu kỳ stall và hạn chế việc flush pipeline khi gặp các lệnh branch [7, 3].

Trong các bộ xử lý RISC-V pipeline, một phương pháp phổ biến là sử dụng Branch Target Buffer (BTB) kết hợp với Pattern History Table (PHT) [12]. BTB có nhiệm vụ lưu trữ địa chỉ đích của các lệnh rẽ nhánh đã được thực thi trước đó, cho phép bộ xử lý nhanh chóng xác định địa chỉ cần truy cập khi gặp lại cùng một lệnh branch [12]. Trong khi đó, PHT sử dụng các bộ đếm bão hòa 2-bit để lưu trữ lịch sử thực thi của nhánh và dự đoán hướng thực thi trong những lần xuất hiện tiếp theo [12].

Nhờ sự kết hợp giữa BTB và PHT, bộ xử lý có thể dự đoán đồng thời cả địa chỉ đích và hướng thực thi của lệnh rẽ nhánh. Điều này giúp giảm số lần pipeline phải dừng hoặc xóa các lệnh đã nạp sai, từ đó cải thiện CPI và tăng thông lượng xử lý của hệ thống [7, 12, 13].

Bên cạnh các phương pháp dự đoán nhánh cơ bản, nhiều nghiên cứu hiện đại cũng đã đề xuất các kiến trúc predictor tiên tiến nhằm nâng cao độ chính xác dự đoán. Một số kỹ thuật tiêu biểu bao gồm:

- LXOR Predictor [3]
- TAGE-SC-L Predictor [5]
- Multiperspective Perceptron Predictor [4]
- CNN-based Branch Predictor [6]

Các phương pháp này sử dụng nhiều nguồn thông tin lịch sử khác nhau hoặc áp dụng các kỹ thuật học máy nhằm nâng cao khả năng dự đoán nhánh, đặc biệt trong các bộ xử lý hiệu năng cao [3, 6, 9]. Tuy nhiên, các kiến trúc predictor tiên tiến thường yêu cầu tài nguyên phần cứng lớn và độ phức tạp thiết kế cao hơn so với các bộ dự đoán dựa trên BTB và PHT.

Trong phạm vi đề tài này, cơ chế dự đoán nhánh động được xây dựng dựa trên sự kết hợp giữa Branch Target Buffer (BTB) và Pattern History Table (PHT) sử dụng bộ đếm bão hòa 2-bit. Giải pháp này đảm bảo cân bằng giữa độ chính xác dự đoán, mức sử dụng tài nguyên phần cứng và khả năng triển khai trên FPGA, đồng thời phù hợp với mục tiêu xây dựng bộ xử lý RV32I pipeline 5 tầng ở mức RTL.

2.1.2 Ưu và nhược điểm của RISC-V

RISC-V là một kiến trúc tập lệnh mở (Open Instruction Set Architecture – ISA) được phát triển nhằm hướng tới các hệ thống xử lý hiện đại với khả năng mở rộng linh hoạt và cấu trúc đơn giản [1]. Nhờ các đặc điểm này, RISC-V hiện đang được sử dụng rộng rãi trong lĩnh vực hệ thống nhúng, vi xử lý và nghiên cứu học thuật [1, 11].

Một trong những ưu điểm nổi bật nhất của RISC-V là tính mã nguồn mở. Không giống như các kiến trúc thương mại như ARM hoặc x86, RISC-V cho phép các cá nhân và tổ chức tự do nghiên cứu, phát triển và thương mại hóa mà không cần trả chi phí bản quyền [1]. Điều này giúp giảm đáng kể chi phí thiết kế phần

cứng cũng như tạo điều kiện thuận lợi cho việc phát triển các bộ xử lý tùy chỉnh theo từng ứng dụng cụ thể [1].

Bên cạnh đó, RISC-V được xây dựng theo triết lý RISC với tập lệnh đơn giản, số lượng lệnh vừa phải và độ dài lệnh cố định [1]. Nhờ đó, quá trình giải mã lệnh và thiết kế phần cứng trở nên đơn giản hơn so với các kiến trúc phức tạp khác. Điều này giúp giảm diện tích chip, giảm mức tiêu thụ công suất và tăng khả năng tối ưu hiệu năng của hệ thống [1].

ngoài ra, RISC-V có tính mô-đun cao, cho phép bổ sung các phần mở rộng như phép nhân/chia, xử lý số thực hoặc xử lý vectơ tùy theo yêu cầu hệ thống [1]. Nhờ đó, RISC-V được ứng dụng rộng rãi trong các thiết bị IoT, hệ thống nhúng và các bộ xử lý hiệu năng cao [1, 11].

Một ưu điểm quan trọng khác của RISC-V là khả năng hỗ trợ tốt cho nghiên cứu và giáo dục. Do kiến trúc được công khai hoàn toàn nên sinh viên và nhà nghiên cứu có thể dễ dàng tiếp cận, chỉnh sửa và phát triển các bộ xử lý dựa trên RISC-V mà không gặp rào cản về bản quyền hoặc tài liệu kỹ thuật [1].

Tuy nhiên, bên cạnh các ưu điểm trên, RISC-V vẫn tồn tại một số hạn chế nhất định. Do là kiến trúc còn tương đối mới nên hệ sinh thái phần mềm và công cụ hỗ trợ chưa hoàn thiện bằng ARM hoặc x86 [1]. Một số trình biên dịch, gỡ lỗi và thư viện tối ưu cho RISC-V vẫn đang tiếp tục được phát triển [11].

Ngoài ra, số lượng sản phẩm thương mại sử dụng RISC-V hiện nay vẫn còn hạn chế hơn so với các kiến trúc truyền thống. Điều này khiến khả năng tương thích phần mềm và mức độ phổ biến của RISC-V chưa thực sự đồng đều trên thị trường [11].

Bên cạnh những ưu điểm nổi bật, RISC-V cũng tồn tại hạn chế về tính tương thích. Việc cho phép bổ sung các phần mở rộng tùy chỉnh giúp tăng tính linh hoạt, nhưng đồng thời có thể gây khó khăn trong quá trình phát triển phần mềm nếu các phần mở rộng đó không được chuẩn hóa [1].

Tóm lại, RISC-V là một kiến trúc tập lệnh hiện đại với nhiều ưu điểm nổi bật về tính mở, khả năng mở rộng và thiết kế phần cứng đơn giản. Mặc dù vẫn còn một số hạn chế liên quan đến hệ sinh thái và mức độ phổ biến, RISC-V vẫn được

đánh giá là một trong những hướng phát triển tiềm năng của ngành công nghiệp bán dẫn hiện nay [1, 11].

2.1.3 Tập lệnh RV32I

RV32I là tập lệnh cơ sở 32-bit của kiến trúc RISC-V, được thiết kế nhằm cung cấp đầy đủ các chức năng cần thiết cho một bộ xử lý tổng quát nhưng vẫn đảm bảo tính đơn giản của kiến trúc RISC [1]. Đây là nền tảng cho nhiều bộ xử lý RISC-V hiện nay, đặc biệt trong các hệ thống nhúng và nghiên cứu học thuật.

Trong RV32I, mỗi thanh ghi có độ rộng 32-bit và không gian địa chỉ cũng sử dụng 32-bit. Tập lệnh này bao gồm nhiều nhóm lệnh cơ bản như lệnh số học, logic, truy cập bộ nhớ, lệnh nhảy và lệnh rẽ nhánh. Kiến trúc RV32I sử dụng 32 thanh ghi đa dụng từ x0 đến x31 [1]. Trong đó, thanh ghi x0 luôn có giá trị bằng 0 nhằm đơn giản hóa phần cứng và giảm số lượng lệnh cần thiết trong quá trình xử lý.

RV32I sử dụng mô hình nạp/lưu dữ liệu (Load/Store), trong đó việc truy cập bộ nhớ chỉ được thực hiện thông qua các lệnh nạp và lưu dữ liệu, còn các phép toán được thực hiện trên thanh ghi [1]. Nhờ đó, cấu trúc phần cứng được đơn giản hóa và hiệu năng xử lý được cải thiện [1].

Các lệnh số học và logic trong RV32I được sử dụng để thực hiện các phép toán cơ bản giữa các thanh ghi. Một số lệnh phổ biến gồm ADD, SUB, AND, OR, XOR, SLL, SRL, SRA và SLT [1].

Nhờ cấu trúc đơn giản, số lượng lệnh vừa phải và dễ triển khai trên phần cứng, RV32I hiện đang được sử dụng phổ biến trong các bộ xử lý RISC-V hiện đại [11]. Ngoài ra, nhiều nghiên cứu cũng sử dụng RV32I làm nền tảng để phát triển và tối ưu các hệ thống xử lý nhúng cũng như các bộ xử lý hiệu năng cao [13].

2.1.3.1 Tập thanh ghi

Kiến trúc RV32I của RISC-V sử dụng 32 thanh ghi đa dụng được đánh số từ x0 đến x31, mỗi thanh ghi có độ rộng 32-bit [1]. Các thanh ghi này được sử dụng để lưu trữ dữ liệu, toán hạng và kết quả trong quá trình thực thi chương trình.

Trong đó, thanh ghi x0 là thanh ghi đặc biệt luôn giữ giá trị bằng 0 và không thể ghi dữ liệu [1]. Thanh ghi x1 (ra) được sử dụng để lưu địa chỉ trả về khi thực hiện lời gọi hàm, còn thanh ghi x2 (sp) đóng vai trò con trỏ ngăn xếp phục vụ quản lý vùng stack của chương trình [1].

Ngoài ra, RV32I còn định nghĩa các nhóm thanh ghi tạm thời (t0–t6), thanh ghi lưu trữ (s0–s11) và thanh ghi tham số (a0–a7) nhằm hỗ trợ việc truyền dữ liệu và thực hiện lời gọi hàm một cách hiệu quả [1].

Bảng 2.1 Chức năng các thanh ghi của RV32I

Thanh ghi	Tên ABI	Chức năng
x0	zero	Luôn mang giá trị 0
x1	ra	Lưu địa chỉ trả về
x2	sp	Stack Pointer
x3	gp	Global Pointer
x4	tp	Thread Pointer
x5–x7	t0–t2	Thanh ghi tạm thời
x8–x9	s0–s1	Thanh ghi lưu dữ liệu
x10–x17	a0–a7	Thanh ghi truyền tham số
x18–x27	s2–s11	Thanh ghi lưu dữ liệu
x28–x31	t3–t6	Thanh ghi tạm thời

2.1.3.2 Định dạng lệnh

Trong kiến trúc tập lệnh cơ sở RV32I, tất cả các cấu trúc mã máy đều có độ dài cố định là 32-bit [1]. Đây là một đặc điểm cơ bản trong triết lý thiết kế RISC, khi mỗi lệnh chiếm dụng đúng một từ (word) bộ nhớ [1]. Quy chuẩn này cho phép bộ vi xử lý xác định chính xác ranh giới bắt đầu và kết thúc của từng dòng lệnh

một cách tuyến tính mà không cần đến các mạch logic nhận dạng độ dài phức tạp [11]. Nhờ đó, quá trình giải mã tại tầng ID được tinh giản tối đa, giảm thiểu đáng kể diện tích tài nguyên phần cứng và tạo điều kiện tối ưu để xây dựng các tầng pipeline hiệu năng cao [11, 13].

Bảng 2.2 Tổng quan định dạng lệnh trong RV32I

Định dạng	Ứng dụng chính
R-type	Lệnh số học và logic giữa hai thanh ghi: ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT
I-type	Lệnh immediate (ADDI, ANDI, ...), lệnh load (LW, LH, LB, ...) và lệnh nhảy gián tiếp JALR
S-type	Lệnh store (SW, SH, SB): ghi dữ liệu từ thanh ghi ra bộ nhớ
B-type	Lệnh rẽ nhánh có điều kiện: BEQ, BNE, BLT, BGE, BLTU, BGEU
U-type	Lệnh nạp immediate kích thước lớn: LUI, AUIPC
J-type	Lệnh nhảy không điều kiện có lưu địa chỉ trả về: JAL

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Hình 2.1 Định dạng lệnh RV32I

Trong kiến trúc RV32I, mỗi lệnh 32-bit được chia thành nhiều trường chức năng với vị trí bit được quy định cố định [1]. Các trường quan trọng như opcode, thanh ghi đích (rd) và các thanh ghi nguồn (rs1, rs2) luôn xuất hiện tại cùng một vị trí trong các định dạng lệnh khác nhau [1]. Nhờ đó, bộ xử lý có thể thực hiện quá trình giải mã lệnh và truy xuất thanh ghi một cách hiệu quả, giúp đơn giản hóa phần cứng và hỗ trợ tốt cho thiết kế pipeline [1, 11].

Việc duy trì vị trí cố định của các trường lệnh cũng góp phần giảm độ phức tạp của bộ điều khiển, đồng thời nâng cao hiệu quả thực thi của bộ xử lý [1, 13]. Các thành phần cấu trúc chính này bao gồm:

Bảng 2.3 Các trường của RV32I

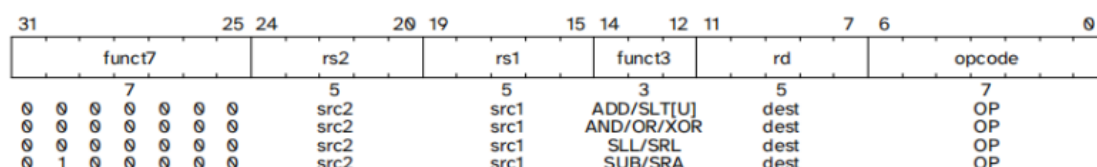
Trường	Mô tả
opcode	Chiếm 7 bit thấp nhất [6:0]. Xác định nhóm lệnh (load, store, arithmetic, branch, jump, ...). Một giá trị opcode có thể ứng với nhiều lệnh khác nhau, được phân biệt thêm bởi funct3 và funct7.
rd	Chiếm 5 bit [11:7]. Chỉ định thanh ghi đích (destination register) — nơi lưu kết quả sau khi thực thi lệnh. Có mặt trong các định dạng R, I, U, J.
funct3	Chiếm 3 bit [14:12]. Kết hợp với opcode để xác định chính xác phép toán cần thực hiện. Ví dụ: cùng opcode OP, funct3=000 là ADD/SUB, funct3=111 là AND.
rs1	Chiếm 5 bit [19:15]. Chỉ định thanh ghi nguồn thứ nhất (source register 1). Vị trí cố định này cho phép phân cứng đọc rs1 song song với việc giải mã opcode.
rs2	Chiếm 5 bit [24:20]. Chỉ định thanh ghi nguồn thứ hai (source register 2). Chỉ xuất hiện trong định dạng R, S và B.
funct7	Chiếm 7 bit [31:25]. Kết hợp với funct3 và opcode để phân biệt các lệnh có cùng nhóm. Ví dụ: ADD và SUB có cùng opcode và funct3 nhưng khác funct7 (0000000 và 0100000).
immediate	Giá trị hằng số hoặc offset được mã hóa trực tiếp vào lệnh. Độ rộng và vị trí bit của immediate thay đổi theo từng định dạng (12-bit cho I/S/B, 20-bit cho U/J). Luôn được sign-extend lên 32-bit trước khi sử dụng.

Định dạng R-type

Định dạng R-type được thiết kế dành riêng cho các phép toán số học và luận lý giữa hai toán hạng thanh ghi mà không sử dụng giá trị tức thời immediate [14].

Do đó, toàn bộ 32 bit của lệnh được phân chia hoàn toàn cho các trường định danh thanh ghi và mã thao tác điều khiển [14].

Khi thực thi, CPU sẽ nạp giá trị từ hai thanh ghi nguồn rs1 và rs2 để cung cấp đầu vào cho bộ ALU; tại đây, phép toán được định hình chính xác nhờ tổ hợp opcode + funct3 + funct7, và kết quả cuối cùng được ghi trả trực tiếp vào thanh ghi đích rd mà hoàn toàn bỏ qua giai đoạn truy xuất bộ nhớ [14].

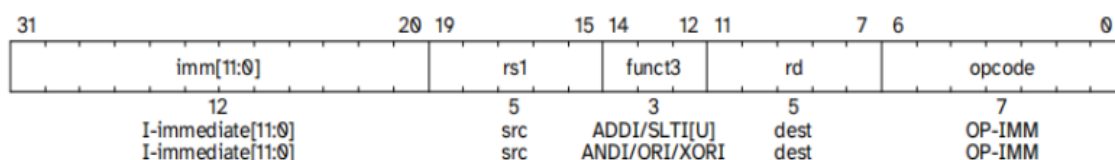


Hình 2.2 Định dạng lệnh R-type

Kiến trúc tập lệnh RV32I cung cấp nền tảng xử lý toàn diện cho các phép toán cơ bản bao gồm: nhóm toán tử số học và logic (ADD, SUB, AND, OR, XOR), nhóm lệnh so sánh đại số có dấu hoặc không dấu (SLT, SLTU), và các toán tử dịch chuyển logic hoặc số học (SLL, SRL, SRA). Đối với những thao tác tính toán tích hợp toán hạng tức thời, RISC-V quy chuẩn hóa việc sử dụng định dạng lệnh I-type nhằm duy trì tính nhất quán và tinh gọn cho cấu trúc phần cứng giải mã [1].

Định dạng I-type

Định dạng mã hóa chi tiết của cấu trúc lệnh I-type dành cho các phép toán đại số tức thời được minh họa cụ thể trong Hình 2.3, cấu hình này cho phép bóc tách một toán hạng trực tiếp từ trường immediate (imm) tích hợp sẵn trong cấu trúc dòng lệnh.



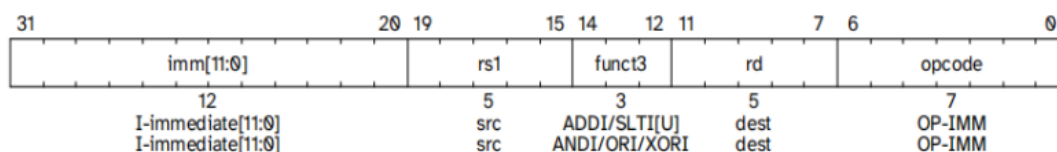
Hình 2.3 Định dạng lệnh I-type

Cấu trúc định dạng I-type được áp dụng đồng bộ cho ba lớp lệnh quan trọng trong kiến trúc RISC-V: các lệnh toán tử số học/logic immediate, lệnh load dữ liệu và lệnh nhảy gián tiếp JALR [1]. Trước khi khối ALU thực hiện tính toán, trường

immediate 12-bit đóng gói trong mã lệnh sẽ được mở rộng dấu thành 32-bit [1]. Cơ chế này cho phép các lệnh I-type làm việc trực tiếp với các giá trị hằng số thuộc khoảng $[-2048, +2047]$ một cách hiệu quả [1].

a) Nhóm lệnh số học và logic với immediate

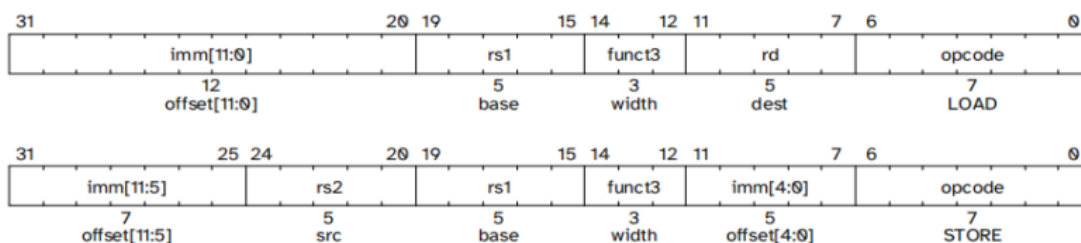
Nhóm lệnh này thực hiện các phép toán tương tự R-type nhưng thay toán hạng rs2 bằng giá trị hằng số immediate. CPU đọc giá trị từ rs1, sign-extend immediate lên 32-bit, thực hiện phép toán qua ALU và ghi kết quả vào rd.



Hình 2.4 Định dạng mã lệnh I-type cho các phép toán immediate trong RISC-V

b) Nhóm lệnh load

Nhóm lệnh load đọc dữ liệu từ bộ nhớ về thanh ghi. Địa chỉ bộ nhớ được tính bằng cách cộng rs1 với immediate (sign-extended). Dữ liệu đọc về được mở rộng lên 32-bit trước khi lưu vào rd — sign-extend cho lệnh có dấu (LW, LH, LB) hoặc zero-extend cho lệnh không dấu (LHU, LBU).



Hình 2.5 Định dạng mã lệnh Load và Store trong RISC-V

c) Lệnh nhảy gián tiếp JALR

Lệnh JALR nhảy đến địa chỉ được tính từ thanh ghi cộng với immediate, đồng thời lưu địa chỉ lệnh tiếp theo (PC+4) vào thanh ghi rd. Bit 0 của địa chỉ đích luôn bị xóa về 0 để đảm bảo căn chỉnh lệnh. JALR thường được dùng để trả về từ hàm (ret: JALR x0, ra, 0) hoặc gọi hàm qua con trỏ hàm.

Định dạng S-type

Định dạng S-type được sử dụng cho các lệnh ghi dữ liệu từ thanh ghi ra bộ nhớ. Đây là định dạng không có trường rd vì lệnh store không tạo ra kết quả ghi.

vào thanh ghi. Thay vào đó, vị trí rd được tận dụng để chứa 5 bit thấp của immediate, trong khi 7 bit cao của immediate nằm tại vị trí funct7 [1].

Bảng 2.4. Danh sách lệnh S-type

Lệnh	funct3	Hoạt động và mô tả
SW	010	Mem[rs1 + imm](32-bit) = rs2 — Ghi 4 byte (1 word) từ rs2 ra bộ nhớ. Địa chỉ phải căn chỉnh 4 byte
SH	001	Mem[rs1 + imm](16-bit) = rs2[15:0] — Ghi 2 byte thấp từ rs2 ra bộ nhớ. Địa chỉ căn chỉnh 2 byte
SB	000	Mem[rs1 + imm](8-bit) = rs2[7:0] — Ghi 1 byte thấp nhất từ rs2 ra bộ nhớ

Định dạng B-type

Định dạng B-type (Branch-type) được sử dụng cho tất cả các lệnh rẽ nhánh có điều kiện. CPU so sánh giá trị của hai thanh ghi rs1 và rs2; nếu điều kiện thỏa mãn, PC được cộng thêm giá trị offset (immediate) để nhảy đến địa chỉ đích. Nếu điều kiện không thỏa mãn, CPU tiếp tục thực hiện lệnh kế tiếp (PC + 4).

Bảng 2.5. Danh sách lệnh B-type

Lệnh	funct3	Hoạt động và mô tả
BEQ	000	Nhảy nếu rs1 == rs2 (Branch if Equal). Dừng kiểm tra bằng nhau
BNE	001	Nhảy nếu rs1 != rs2 (Branch if Not Equal). Dừng trong vòng lặp do-while
BLT	100	Nhảy nếu rs1 < rs2 (signed). Branch if Less Than — so sánh có dấu

Lệnh	funct3	Hoạt động và mô tả
BGE	101	Nhảy nếu $rs1 \geq rs2$ (signed). Branch if Greater or Equal — so sánh có dấu
BLTU	110	Nhảy nếu $rs1 < rs2$ (unsigned). Branch if Less Than Unsigned
BGEU	111	Nhảy nếu $rs1 \geq rs2$ (unsigned). Branch if Greater or Equal Unsigned

Định dạng U-type

Định dạng U-type (Upper immediate-type) được sử dụng cho các lệnh cần nạp giá trị immediate kích thước lớn. Trường immediate 20-bit được đặt vào 20 bit cao [31:12] của kết quả, trong khi 12 bit thấp được xử lý tùy theo lệnh. Định dạng này chỉ gồm hai lệnh: LUI và AUIPC.

Bảng 2.6. Danh sách lệnh U-type

Lệnh	opcode	Hoạt động và mô tả
LUI	0110111	$rd = imm \ll 12$ — Load Upper Immediate: nạp immediate vào 20 bit cao của rd, xóa 12 bit thấp về 0. Dùng để xây dựng hằng số 32-bit kết hợp với ADDI
AUIPC	0010111	$rd = PC + (imm \ll 12)$ — Add Upper Immediate to PC: cộng immediate (dịch trái 12 bit) với giá trị PC hiện tại và lưu vào rd. Dùng để tính địa chỉ vị trí độc lập (PIC — Position Independent Code)

Định dạng J-type

Định dạng J-type (Jump-type) được sử dụng riêng cho lệnh JAL (Jump And Link). Đây là lệnh nhảy không điều kiện có khoảng nhảy lớn hơn B-type nhờ sử dụng immediate 21-bit ($imm[20:1]$, không có bit 0). Khoảng nhảy hợp lệ là ± 1 MB

so với PC hiện tại. Tương tự B-type, bit 0 của offset không cần mã hóa vì địa chỉ đích luôn căn chỉnh 2 byte.

Lệnh JAL hoạt động theo hai bước đồng thời: ghi địa chỉ lệnh tiếp theo ($PC + 4$) vào thanh ghi rd (để dùng làm địa chỉ trả về); và cộng immediate (sign-extended) vào PC hiện tại để nhảy đến địa chỉ đích.

Bảng 2.7. Lệnh J-type và ứng dụng

Cú pháp	Ứng dụng	Mô tả hoạt động
JAL rd, offset	Gọi hàm	$rd = PC + 4$; $PC = PC + \text{sign_ext}(\text{offset})$. rd lưu địa chỉ trả về. Dùng JAL x1, func để gọi hàm func
JAL x0, offset	Nhảy	Vì x0 luôn bằng 0 nên địa chỉ trả về bị bỏ qua. Tương đương lệnh J (jump) trong các ISA khác

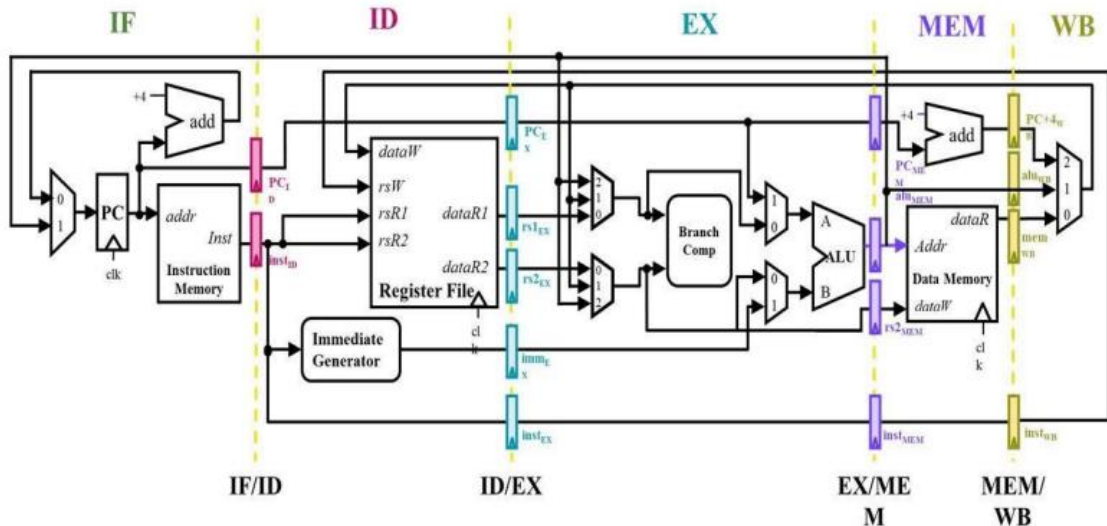
2.2 Kỹ thuật Pipeline

2.2.1 Giới thiệu

Kỹ thuật đường ống (pipeline) là một trong những phương pháp cốt lõi trong thiết kế kiến trúc máy tính hiện đại nhằm nâng cao hiệu suất và tối ưu hóa luồng xử lý của bộ vi xử lý [11, 13]. Thay vì thực hiện tuần tự từng lệnh một cách biệt lập, pipeline chia nhỏ quá trình xử lý của một lệnh thành nhiều giai đoạn logic độc lập có thời gian thực thi ngắn hơn [13]. Điều này cho phép vi xử lý thực thi đồng thời nhiều cấu phần của các lệnh khác nhau trong cùng một chu kỳ xung nhịp [13], từ đó tăng đáng kể thông lượng lệnh tổng thể của hệ thống.

2.2.2 Hoạt động của pipeline 5 tầng

Trong kiến trúc RISC-V, đường ống 5 tầng (IF, ID, EX, MEM và WB) được xem là mô hình xử lý lệnh chuẩn mực nhờ khả năng đơn giản hóa cấu trúc phần cứng và tối ưu hóa tính toán song song. Cấu trúc kinh điển này không chỉ áp dụng rộng rãi trên các lõi RV32I mà còn đóng vai trò là nền tảng cốt lõi cho các giải pháp cải tiến hiệu năng thế hệ mới [13, 14].



Hình 2.6 Sơ đồ đường ống 5 giai đoạn

Nhận lệnh (IF): Tầng IF truy xuất mã lệnh từ bộ nhớ chương trình theo bộ đếm chương trình và tính toán địa chỉ kế tiếp [1]. IF hoạt động liên tục cung cấp dòng lệnh ổn định cho pipeline và phải hỗ trợ cập nhật PC khi xảy ra nhánh hoặc jump [1].

Giải mã (ID): Tầng ID phân tích opcode để xác định loại lệnh, đọc các thanh ghi nguồn, mở rộng giá trị hằng số và sinh tín hiệu điều khiển [1]. Khối logic tại đây cũng phát hiện xung đột và chèn chu kỳ treo (stall) nếu xảy ra hiện tượng *load-use* nhằm bảo toàn dữ liệu [11].

Thực thi (EX): Sử dụng bộ tính toán số học - logic (ALU) làm trung tâm để thực hiện các phép toán, tính địa chỉ bộ nhớ và phân giải điều kiện rẽ nhánh [1, 14]. Để hạn chế tối đa việc stall do xung đột dữ liệu, cơ chế chuyển tiếp (data forwarding) được áp dụng để truyền ngược kết quả từ tầng MEM/WB về đầu vào ALU [14]. Việc dự đoán nhánh sai tại EX sẽ chịu mức phạt 1–2 chu kỳ để xóa luồng lệnh lỗi [2, 13].

Truy xuất bộ nhớ (MEM): Tầng này chỉ phục vụ các lệnh Load/Store để đọc/ghi dữ liệu với bộ nhớ [1]; các lệnh khác chỉ đi qua mà không thực hiện thao tác [14]. Việc tách riêng tầng MEM trong kiến trúc Harvard hoặc hệ thống có cache giúp giảm tranh chấp tài nguyên và tối ưu băng thông [1, 8].

Ghi lại (WB): Giai đoạn hoàn tất vòng đời của lệnh, thực hiện ghi kết quả từ ALU hoặc dữ liệu từ bộ nhớ vào thanh ghi đích [1, 14]. Tập thanh ghi thường hỗ trợ cơ chế đọc/ghi trong cùng chu kỳ để tránh xung đột với tầng ID [1].

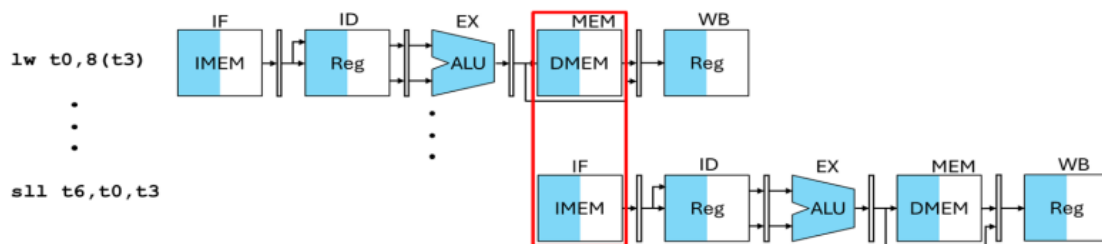
2.2.3 Xung đột trong pipeline

Kỹ thuật đường ống phân chia chu trình xử lý lệnh thành các giai đoạn tuần tự (IF–ID–EX–MEM–WB) để tăng thông lượng hệ thống [1, 14]. Tuy nhiên, cơ chế này cũng phát sinh các hiện tượng xung đột (pipeline hazards) làm gián đoạn luồng thực thi song song [11, 14]. Trong kiến trúc vi xử lý, hazards được phân thành ba loại chính:

- **Xung đột cấu trúc (Structural Hazards):** Xảy ra khi nhiều lệnh cùng tranh chấp một tài nguyên phần cứng (bộ nhớ, tập thanh ghi) trong một chu kỳ [1].
- **Xung đột dữ liệu (Data Hazards):** Xuất hiện khi một lệnh phụ thuộc vào kết quả chưa được ghi lại của lệnh phía trước; thường được xử lý bằng cơ chế chuyển tiếp dữ liệu (data forwarding) hoặc chèn chu kỳ treo (stall) [1, 14].
- **Xung đột điều khiển (Control Hazards):** Phát sinh do sự chậm trễ khi xác định địa chỉ nhảy của các lệnh rẽ nhánh (Branch/Jump) tại tầng IF [1, 14]. Hệ thống thường tích hợp bộ dự đoán nhánh (BPU) hoặc áp dụng khe trễ (delay slots) để giảm thiểu tổn thất chu kỳ [2, 13, 4].

a) Xung đột cấu trúc (Structural Hazards)

Xung đột cấu trúc xảy ra khi nhiều giai đoạn trong đường ống (pipeline) cùng tranh chấp một tài nguyên phần cứng trong một chu kỳ clock [1]. Hệ thống dùng chung một bộ nhớ RAM vật lý cho cả bộ nhớ lệnh và bộ nhớ dữ liệu [1, 14]. Khi lệnh lw truy xuất dữ liệu ở giai đoạn MEM, cùng lúc tầng IF cần lấy lệnh kế tiếp từ bộ nhớ [1]. Do RAM không thể đáp ứng đồng thời hai yêu cầu độc lập, pipeline phải chèn một chu kỳ treo (stall) để trì hoãn tầng IF, gây sụt giảm hiệu năng [1, 14].

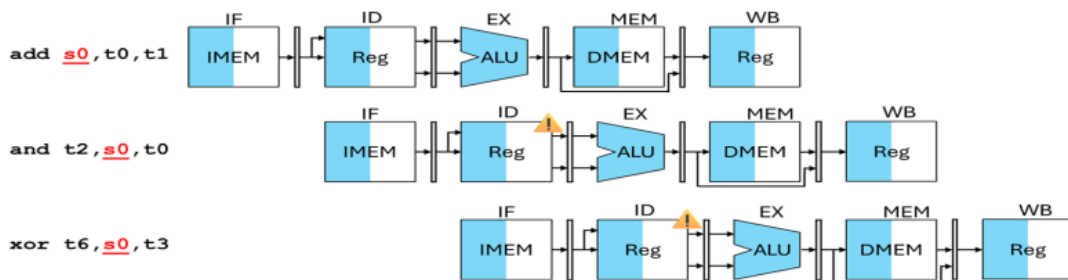


Hình 2.7 Xung đột cấu trúc khi dùng chung bộ nhớ

b) Xung đột dữ liệu (Data Hazards)

Xung đột dữ liệu xảy ra khi một lệnh cần sử dụng kết quả của lệnh phía trước nhưng toán hạng đó chưa kịp ghi vào tập thanh ghi [1]. Trong kiến trúc 5 tầng, việc kết quả chỉ sẵn sàng từ giai đoạn EX/MEM tạo ra khoảng trễ nếu lệnh kế tiếp muốn truy cập sớm dữ liệu này tại tầng ID [11, 14].

Hình 2. minh họa xung đột dữ liệu trong pipeline 5-stage khi lệnh sau sử dụng toán hạng được tạo ra bởi lệnh trước.

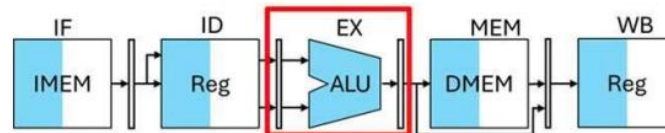


Hình 2.8 Minh họa xung đột dữ liệu

c) Xung đột điều khiển (Control hazard)

Xung đột điều khiển xuất hiện khi pipeline gặp các lệnh có khả năng thay đổi giá trị PC, chẳng hạn như các lệnh nhánh có điều kiện (B-type), lệnh nhảy JAL hoặc JALR. Trong khi pipeline vẫn tiếp tục lấy lệnh theo hướng hiện tại, kết quả điều kiện của nhánh chỉ được xác định ở giai đoạn EX. Nếu dự đoán sai hướng nhánh, các lệnh đã được nạp trước đó sẽ phải bị loại bỏ (flush), gây mất chu kỳ thực thi [3].

Hình 2.9 minh họa thời điểm hướng nhánh được xác định trong pipeline 5-stage, dẫn đến nguy cơ phát sinh control hazard.



Hình 2.9 Địa chỉ PC tại EX

2.3 Cơ chế dự đoán nhánh

Trong bộ xử lý pipeline, các lệnh rẽ nhánh (branch instruction) là nguyên nhân chính gây ra control hazard. Khi gặp một lệnh branch, bộ xử lý cần xác định hướng thực thi tiếp theo của chương trình, bao gồm việc nhánh có được thực hiện hay không và địa chỉ lệnh kế tiếp là gì. Tuy nhiên, kết quả này thường chỉ được xác định ở các giai đoạn sau của pipeline, khiến bộ xử lý phải chờ hoặc hủy các lệnh đã nạp trước đó nếu dự đoán sai [1, 12].

Để giảm ảnh hưởng của control hazard, các bộ xử lý hiện đại sử dụng cơ chế dự đoán nhánh (Branch Prediction). Mục tiêu của kỹ thuật này là dự đoán trước hướng thực thi của lệnh nhánh để bộ xử lý tiếp tục nạp và thực thi các lệnh kế tiếp mà không cần chờ kết quả thực tế [1]. Nếu dự đoán chính xác, số lần stall và flush pipeline sẽ giảm đáng kể, từ đó cải thiện hiệu năng và giảm giá trị CPI của hệ thống [7, 13].

Tùy theo phương pháp dự đoán, cơ chế dự đoán nhánh thường được chia thành hai nhóm chính: dự đoán nhánh tĩnh và dự đoán nhánh động [12].

2.3.1 Cơ chế dự đoán nhánh tĩnh

Dự đoán nhánh tĩnh là phương pháp đưa ra quyết định dự đoán dựa trên các quy tắc cố định và không sử dụng thông tin lịch sử thực thi của chương trình [1]. Do không yêu cầu phần cứng lưu trữ trạng thái hay lịch sử nhánh, phương pháp này có cấu trúc đơn giản và chi phí triển khai thấp.

Một số kỹ thuật dự đoán nhánh tĩnh phổ biến bao gồm:

- **Always Not Taken:** giả định rằng mọi lệnh branch đều không được thực hiện.
- **Always Taken:** giả định rằng mọi lệnh branch đều được thực hiện.
- **Backward Taken – Forward Not Taken:** các nhánh lùi thường được dự đoán là Taken, trong khi các nhánh tiến được dự đoán là Not Taken [1].

Ưu điểm của dự đoán nhánh tĩnh là cấu trúc đơn giản, không yêu cầu bộ nhớ lưu lịch sử và có thể áp dụng cho các hệ thống có tài nguyên phần cứng hạn chế. Tuy nhiên, do không thích nghi với hành vi thực tế của chương trình nên độ chính xác dự đoán thường không cao, đặc biệt đối với các ứng dụng có luồng điều khiển phức tạp [12].

Vì những hạn chế trên, các bộ xử lý hiện đại thường sử dụng cơ chế dự đoán nhánh động nhằm nâng cao độ chính xác và cải thiện hiệu năng hệ thống.

2.3.2 Cơ chế dự đoán nhánh động

Khác với dự đoán nhánh tĩnh, dự đoán nhánh động sử dụng thông tin lịch sử thực thi của các lệnh branch để đưa ra quyết định dự đoán cho những lần xuất hiện tiếp theo [12]. Thông qua việc lưu trữ và cập nhật trạng thái của các nhánh đã thực thi, bộ dự đoán có khả năng thích nghi với đặc điểm của chương trình, từ đó đạt độ chính xác cao hơn đáng kể so với các phương pháp dự đoán tĩnh [7].

Một trong những phương pháp phổ biến nhất là sử dụng bộ đếm bão hòa 2-bit (2-bit Saturating Counter). Mỗi lệnh branch được gán một trạng thái dự đoán và trạng thái này sẽ được cập nhật dựa trên kết quả thực tế của lần thực thi gần nhất. Nhờ cơ chế này, bộ dự đoán có khả năng chống lại các thay đổi ngắn hạn và duy trì độ ổn định trong quá trình dự đoán [12].

Trong các bộ xử lý hiện đại, cơ chế dự đoán nhánh động thường được xây dựng dựa trên sự kết hợp của các thành phần như Branch Target Buffer (BTB) và Pattern History Table (PHT). BTB được sử dụng để lưu địa chỉ đích của các lệnh branch, trong khi PHT lưu trữ lịch sử thực thi và trạng thái dự đoán của từng nhánh [12]. Sự kết hợp này giúp bộ xử lý dự đoán đồng thời cả hướng thực thi và địa chỉ đích của lệnh branch, từ đó giảm số lần flush pipeline và cải thiện hiệu năng xử lý [7, 13].

Trong phạm vi đề tài này, cơ chế dự đoán nhánh động được xây dựng dựa trên Branch Target Buffer (BTB) kết hợp với Pattern History Table (PHT) sử dụng bộ đếm bão hòa 2-bit. Kiến trúc này cho phép đạt được độ chính xác dự đoán tương đối cao trong khi vẫn đảm bảo mức sử dụng tài nguyên phần cứng phù hợp với bộ xử lý RV32I pipeline 5 tầng

Chương 3. THIẾT KẾ HỆ THỐNG

3.1 Tổng quan về hệ thống

Đề tài xây dựng bộ xử lý RISC-V 32-bit dựa trên tập lệnh RV32I với kiến trúc pipeline 5 tầng gồm: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM) và Write Back (WB). Kiến trúc này cho phép nhiều lệnh được xử lý đồng thời, góp phần nâng cao hiệu năng của hệ thống.

Bộ xử lý bao gồm các khối chức năng chính như Program Counter, Instruction Memory, Register File, ALU và Control Logic. Để đảm bảo hoạt động chính xác của pipeline, hệ thống tích hợp cơ chế xử lý hazard dữ liệu bằng forwarding kết hợp stall một chu kỳ cho trường hợp Load-Use.

Điểm nổi bật của đề tài là việc tích hợp cơ chế dự đoán nhánh động tại tầng IF sử dụng Branch Target Buffer (BTB) và Pattern History Table (PHT). Cơ chế này cho phép dự đoán hướng thực thi của lệnh rẽ nhánh từ sớm, giúp giảm số lần flush pipeline và cải thiện hiệu năng tổng thể của bộ xử lý.

3.1.1 Yêu cầu của hệ thống

Hệ thống cần thỏa mãn các yêu cầu kỹ thuật sau:

- Thực thi đầy đủ tập lệnh RV32I cơ bản: R-type, I-type (ALU, LOAD, JALR), S-type, B-type, U-type (LUI, AUIPC) và J-type (JAL).
- Tổ chức pipeline 5 tầng: IF (Instruction Fetch), ID (Instruction Decode), EX (Execute), MEM (Memory Access), WB (Write Back).
- Xử lý đúng các hazard dữ liệu loại Load-Use bằng cơ chế stall 1 chu kỳ.
- Xử lý hazard dữ liệu tổng quát bằng forwarding từ tầng EX/MEM và MEM/WB về tầng EX.
- Tích hợp dự đoán nhánh động với BTB (Branch Target Buffer) 256 entry và PHT (Pattern History Table) 256 entry sử dụng bộ đếm bão hoà 2-bit.
- Có khả năng phát hiện và phục hồi khi dự đoán sai (mispredict) bằng cách flush pipeline và nạp lại PC đúng từ tầng EX.
- Bộ nhớ lệnh (IMEM) và bộ nhớ dữ liệu (DMEM) được thiết kế độc lập, hỗ trợ khởi tạo từ file .hex.

3.1.2 Mục tiêu của hệ thống

Mục tiêu tổng thể: Xây dựng bộ xử lý RISC-V pipeline vận hành thực tế, đạt hiệu năng tối ưu nhờ tích hợp cơ chế dự đoán nhánh động.

Các mục tiêu cụ thể:

- Tính đúng đắn tuyệt đối: Thực thi chính xác mọi lệnh RV32I, xử lý tốt các trường hợp biên như ghi vào x0, tự phụ thuộc dữ liệu, chuỗi Load-Use liên tiếp, branch not-taken và tính địa chỉ nhảy từ JALR.
- Tối ưu hóa CPI: Sử dụng cơ chế BTB + PHT để dự đoán đúng phần lớn các lệnh rẽ nhánh (đặc biệt là backward branch trong vòng lặp), giảm thiểu tối đa hình phạt flush 2 chu kỳ hoang phí của pipeline 5 tầng.
- Mô-đun hóa và mở rộng: Kiến trúc phân khối linh hoạt, cho phép dễ dàng nâng cấp hoặc thay thế các thuật toán dự đoán phức tạp hơn (gshare, tournament predictor) mà không ảnh hưởng đến cấu trúc datapath chung.

3.2 RISC-V 5 tầng tích hợp cơ chế dự đoán nhánh

Kiến trúc tổng thể của bộ xử lý được tổ chức theo mô hình pipeline 5 tầng kinh điển, trong đó mỗi tầng thực hiện một giai đoạn xử lý lệnh độc lập và hoạt động song song với các tầng còn lại. Điểm khác biệt so với pipeline cơ bản là sự tích hợp của khối dự đoán nhánh động (BTB + PHT) tại tầng IF, cho phép hệ thống đưa ra quyết định rẽ nhánh trước khi lệnh được giải mã.

3.2.1 Hạn chế của kiến trúc RISC-V Pipeline truyền thống

Kiến trúc RV32I pipeline 5 tầng cho phép nhiều lệnh được xử lý đồng thời, giúp tăng thông lượng và giảm CPI. Tuy nhiên, hiệu năng thực tế vẫn bị ảnh hưởng bởi các hazard, đặc biệt là hazard điều khiển do các lệnh branch và jump gây ra.

Trong thiết kế pipeline truyền thống, kết quả của lệnh branch chỉ được xác định tại tầng EX. Trong thời gian chờ đánh giá điều kiện nhánh, tầng IF vẫn tiếp tục nạp các lệnh theo hướng tuần tự (PC+4). Nếu nhánh được thực hiện (taken), các lệnh đã nạp sai phải bị loại bỏ khỏi pipeline thông qua cơ chế flush.

Đối với pipeline 5 tầng, mỗi lần branch taken thường gây mất khoảng hai chu kỳ xử lý. Khi tần suất branch cao, số lần flush tăng lên đáng kể, làm giảm hiệu năng và tăng giá trị CPI của hệ thống.

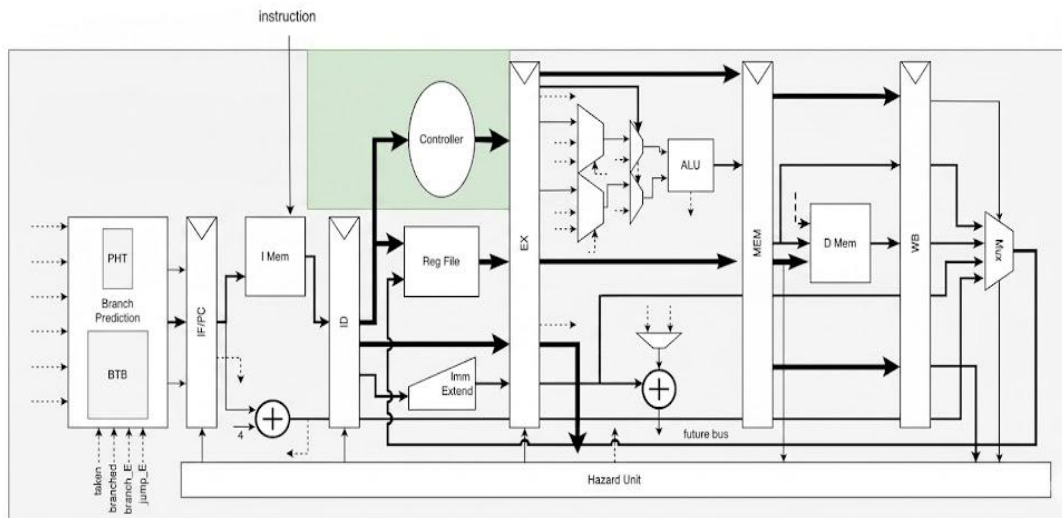
3.2.2 Kiến trúc RISC-V cải tiến với dự đoán nhánh động

Điểm cải tiến chính của hệ thống là việc bổ sung hai khối dự đoán nhánh gồm Branch Target Buffer (BTB) và Pattern History Table (PHT) tại tầng IF. BTB lưu địa chỉ đích của các lệnh branch, trong khi PHT lưu lịch sử thực thi taken/not-taken.

Dựa trên thông tin này, bộ xử lý có thể dự đoán hướng thực thi của nhánh ngay khi fetch lệnh mà không cần chờ đến tầng EX. Khi dự đoán đúng, pipeline tiếp tục hoạt động bình thường; chỉ khi xảy ra misprediction mới cần flush để khôi phục luồng thực thi chính xác.

Nhờ cơ chế dự đoán nhánh động, số chu kỳ mất do control hazard được giảm đáng kể, góp phần cải thiện hiệu năng của bộ xử lý.

3.2.3 Sơ đồ khối tổng thể



Hình 3.1: Sơ đồ tổng quan pipeline RISC-V 5 tầng với dự đoán nhánh [2]

Hình 3.1 mô tả kiến trúc tổng thể của bộ xử lý RISC-V pipeline 5 tầng gồm IF, ID, EX, MEM và WB.

Tại tầng IF, PC được sử dụng để đọc lệnh từ IMEM và tra cứu BTB/PHT nhằm dự đoán nhánh. Lệnh và giá trị PC sau đó được lưu vào thanh ghi IF/ID.

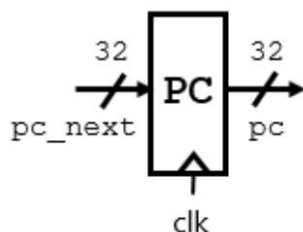
Tại tầng ID, lệnh được giải mã bởi Control Unit, các toán hạng được đọc từ Register File và giá trị immediate được tạo bởi ImmGen. Hazard Detection Unit đồng thời kiểm tra Load-Use hazard trước khi dữ liệu được chuyển sang ID/EX.

Tại tầng EX, ALU thực hiện các phép tính, Branch Unit đánh giá điều kiện nhánh và xác định kết quả thực tế của lệnh rẽ nhánh. Các kết quả xử lý được lưu vào EX/MEM.

Tại tầng MEM, bộ xử lý thực hiện các thao tác đọc hoặc ghi dữ liệu với Data Memory. Kết quả sau đó được chuyển tới thanh ghi MEM/WB.

Tại tầng WB, dữ liệu từ ALU, Data Memory hoặc PC+4 được lựa chọn để ghi trở lại Register File, hoàn tất quá trình thực thi lệnh.

3.3. Thiết kế chi tiết các khối RISC-V Pipeline



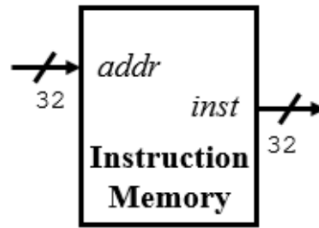
Hình 3.2: Sơ đồ Program Counter

Program Counter (PC) là thanh ghi 32-bit lưu địa chỉ lệnh hiện tại của bộ xử lý. Khi reset, PC được khởi tạo về địa chỉ 0x00000000, là điểm bắt đầu thực thi chương trình.

Từ giá trị PC hiện tại, bộ cộng tạo ra $pc_plus4 = PC + 4$, tương ứng với địa chỉ lệnh kế tiếp trong luồng thực thi tuần tự. Giá trị này được sử dụng để cập nhật PC và làm địa chỉ trả về cho các lệnh JAL/JALR.

Bảng 3.1: Tín hiệu của module Program counter

Tín hiệu	Hướng	Độ rộng	Mô tả
clk	Input	1 bit	Xung nhịp hệ thống, tác động sườn lên
pc_next	Input	32 bit	Giá trị PC cần nạp vào chu kỳ tiếp theo
pc	Output	32 bit	Giá trị PC hiện tại, đưa vào IMEM và IF



Hình 3.3: Instruction memory

Instruction Memory (IMEM) là bộ nhớ chỉ đọc dùng để lưu trữ chương trình dưới dạng các word 32-bit. Nội dung bộ nhớ được khởi tạo từ file Intel HEX trước khi hệ thống bắt đầu thực thi.

IMEM hỗ trợ đọc bất đồng bộ, cho phép lệnh được truy xuất ngay khi địa chỉ PC thay đổi. Nếu địa chỉ truy cập vượt quá phạm vi bộ nhớ, module sẽ trả về lệnh NOP (0x00000013) để đảm bảo hoạt động an toàn của hệ thống.

Bảng 3.2: Tín hiệu của module IMEM

Tín hiệu	Hướng	Độ rộng	Mô tả
addr	Input	32 bit	Địa chỉ byte cần đọc
inst	Output	32 bit	Lệnh tại addr[31:2]; NOP nếu OOB



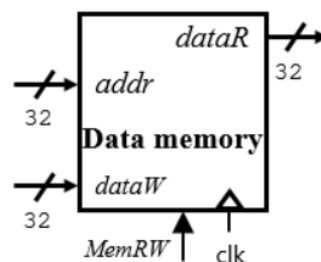
Hình 3.4: Register file

Register File lưu trữ trạng thái của bộ xử lý dưới dạng 32 thanh ghi đa dụng 32-bit (x0–x31) theo chuẩn RISC-V. Module hỗ trợ hai cổng đọc đồng thời cho các toán hạng rs1, rs2 và một cổng ghi kết quả từ tầng WB.

Thanh ghi x0 được xử lý theo đặc tả RISC-V: mọi lần đọc luôn trả về giá trị 0 và mọi yêu cầu ghi vào x0 đều bị bỏ qua. Điều này đảm bảo tính tương thích với tập lệnh RV32I.

Bảng 3.3: Tín hiệu của module Register File

Tín hiệu	Hướng	Độ rộng	Mô tả
clk	Input	1 bit	Xung nhịp hệ thống
rsR1	Input	5 bit	Địa chỉ thanh ghi nguồn 1 (cổng đọc 1)
rsR2	Input	5 bit	Địa chỉ thanh ghi nguồn 2 (cổng đọc 2)
rsW	Input	5 bit	Địa chỉ thanh ghi đích (cổng ghi)
dataW	Input	32 bit	Dữ liệu cần ghi vào thanh ghi rsW
RegWEn	Input	1 bit	Write enable: 1 = cho phép ghi tại sườn lên clk
dataR1	Output	32 bit	Giá trị thanh ghi rsR1 (có write-through nếu cần)
dataR2	Output	32 bit	Giá trị thanh ghi rsR2 (có write-through nếu cần)



Hình 3.5: Data memory

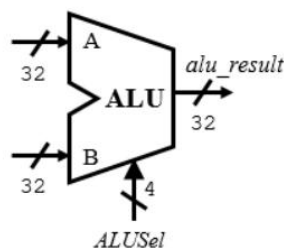
Data Memory là bộ nhớ dữ liệu dùng để thực hiện các lệnh LOAD và STORE trong bộ xử lý. Địa chỉ truy cập được cung cấp từ kết quả tính toán của ALU qua ngõ vào *addr*, trong khi dữ liệu ghi được đưa vào qua *dataW*. Tín hiệu

MemRW xác định chế độ đọc hoặc ghi của bộ nhớ. Module hỗ trợ đọc dữ liệu bất đồng bộ và ghi dữ liệu đồng bộ theo cạnh lên của xung clock. Trong datapath, Data Memory đóng vai trò lưu trữ dữ liệu và cho phép các lệnh như LW và SW trao đổi dữ liệu với bộ nhớ một cách chính xác.

Bảng 3.4: Tín hiệu của module Data memory

Tín hiệu	Hướng	Độ rộng	Mô tả
clk	Input	1 bit	Xung nhịp; ghi xảy ra tại sườn lên
addr	Input	32 bit	Địa chỉ byte; [31:2] làm chỉ số word; [1:0] phải = 00
dataW	Input	32 bit	Dữ liệu cần ghi (từ rs2 đã forwarding)
MemRW	Input	1 bit	1 = ghi; 0 = chỉ đọc
dataR	Output	32 bit	Dữ liệu đọc tổ hợp tại addr

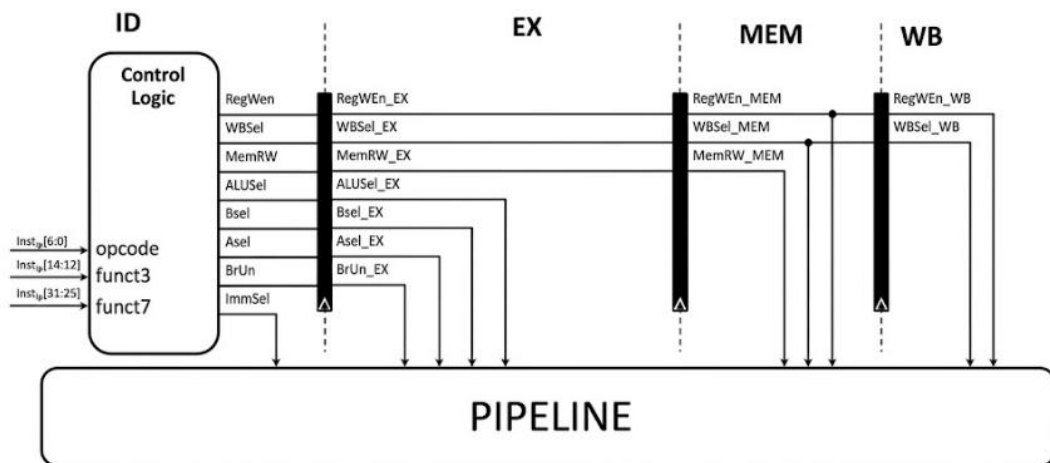
ALU (Arithmetic Logic Unit) là khối thực hiện các phép toán số học và logic trong bộ xử lý. Module nhận hai toán hạng 32 bit cùng tín hiệu điều khiển ALUSel để tạo ra kết quả `alu_result`. Kết quả này được sử dụng cho các lệnh tính toán, truy cập bộ nhớ và điều khiển luồng chương trình.



Hình 3.6: Cấu trúc ALU

Bảng 3.5: Tín hiệu module ALU

Tín hiệu	Hướng	Độ rộng	Mô tả
A	Input	32 bit	Toán hạng trái: rs1 hoặc PC (theo ASel)
B	Input	32 bit	Toán hạng phải: rs2 hoặc Imm (theo BSel)
ALUSel	Input	ALUSel_t	Chọn phép tính (enum 4-bit)
alu	Output	32 bit	Kết quả tính toán



Hình 3.7: Control Logic

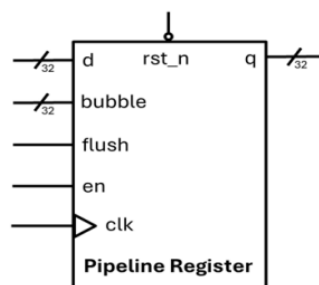
Control_Loic là đơn vị giải mã lệnh, sinh ra toàn bộ tín hiệu điều khiển datapath từ ba trường: opcode (7 bit), funct3 (3 bit) và funct7 (7 bit). Module này là thuần tổ hợp (purely combinational) và hoạt động tại tầng ID.

Module được thiết kế theo kiểu tổ hợp và hỗ trợ đầy đủ các nhóm lệnh RV32I như R-type, I-type, Load/Store, Branch, LUI, AUIPC, JAL và JALR.

Bảng 3.6: Tín hiệu module Control Logic

Tín hiệu	Hướng	Độ rộng	Mô tả
opcode	Input	7 bit	Trường opcode lệnh RISC-V [6:0]

Tín hiệu	Hướng	Độ rộng	Mô tả
funct3	Input	3 bit	Trường funct3 [14:12] xác định biến thể lệnh
funct7	Input	7 bit	Trường funct7 [31:25] phân biệt ADD/SUB, SRL/SRA
ImmSel	Output	ImmSel_t	Chọn kiểu mở rộng tức thời: I, S, B, U, J
BrUn	Output	1 bit	1 = so sánh không dấu (BLTU, BGEU)
ASel	Output	1 bit	0 = A = rs1; 1 = A = PC (AUIPC, JAL, B-type)
BSel	Output	1 bit	0 = B = rs2; 1 = B = Imm (I/S/B/U/J-type)
ALUSel	Output	ALUSel_t	Chọn phép tính ALU: ADD, SUB, SLT, AND, OR...
MemRW	Output	1 bit	1 = ghi bộ nhớ (Store); 0 = đọc hoặc không truy cập
RegWEn	Output	1 bit	1 = ghi kết quả về thanh ghi đích
WBSel	Output	WBSel_t	Chọn nguồn WB: ALU result, MEM data, PC+4



Hình 3.8: pipeline register

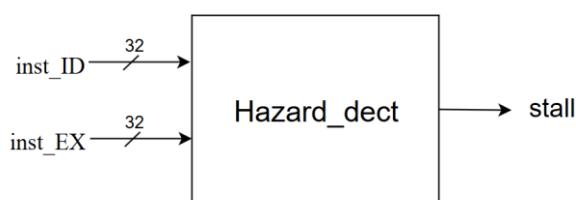
Các thanh ghi pipeline được thực hiện bằng module pipe_reg #(W) tham số hoá theo độ rộng. Module này hỗ trợ ba chế độ hoạt động với thứ tự ưu tiên:

- flush = 1 (ưu tiên cao nhất): $q \leftarrow \text{bubble}$ (giá trị NOP, ví dụ 0x00000013 cho inst)
- en = 1: $q \leftarrow d$ (cập nhật bình thường)
- en = 0 và flush = 0: q giữ nguyên (stall)

Bảng 3.7: các tín hiệu đi qua từng thanh ghi pipeline

Thanh ghi	Tín hiệu chứa
IF/ID	pc_ID, inst_ID, pred_taken_ID, pred_target_ID
ID/EX	pc_EX, dataR1_EX, dataR2_EX, imm_EX, inst_EX, ctrl_EX, pred_taken_EX, pred_target_EX
EX/MEM	pc_MEM, alu_MEM, dataR2_MEM, inst_MEM, ctrl_MEM, pc_plus4_mem
MEM/WB	alu_WB, mem_WB, pc_plus4_mem_WB, inst_WB, ctrl_WB

Việc mang theo inst_EX, inst_MEM, inst_WB qua pipeline (thay vì chỉ rd) cho phép Forwarding Unit và Hazard Unit trích xuất đầy đủ thông tin rs1, rs2, rd, opcode mà không cần thêm thanh ghi riêng cho từng trường.



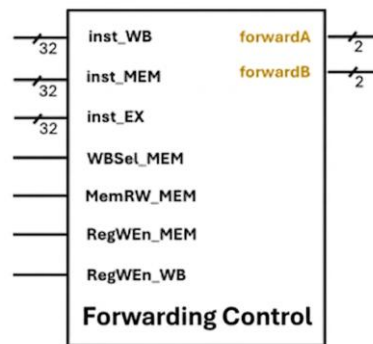
Hình 3.9: Hazard detection

Hazard Detection Unit có nhiệm vụ phát hiện Load-Use hazard và tạo tín hiệu stall để dừng pipeline một chu kỳ. Đây là loại hazard dữ liệu duy nhất trong thiết kế cần stall, trong khi các hazard khác được xử lý bằng forwarding.

Load-Use hazard xảy ra khi một lệnh LOAD chưa hoàn tất nhưng lệnh kế tiếp đã cần sử dụng dữ liệu của nó. Khi đó, hệ thống chèn một chu kỳ stall để đảm bảo dữ liệu được cập nhật trước khi sử dụng.

Bảng 3.8: Tín hiệu module Hazard Detection

Tín hiệu	Hướng	Độ rộng	Mô tả
inst_ID	Input	32 bit	Lệnh hiện tại tại ID stage (cần rs1, rs2)
inst_EX	Input	32 bit	Lệnh tại EX stage (kiểm tra opcode LOAD và rd)
stall	Output	1 bit	1 = phát hiện load-use hazard; giữ IF/ID, flush ID/EX

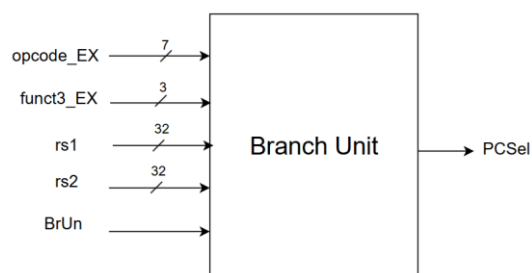
**Hình 3.10. Forwarding control logic**

Forwarding_Control_Logic (hay còn gọi là Forwarding Unit) giải quyết phần lớn data hazard mà không cần stall bằng cách chuyển tiếp (forward) kết quả từ tầng MEM hoặc WB về đầu vào ALU tại tầng EX. Module này so sánh địa chỉ thanh ghi đích của các lệnh đang ở MEM/WB với thanh ghi nguồn của lệnh đang ở EX.

Đầu ra forwardA và forwardB mỗi tín hiệu 2-bit, điều khiển MUX chọn nguồn cho toán hạng A và B của ALU. Ưu tiên forwarding từ MEM (2'b10) cao hơn WB (2'b01) để đảm bảo tính chính xác khi cả hai tầng đều có kết quả hợp lệ cho cùng một thanh ghi.

Bảng 3.9: Tín hiệu module Forwarding control logic

Tín hiệu	Hướng	Độ rộng	Mô tả
RegWEn_MEM	Input	1 bit	RegWEn của lệnh tại MEM stage
RegWEn_WB	Input	1 bit	RegWEn của lệnh tại WB stage
MemRW_MEM	Input	1 bit	MemRW của lệnh tại MEM
WBSel_MEM	Input	WBSel_t	Nguồn WB của lệnh MEM (ALU/MEM/PC4)
inst_EX	Input	32 bit	Lệnh tại EX: lấy rs1=[19:15], rs2=[24:20]
inst_MEM	Input	32 bit	Lệnh tại MEM: lấy rd=[11:7]
inst_WB	Input	32 bit	Lệnh tại WB: lấy rd=[11:7]
forwardA	Output	2 bit	00=no fwd; 10=from MEM; 01=from WB (cho rs1)
forwardB	Output	2 bit	00=no fwd; 10=from MEM; 01=from WB (cho rs2)

**Hình 3.11 Branch Unit**

Branch Unit là khối tổ hợp dùng để đánh giá điều kiện rẽ nhánh và tạo tín hiệu PCSel, quyết định luồng thực thi tiếp theo của bộ xử lý.

Module nhận các tín hiệu opcode_EX, funct3_EX, rs1, rs2 và BrUn để thực hiện các phép so sánh có dấu hoặc không dấu. Dựa trên kết quả so sánh, Branch

Unit xác định các lệnh nhánh có điều kiện (BEQ, BNE, BLT, BGE, ...) có được thực hiện hay không, đồng thời luôn kích hoạt chuyển hướng đối với các lệnh nhảy vô điều kiện JAL và JALR.

Tín hiệu PCSel được sử dụng tại tầng EX để xác định kết quả nhánh thực tế và phát hiện branch misprediction, từ đó kích hoạt cơ chế flush pipeline khi cần.

Bảng 3.10: Mô tả tín hiệu Branch Unit

Tín hiệu	Hướng	Độ rộng	Mô tả
opcode_EX	Input	7-bit	Xác định loại lệnh tại tầng EX (Branch, JAL, JALR, ...).
funct3_EX	Input	3-bit	Xác định điều kiện nhánh như BEQ, BNE, BLT, BGE, BLTU, BGEU.
rs1[31:0]	Input	32-bit	Toán hạng thứ nhất dùng cho so sánh.
rs2[31:0]	Input	32-bit	Toán hạng thứ hai dùng cho so sánh.
BrUn	Input	1-bit	Chọn so sánh có dấu hoặc không dấu.
PCSel	Output	1-bit	Quyết định chuyển hướng PC hay tiếp tục tại PC+4.

3.4 Thiết kế khối dự đoán nhánh

3.4.1 Ý tưởng thiết kế

Trong kiến trúc RV32I pipeline truyền thống, kết quả của lệnh branch chỉ được xác định tại tầng Execute (EX). Điều này làm phát sinh Control Hazard do bộ xử lý không biết hướng thực thi tiếp theo của chương trình cho đến khi phép so sánh hoàn thành.

Khi gặp một lệnh branch, pipeline vẫn tiếp tục nạp các lệnh theo hướng tuần tự PC+4. Nếu branch thực sự được thực hiện (taken), các lệnh đã được nạp trước đó trở nên không hợp lệ và phải bị loại bỏ khỏi pipeline. Quá trình này làm mất nhiều chu kỳ xử lý và làm giảm hiệu năng hệ thống.

Để khắc phục hạn chế này, đề tài đề xuất sử dụng cơ chế dự đoán nhánh động (Dynamic Branch Prediction). Ý tưởng chính là dự đoán kết quả của branch ngay tại tầng IF dựa trên lịch sử thực thi trước đó. Nếu dự đoán đúng, pipeline tiếp tục hoạt động liên tục mà không cần flush. Nếu dự đoán sai, hệ thống sẽ phục hồi bằng cơ chế misprediction recovery.

Khối dự đoán nhánh trong đề tài gồm hai thành phần chính:

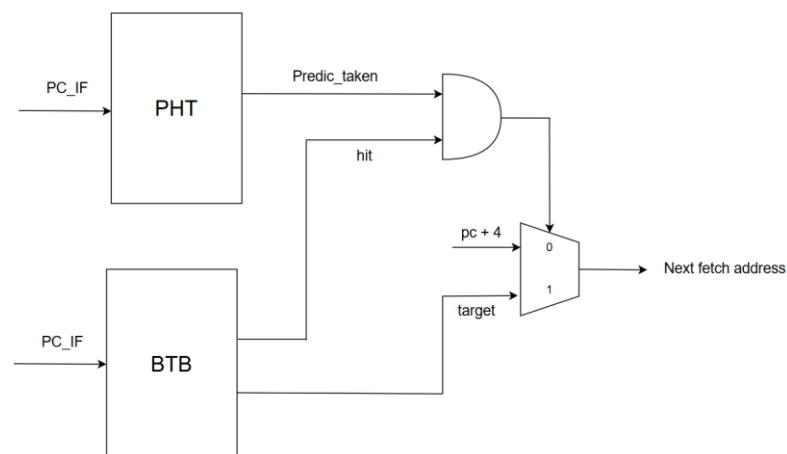
- Branch Target Buffer (BTB)
- Pattern History Table (PHT)

Trong đó:

- BTB dự đoán địa chỉ đích của branch.
- PHT dự đoán branch sẽ Taken hay Not Taken.

Sự kết hợp của hai khối này cho phép bộ xử lý xác định hướng thực thi tiếp theo ngay từ tầng IF.

3.4.2 Kiến trúc tổng thể của khối dự đoán nhánh



Hình 3.12: Kiến trúc khối dự đoán nhánh

Hình 3.12 trình bày kiến trúc tổng thể của khối dự đoán nhánh được tích hợp tại tầng IF của bộ xử lý. Khối dự đoán nhánh gồm hai thành phần chính là Branch Target Buffer (BTB) và Pattern History Table (PHT).

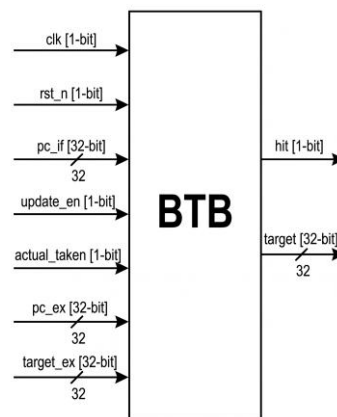
Tại mỗi chu kỳ fetch, địa chỉ PC hiện tại được sử dụng đồng thời để tra cứu BTB và PHT. BTB có nhiệm vụ lưu trữ địa chỉ đích của các lệnh branch hoặc jump đã được thực hiện trước đó. Nếu tìm thấy entry tương ứng, BTB sẽ phát tín hiệu hit và xuất ra địa chỉ đích target.

Song song với đó, PHT dự đoán hướng thực thi của nhánh thông qua tín hiệu `predict_taken`. Giá trị này được xác định dựa trên lịch sử thực thi của branch được lưu trong bộ đếm bão hòa 2-bit.

Tín hiệu `predict_taken` từ PHT và tín hiệu hit từ BTB được đưa vào cổng AND để tạo tín hiệu `predict_redirect`. Chỉ khi BTB tìm thấy entry hợp lệ và PHT dự đoán nhánh được thực hiện (Taken), hệ thống mới chuyển hướng luồng thực thi tới địa chỉ đích từ BTB.

Tín hiệu `predict_redirect` điều khiển bộ chọn MUX để xác định địa chỉ fetch tiếp theo. Nếu `predict_redirect = 1`, bộ xử lý sử dụng địa chỉ đích `target` từ BTB. Ngược lại, hệ thống tiếp tục thực hiện tuần tự với địa chỉ `PC + 4`. Nhờ cơ chế này, bộ xử lý có thể đưa ra quyết định rẽ nhánh ngay tại tầng IF mà không cần chờ đến tầng EX, từ đó giảm số lần flush pipeline và cải thiện hiệu năng thực thi.

3.4.3 Branch Target Buffer (BTB)



Hình 3.13: Branch Target Buffer

Branch Target Buffer (BTB) là thành phần chịu trách nhiệm cung cấp địa chỉ đích của các lệnh branch hoặc jump ngay tại tầng IF trước khi lệnh được thực thi. Nhờ đó bộ xử lý có thể chuyển hướng luồng thực thi sớm mà không cần chờ đến tầng EX để tính toán địa chỉ đích.

BTB được tổ chức dưới dạng bảng tra cứu trực tiếp (direct-mapped table) gồm 256 entry. Mỗi entry bao gồm ba trường: bit valid dùng để xác định entry hợp

lệ, trường tag dùng để xác thực địa chỉ PC đang tra cứu và trường target dùng để lưu địa chỉ đích tương ứng của branch hoặc jump. Với cấu hình ENTRY = 256, BTB sử dụng 8 bit index và 22 bit tag.

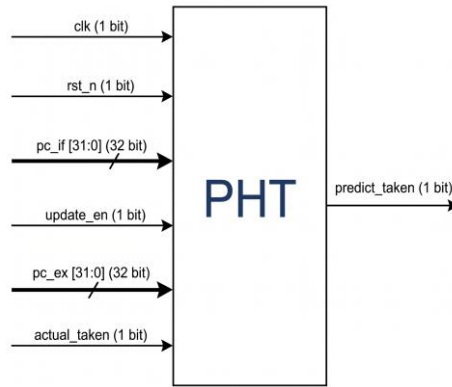
Tại tầng IF, địa chỉ PC hiện tại được sử dụng để tra cứu BTB. Nếu tìm thấy entry hợp lệ và tag khớp, tín hiệu hit được kích hoạt và địa chỉ đích target được xuất ra. Ngược lại, BTB trả về trạng thái miss và bộ xử lý tiếp tục thực hiện tuần tự theo địa chỉ PC + 4.

Quá trình cập nhật xảy ra tại tầng EX: pc_ex là địa chỉ của lệnh branch/jump đang thực thi, target_ex là địa chỉ đích thực tế (kết quả ALU), actual_taken là kết quả Branch Unit. Thông tin này được ghi vào entry tương ứng với index của pc_ex. Việc cập nhật ghi đè entry cũ nếu có xung đột index (alias), đây là hành vi bình thường của direct-mapped cache và sẽ gây ra mispredict nhất thời nếu alias xảy ra thường xuyên.

Bảng 3.11: Tín hiệu module BTB

Tín hiệu	Hướng	Độ rộng	Mô tả
clk	Input	1 bit	Xung nhịp; cập nhật BTB tại sườn lên
rst_n	Input	1 bit	Reset: xóa valid của tất cả 256 entry
pc_if	Input	32 bit	PC tại IF để tra cứu BTB
hit	Output	1 bit	1 = tìm thấy entry hợp lệ cho pc_if
target	Output	32 bit	Địa chỉ đích dự đoán (0 nếu miss)
update_en	Input	1 bit	1 = cập nhật BTB (khi có branch/jump tại EX)
actual_taken	Input	1 bit	1 = nhánh thực sự được thực hiện
pc_ex	Input	32 bit	PC của lệnh branch tại EX stage
target_ex	Input	32 bit	Địa chỉ đích thực tế tính bởi ALU

3.4.4 Pattern History Table (PHT)



Hình 3.14: Pattern History Table.

Pattern History Table (PHT) là thành phần chịu trách nhiệm dự đoán hướng thực thi của các lệnh branch, tức là dự đoán nhánh sẽ được thực hiện (Taken) hay không thực hiện (Not Taken). PHT được tổ chức dưới dạng một mảng gồm 256 bộ đếm bão hòa 2-bit (2-bit saturating counter), trong đó mỗi bộ đếm tương ứng với một nhóm địa chỉ PC có cùng giá trị index.

Mỗi bộ đếm 2-bit có thể biểu diễn bốn trạng thái: Strongly Not Taken (00), Weakly Not Taken (01), Weakly Taken (10) và Strongly Taken (11). Hướng dự đoán được xác định bởi bit cao nhất (MSB) của bộ đếm. Khi MSB bằng 1, PHT dự đoán nhánh sẽ được thực hiện (Taken); ngược lại, khi MSB bằng 0, PHT dự đoán nhánh sẽ không được thực hiện (Not Taken).

Ưu điểm của bộ đếm bão hòa 2-bit so với bộ dự đoán 1-bit là khả năng duy trì tính ổn định khi xuất hiện các trường hợp ngoại lệ ngắn hạn. Ví dụ, nếu một branch thường xuyên ở trạng thái Strongly Taken (11), một lần Not Taken bất thường chỉ làm bộ đếm giảm xuống Weakly Taken (10) và vẫn tiếp tục dự đoán Taken ở lần tiếp theo. Điều này giúp giảm hiện tượng dao động dự đoán và đặc biệt hiệu quả đối với các vòng lặp, nơi branch thường được thực hiện nhiều lần liên tiếp và chỉ không thực hiện ở lần lặp cuối cùng.

Bảng 3.12: Tín hiệu module PHT

Tín hiệu	Hướng	Độ rộng	Mô tả
clk	Input	1 bit	Xung nhịp; cập nhật PHT tại sườn lên

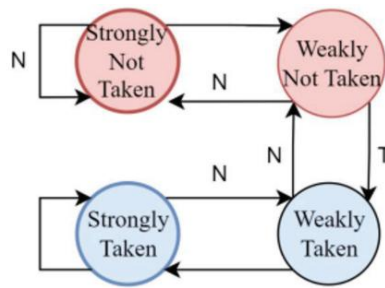
Tín hiệu	Hướng	Độ rộng	Mô tả
rst_n	Input	1 bit	Reset: khởi tạo tất cả entry về 2'b01
pc_if	Input	32 bit	PC tại IF stage để tra cứu dự đoán
predict_taken	Output	1 bit	1 = dự đoán taken; 0 = dự đoán not-taken
update_en	Input	1 bit	1 = cập nhật PHT tại EX stage
pc_ex	Input	32 bit	PC của lệnh branch tại EX để chọn entry cập nhật
actual_taken	Input	1 bit	Kết quả thực tế từ Branch_Unit/PCSel

Toàn bộ PHT được khởi tạo ở trạng thái 2'b01 (Weakly Not Taken). Lựa chọn này giúp bộ dự đoán có xu hướng thận trọng ở giai đoạn đầu nhưng vẫn có thể nhanh chóng thích nghi với hành vi thực tế của chương trình sau một vài lần cập nhật.

PHT được cập nhật tại tầng EX khi một lệnh branch hoặc jump hoàn thành. Nếu nhánh được thực hiện (actual_taken = 1), bộ đếm sẽ tăng lên một mức; ngược lại bộ đếm sẽ giảm xuống một mức. Quá trình cập nhật sử dụng cơ chế bão hòa, nghĩa là bộ đếm không thể tăng vượt quá trạng thái Strongly Taken (11) hoặc giảm thấp hơn trạng thái Strongly Not Taken (00).

Bộ đếm bão hòa 2-bit cho phép lưu trữ mức độ tin tưởng của bộ dự đoán thay vì chỉ lưu kết quả Taken hoặc Not Taken. Khi xảy ra một lần dự đoán sai, trạng thái chỉ dịch chuyển một mức thay vì đảo hoàn toàn sang hướng ngược lại. Nhờ đó bộ dự đoán ổn định hơn và giảm ảnh hưởng của các trường hợp ngoại lệ ngắn hạn.

Trong kiến trúc đề xuất, PHT chịu trách nhiệm dự đoán hướng thực thi của nhánh (Taken/Not Taken), trong khi BTB cung cấp địa chỉ đích tương ứng. Hai thành phần này phối hợp với nhau để tạo ra quyết định dự đoán hoàn chỉnh tại tầng IF.



Hình 3.15: Máy trạng thái bộ đếm bão hòa 2-bit của PHT

3.5 Phương pháp đánh giá

Để so sánh hiệu năng giữa hai kiến trúc một cách định lượng và nhất quán, ba chỉ số đo lường được sử dụng:

Cycles per Instruction (CPI) là số chu kỳ xung nhịp trung bình cần thiết để hoàn thành một lệnh. So sánh CPI giữa hai kiến trúc phản ánh mức độ giảm thiểu overhead pipeline mà bộ dự đoán nhánh mang lại. CPI được tính theo công thức:

$$CPI = \frac{\text{Cycles}}{\text{Instructions}}$$

Tần số hoạt động tối đa (Fmax) là tần số xung nhịp cao nhất mà thiết kế có thể hoạt động đúng sau khi synthesis và phân tích timing. Việc tích hợp thêm BTB và PHT vào pipeline làm kéo dài critical path, do đó tần số hoạt động của kiến trúc có bộ dự đoán nhánh sẽ thấp hơn kiến trúc cơ bản. Chỉ số này phản ánh chi phí phần cứng của bộ dự đoán nhánh.

Thời gian thực thi (T_{exec}) là thời gian thực tế để hoàn thành toàn bộ chương trình. Rất quan trọng vì kết hợp đồng thời cả số chu kỳ (hiệu quả pipeline) và Fmax (chi phí phần cứng). Một kiến trúc có CPI thấp hơn nhưng vẫn có thể không cải thiện T_{exec} đáng kể, hoặc thậm chí kém hơn trong một số trường hợp. T_{exec} tính theo công thức:

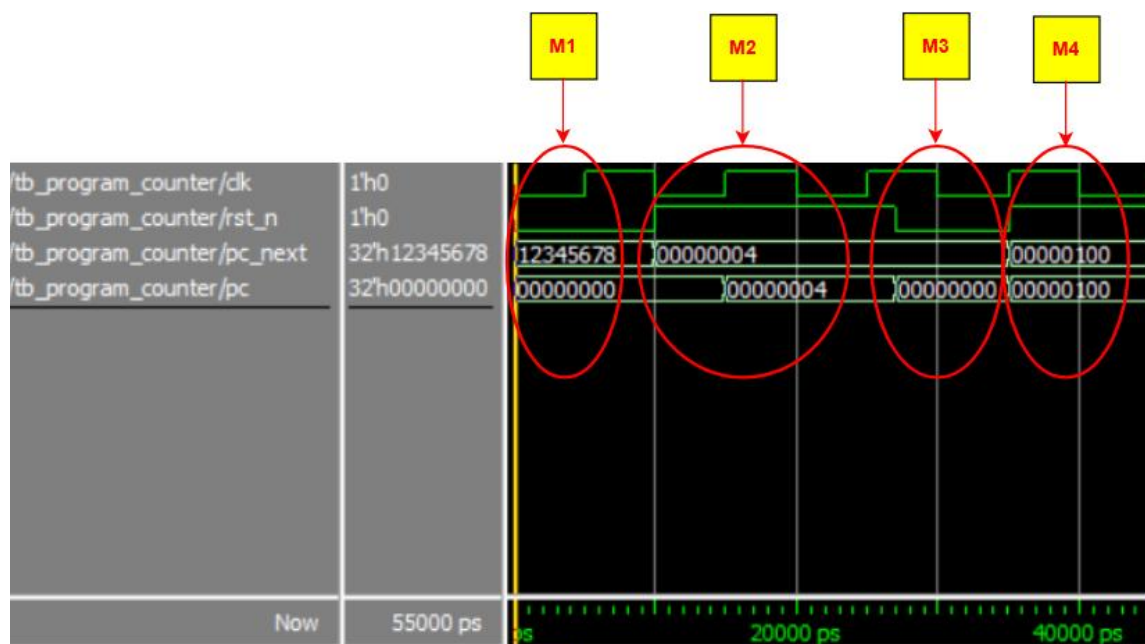
$$T_{\text{exec}} = \frac{\text{Cycles}}{F_{\text{max}}}$$

CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ

4.1 Kết quả mô phỏng từng khối

4.1.1 Khối Program Counter

Hình 4.1 mô tả chi tiết dạng sóng của Program Counter, trong đó giá trị PC được cập nhật và tăng tuần tự theo từng chu kỳ clock trong quá trình thực thi chương trình.



Hình 4.1 Dạng sóng mô tả chức năng cập nhật địa chỉ mới của khối Program Counter

- ❖ Tại thời điểm M1 kiểm tra chức năng reset. Khi `rst_n` = 0, ngõ ra `pc` được đưa về 0x00 và không phụ thuộc vào giá trị `pc_next` (`pc_next` đang là 32'h12345678).
- ❖ Tại thời điểm M2 kiểm tra chức năng cập nhật PC. Sau khi `rst_n` = 1, các giá trị 0x04 được nạp vào `pc`.
- ❖ Tại thời điểm M3 kiểm tra reset trong khi hệ thống đang hoạt động. Khi `rst_n` = 0, `pc` = 0x00 mà không cần chờ cạnh clock, chứng tỏ reset bất đồng bộ hoạt động đúng.

- ❖ Tại thời điểm M4 kiểm tra khả năng hoạt động trở lại sau reset. Sau khi giải phóng reset, giá trị 0x100 được nạp thành công vào pc tại cạnh lên kế tiếp của clock.

=> Kết quả mô phỏng cho thấy Program Counter hoạt động đúng theo thiết kế. Tín hiệu pc luôn được reset về 0x00 khi rst_n ở mức thấp và chỉ cập nhật giá trị mới từ pc_next tại cạnh lên của xung clock khi reset được giải phóng.

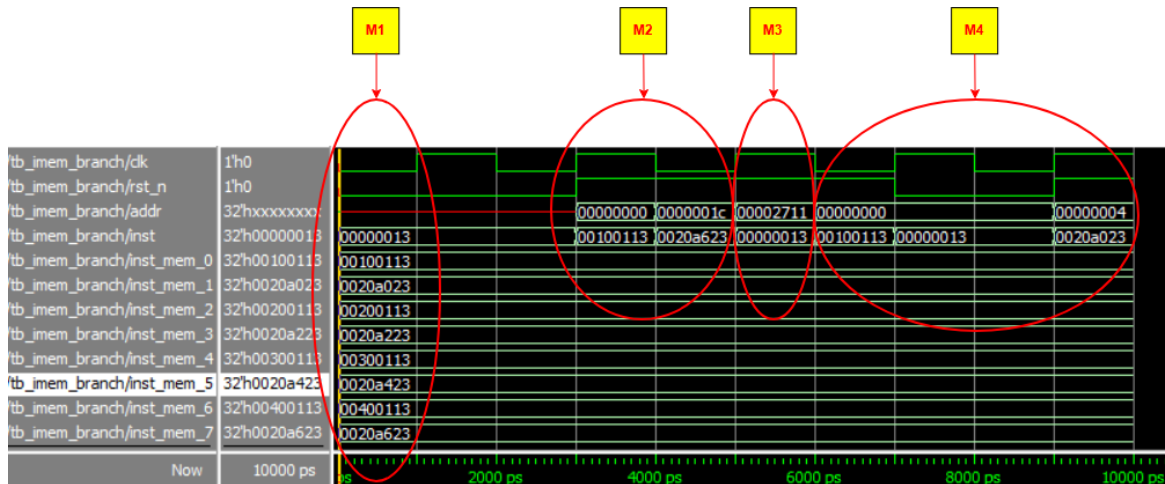
4.1.2 Khối Instruction Memory

Hình 4.2 mô tả dữ liệu nạp vào Instruction Memory một chương trình có nhiệm vụ lưu các giá trị 1, 2, 3, 4 vào 4 ô nhớ 4 byte của CPU có địa chỉ từ mem[0x1000] đến mem[0x100C] thông qua tệp khởi tạo là imem_test.hex. Với từng giá trị PC thì Instruction Memory có nhiệm vụ truy xuất các lệnh tương ứng và dựa vào hình 4.3 ta có thể xác nhận khối này hoạt động hoàn toàn đúng.

1	00100113
2	0020a023
3	00200113
4	0020a223
5	00300113
6	0020a423
7	00400113
8	0020a623

Hình 4.2 Chương trình được nạp vào Instruction Memory

Hình 4.3 mô tả dạng sóng mô phỏng của Instruction Memory, thể hiện giá trị lệnh được đọc ra tương ứng với từng địa chỉ PC trong quá trình thực thi.



Hình 4.3 Dạng sóng mô tả chức năng nạp dữ liệu của khối Instruction Memory

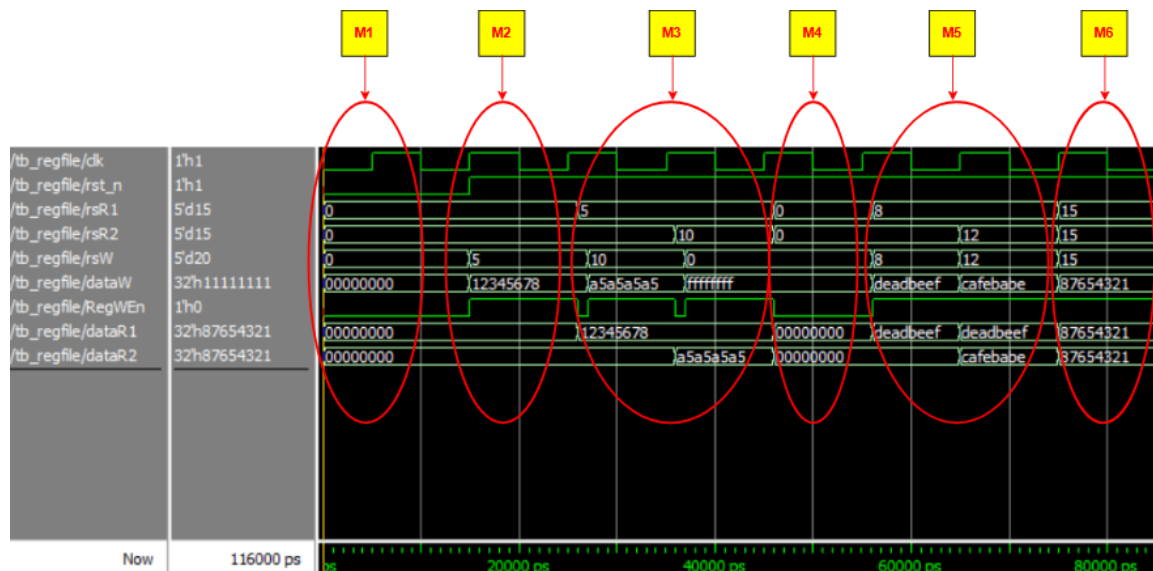
- ❖ Tại thời điểm M1 kiểm tra quá trình khởi tạo bộ nhớ lệnh và trạng thái reset của các tín hiệu. Khi $rst = 1$ thì $instr = 0x13$, chứng tỏ CPU sẽ không làm gì khi có tín hiệu reset. Module Instruction Memory thực hiện nạp dữ liệu từ file `imem_test.hex` hoàn toàn chính xác, có thể đối chiếu tín hiệu từ `inst_mem_0` đến `inst_mem_7` sẽ tương ứng với giá trị từ hàng số 1 đến hàng số 8 trong hình 4.2.
- ❖ Tại thời điểm M2 kiểm tra hoạt động truy xuất lệnh dựa trên giá trị `addr` (PC). Khi `addr = 0x00` thì sẽ nạp lệnh tương ứng vào là `32'h00100113`, tiếp theo với giá trị `addr = 0x1C` thì nó sẽ nạp lệnh tương ứng là `32'h0020A623`.
- ❖ Tại thời điểm M2 kiểm tra trường hợp truy cập ngoài phạm vi bộ nhớ. Đặt `addr = 0x00002711` tương đương `addr[31:2] = 10001`, trong khi `DEPTH_WORDS = 10000` nên điều kiện `addr[31:2] < DEPTH_WORDS` không thỏa mãn, Ngõ ra `inst` về lại giá trị `0x13` (lệnh NOP).
- ❖ Từ 14 – 19ns kiểm tra đọc các địa chỉ liên tiếp có chèn reset ở giữa. Đặt 2 địa chỉ liên tiếp là `0x00 → 0x04` nhưng ở địa chỉ `0x00` cho `reset = 1` nên `instruction` về giá trị `0x13`.

=> Kết quả mô phỏng cho thấy module Instruction Memory hoạt động đúng với yêu cầu thiết kế. Nạp dữ liệu từ file HEX thành công; Đọc đúng dữ liệu tại các địa chỉ hợp lệ; Chuyển đổi địa chỉ byte sang word chính xác thông qua `addr[31:2]`; Hỗ

trợ đọc bất đồng bộ; dữ liệu thay đổi ngay khi địa chỉ thay đổi; Trả về lệnh NOP (0x13) khi truy cập ngoài phạm vi bộ nhớ.

4.1.3 Khối Register File

Hình 4.4 mô tả dạng sóng mô phỏng của Register File, thể hiện quá trình đọc hai thanh ghi nguồn và ghi dữ liệu vào thanh ghi đích theo từng chu kỳ clock.



Hình 4.4 Dạng sóng mô tả chức năng ghi và đọc dữ liệu của khối Register File

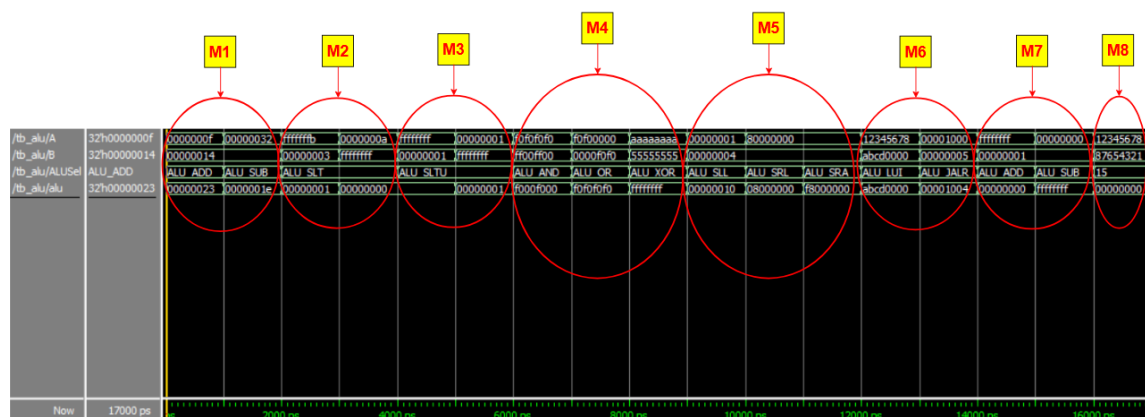
- ❖ Tại thời điểm M1 kiểm tra chức năng reset bất đồng bộ. Khi rst_n = 0, toàn bộ 32 thanh ghi được đưa về giá trị 0x00.
- ❖ Tại thời điểm M2 kiểm tra chức năng ghi dữ liệu. Sau khi nhả reset, tín hiệu RegWEn được kích hoạt và dữ liệu 32'h12345678 được ghi vào thanh ghi x5.
- ❖ Tại thời điểm M3 kiểm tra chức năng đọc dữ liệu. Khi đọc thanh ghi x5, ngõ ra dataR1 trả về giá trị 32'h12345678 đã được ghi trước đó. Đồng thời, sau khi ghi thêm giá trị 32'hA5A5A5A5 vào x10, hai cổng đọc có thể truy xuất đồng thời:
 - dataR1 = 32'h12345678 (x5)
 - dataR2 = 32'hA5A5A5A5 (x10)

- ❖ Tại thời điểm M4 kiểm tra cơ chế bảo vệ thanh ghi x0. Mặc dù tín hiệu ghi được kích hoạt với $rsW = 0$ và dữ liệu ghi là 32'hFFFFFFFF, giá trị của x0 vẫn giữ nguyên 0x00.
- ❖ Tại thời điểm M5 kiểm tra cơ chế Write-Through (bypass). Khi địa chỉ đọc trùng với địa chỉ ghi trong cùng chu kỳ, các cổng đọc nhận trực tiếp dữ liệu mới:
 - $x8 \rightarrow dataR1 = 32'hDEADBEEF$
 - $x12 \rightarrow dataR2 = 32'hCAFEBABE$
- ❖ Tại thời điểm M6 kiểm tra Write-Through đồng thời trên cả hai cổng đọc. Khi $rsR1 = rsR2 = rsW = 15$ và $dataW = 32'h87654321$, cả $dataR1$ và $dataR2$ đều bằng 32'h87654321.

=> Kết quả mô phỏng cho thấy Register File hoạt động đúng theo thiết kế, bao gồm reset bất đồng bộ, ghi đồng bộ, đọc tổ hợp, bảo vệ thanh ghi x0, hỗ trợ hai cổng đọc đồng thời và cơ chế Write-Through (bypass) giúp giảm hazard dữ liệu trong kiến trúc pipeline RV32I.

4.1.4 Khối Arithmetic Logic Unit (ALU)

Hình 4.5 mô tả dạng sóng mô phỏng của ALU, thể hiện quá trình nhận hai toán hạng đầu vào và sinh ra kết quả tính toán tương ứng với các mã điều khiển ALUSel.



Hình 4.5 Dạng sóng mô tả các hoạt động chính của khối xử lý logic số học- ALU

❖ Tại thời điểm M1 thực hiện phép cộng và trừ 2 số:

- $ALUSel = ALU_ADD$, $A = 15$, $B = 20$. Kết quả $alu = 35$.
- $ALUSel = ALU_SUB$, $A = 50$, $B = 20$. Kết quả $alu = 30$.

❖ Tại thời điểm M2 (SLT):

- $A = -5$, $B = 3 \rightarrow alu = 1$ ($A < B$).
- $A = 10$, $B = -1 \rightarrow alu = 0$ ($A > B$).

❖ Tại thời điểm M3 (SLTU):

- $A = 32'hFFFFFFF$, $B = 32'h00000001 \rightarrow alu = 0$.
- $A = 32'h00000001$, $B = 32'hFFFFFFF \rightarrow alu = 1$.

❖ Tại thời điểm M4 thực hiện liên tiếp 3 phép logic:

- AND: $A = 32'hF0F0F0F0$, $B = 32'hFF00FF00$. Kết quả $alu = 32'hF000F000$.
- OR: $A = 32'hF0F00000$, $B = 32'h0000F0F0$. Kết quả $alu = 32'hF0F0F0F0$.
- XOR: $A = 32'hAAAAAAAA$, $B = 32'h55555555$. Kết quả $alu = 32'hFFFFFFF$.

❖ Tại thời điểm M5 thực hiện liên tiếp 3 phép dịch:

- SLL: $A = 1$, $B = 4$. $alu = 32'h00000010$, dịch trái 4 bit.
- SRL: $A = 32'h80000000$, $B = 4$, $alu = 32'h08000000$, đúng với phép dịch phải logic.
- SRA: $A = 32'h80000000$, $B = 4$, $alu = 32'hF8000000$, bit dấu được mở rộng đúng theo phép dịch phải số học.

❖ Tại thời điểm M6 thực hiện liên tiếp 2 phép toán đặc biệt:

- LUI: $B = 32'hABCD0000$, $alu = B$, đúng với chức năng LUI.
- JALR: $A = 0x1000$, $B = 0x05$, $alu = 0x1004$, đúng với JALR (xóa bit thấp nhất của địa chỉ đích).

❖ Tại thời điểm M7 thực hiện 2 phép kiểm tra giới hạn :

- Tràn phép cộng: $A = 32'hFFFFFFF$, $B = 1$, $alu = 0x00$.
- Tràn phép trừ: $A = 0$, $B = 1$, $alu = 32'hFFFFFFF$.

❖ Tại thời điểm M8 ALUSel nhận giá trị không hợp lệ, kết quả $alu = 0x00$.

=> Kết quả mô phỏng cho thấy ALU thực hiện chính xác các chức năng theo chuẩn RV32I, bao gồm các phép toán số học, so sánh, logic, dịch bit, LUI và JALR. Các trường hợp biên như tràn số học, giới hạn độ dịch và mã điều khiển không hợp lệ cũng được xử lý đúng.

4.1.5 Khối Data Memory

Hình 4.6 mô tả dạng sóng mô phỏng của Data Memory, thể hiện quá trình truy cập bộ nhớ với các thao tác đọc và ghi dữ liệu dựa trên tín hiệu MemRW, địa chỉ addr và giá trị dataW.



Hình 4.6 Dạng sóng mô tả chức năng ghi và đọc dữ liệu của khối Data Memory

- ❖ Tại thời điểm M1 kiểm tra chức năng reset. Khi $rst_n = 0$, ngõ ra dataR luôn bằng 0x00, cho dù địa chỉ đã đổi thành 0x05 thì dataR vẫn bằng 0x00.
- ❖ Tại thời điểm M2 kiểm tra ghi và đọc dữ liệu. Khi $rst_n = 1$, $MemRW = 1$, dữ liệu 32'h12345678 được ghi vào mem[0] tại cạnh lên của clock. Sau đó

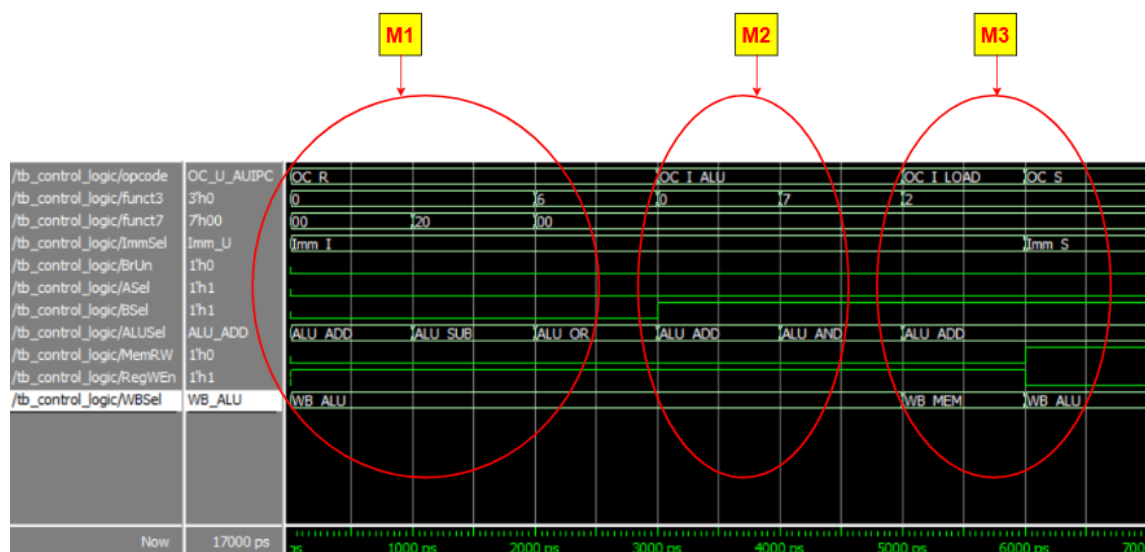
cho MemRW = 0 và giữ địa chỉ 0x00, ngõ ra dataR trả về 32'h12345678, đúng với dữ liệu đã ghi trước đó.

- ❖ Tại thời điểm M3 kiểm tra địa chỉ không căn chỉnh word (unaligned address). Khi địa chỉ là 0x02, bộ nhớ không thực hiện truy cập hợp lệ và dataR luôn bằng 0x00
- ❖ Tại thời điểm M4 kiểm tra truy cập ngoài phạm vi bộ nhớ ($\text{addr}[31:2] \geq \text{DEPTH_WORDS}$). Kết quả cho thấy dữ liệu không được ghi và dataR trả về 0x00.
- ❖ Tại thời điểm M5 kiểm tra thay đổi địa chỉ ghi trước cạnh lên clock. Địa chỉ ban đầu là mem[0] nhưng được đổi sang mem[4] trước cạnh clock, nên dữ liệu cuối cùng được ghi vào mem[4], phù hợp với cơ chế ghi đồng bộ theo cạnh lên.

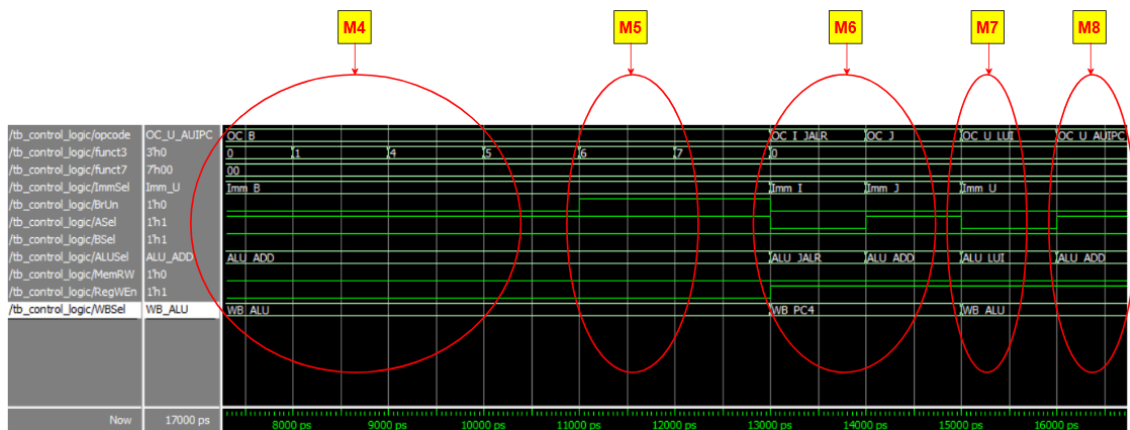
=> Kết quả mô phỏng cho thấy Data Memory thực hiện chính xác các chức năng chính. Các trường hợp đặc biệt như reset, địa chỉ không căn chỉnh, truy cập ngoài phạm vi bộ nhớ và đọc trong chế độ ghi đều được xử lý đúng theo yêu cầu thiết kế.

4.1.6 Khối Control Logic

Hình 4.7A và hình 4.7B mô tả dạng sóng mô phỏng của khối Control Logic, thể hiện sự thay đổi của các tín hiệu điều khiển tương ứng với từng mã lệnh khác nhau.



Hình 4.7A Dạng sóng mô tả chức năng giải mã lệnh khối Control Logic 0 – 7ns



Hình 4.7B Dạng sóng mô tả chức năng giải mã lệnh khối Control Logic 7 – 17ns

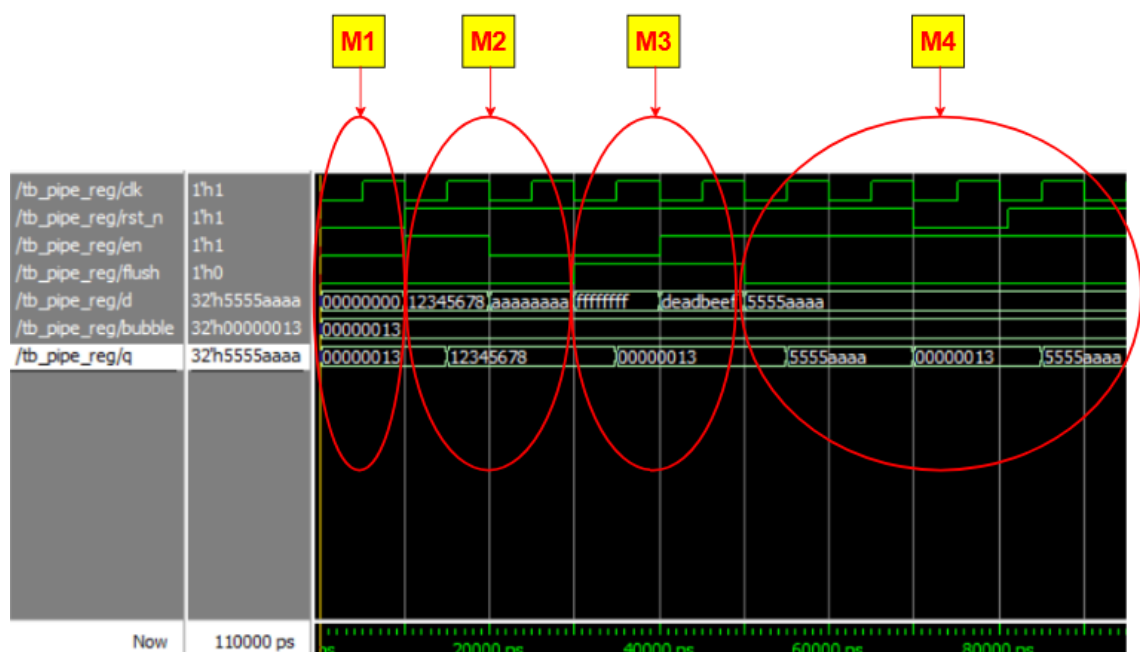
- ❖ Tại thời điểm M1 kiểm tra nhóm lệnh R-type (ADD, SUB, OR). Kết quả là bộ điều khiển giải mã chính xác các phép toán với ALUSel lần lượt là ALU_ADD, ALU_SUB, ALU_OR. Các tín hiệu $RegWEn = 1$, $WBSel = WB_ALU$, $MemRW = 0$.
- ❖ Tại thời điểm M2 kiểm tra nhóm lệnh I-type ALU (ADDI, ANDI). Kết quả là $ImmSel = Imm_I$, $BSel = 1$ và ALUSel đúng với từng phép toán. Dữ liệu được ghi về thanh ghi đích thông qua $WBSel = WB_ALU$ với $RegWEn = 1$.
- ❖ Tại thời điểm M3 kiểm tra các lệnh LOAD/STORE.
 - LW: Bộ điều khiển tạo $ImmSel = Imm_I$, $ALUSel = ALU_ADD$, đồng thời $RegWEn = 1$, $WBSel = WB_MEM$ để ghi dữ liệu đọc từ bộ nhớ về thanh ghi.
 - SW: Bộ điều khiển tạo $ImmSel = Imm_S$, $MemRW = 1$, $RegWEn = 0$, đảm bảo chỉ thực hiện ghi bộ nhớ mà không ghi thanh ghi.
- ❖ Tại thời điểm M4 kiểm tra các lệnh nhánh BEQ, BNE, BLT và BGE. Kết quả cho thấy $ASel = BSel = 1$, ALU_ADD và $BrUn = 1'b0$.
- ❖ Tại thời điểm M5 kiểm tra các lệnh BLTU và BGEU. Kết quả cho thấy $BrUn = 1$.
- ❖ Tại thời điểm M6 kiểm tra các lệnh nhảy JALR và JAL.

- JALR: ImmSel = Imm_I, ALUSel = ALU_ADD, WBSel = WB_PC4.
- JAL: ImmSel = Imm_J, ASel = 1, thực hiện nhảy vô điều kiện và lưu địa chỉ trở về.
- ❖ Tại thời điểm M7 kiểm tra lệnh LUI. Kết quả cho thấy ImmSel = Imm_U, ALUSel = ALU_LUI, RegWEn = 1 và WBSel = WB_ALU.
- ❖ Tại thời điểm M8 kiểm tra lệnh AUIPC. Bộ điều khiển tạo ImmSel = Imm_U, ASel = 1, BSel = 1, ALUSel = ALU_ADD, RegWEn = 1 và WBSel = WB_ALU để thực hiện phép cộng giữa PC và immediate.

=> Kết quả mô phỏng cho thấy module Control_Logic giải mã chính xác toàn bộ các nhóm lệnh cơ sở RV32I gồm R-type, I-type, Load, Store, Branch, Jump và U-type. Các tín hiệu điều khiển quan trọng đều được tạo đúng theo đặc tả kiến trúc.

4.1.7 Khối Pipeline Register

Hình 4.8 mô phỏng của khối Pipeline Register, minh họa cách dữ liệu đầu vào d được chốt vào đầu ra q dưới sự điều khiển của các tín hiệu clk, en, flush và bubble.



Hình 4.8 Dạng sóng mô tả chức năng truyền dữ liệu tuần tự của khối Pipeline Register

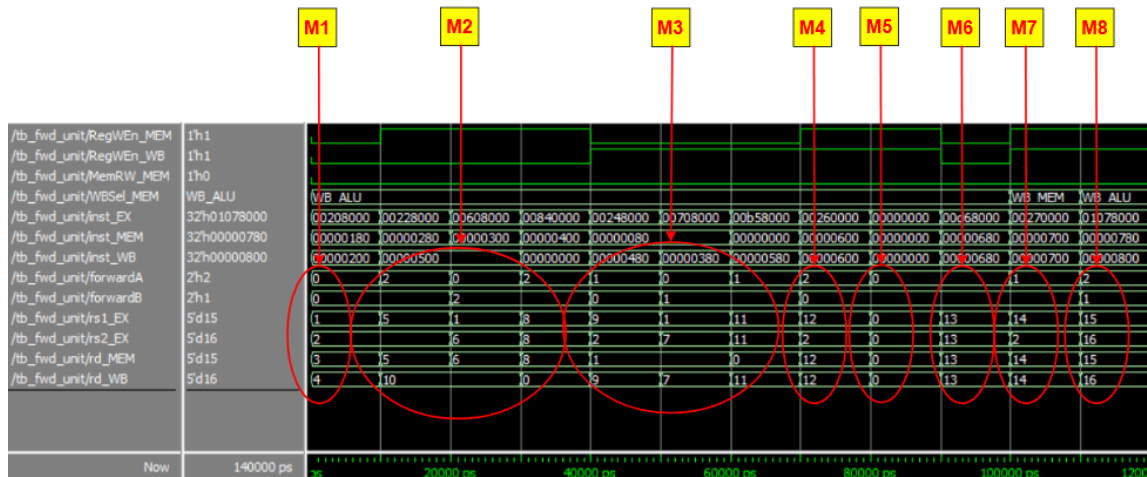
- ❖ Tại thời điểm M1 kiểm tra trạng thái reset. Khi $rst_n = 0$, ngõ ra $q = 0x13$ (bubble).
- ❖ Tại thời điểm M2 kiểm tra cập nhật dữ liệu. Sau khi $rst_n = 1$, $en = 1$ và $flush = 0$, dữ liệu $d = 32'h12345678$ được ghi vào thanh ghi tại cạnh lên clock. Ngược lại khi $en = 0$ thì dữ liệu dù có thay đổi thành $32'HAAAAAA$, ngõ ra q vẫn giữ nguyên giá trị trước đó là $32'h12345678$.
- ❖ Tại thời điểm M3 kiểm tra chức năng flush và độ ưu tiên của tín hiệu flush. Khi $flush = 1$, dù $en = 1$ và dữ liệu đầu vào thay đổi, ngõ ra q luôn được nạp giá trị $bubble = 0x13$ thay vì từ d , chứng tỏ cơ chế chèn bubble hoạt động đúng và flush có độ ưu tiên cao hơn en .
- ❖ Tại thời điểm M4 kiểm tra reset trong quá trình hoạt động. Khi $flush = 0$ và $en = 1$, dữ liệu $d = 32'h5555AAAA$ được truyền chính xác tới ngõ ra q tại cạnh lên clock. Khi $rst_n = 0$, ngõ ra q lập tức trở về giá trị $bubble = 0x13$ mà không cần chờ clock, xác nhận tính chất reset bất đồng bộ. Sau khi rst_n về lại mức 1 thì vẫn giữ giá trị trước đó do không có dữ liệu mới nào.

=> Kết quả mô phỏng cho thấy Pipeline Register hoạt động đúng theo thiết kế. Các chức năng reset bất đồng bộ, cập nhật dữ liệu ($en = 1$), giữ dữ liệu ($en = 0$), chèn bubble khi $flush = 1$ và cơ chế ưu tiên $flush > en$ đều được kiểm chứng thành công.

4.1.8 Khối Hazard Unit

- **Forwarding Unit**

Hình 4.9 mô tả dạng sóng mô phỏng của Forwarding Unit xử lý xung đột dữ liệu trong pipeline.



Hình 4.9 Dạng sóng mô tả chức năng lấy dữ liệu có chọn lọc của khối Forwarding Unit

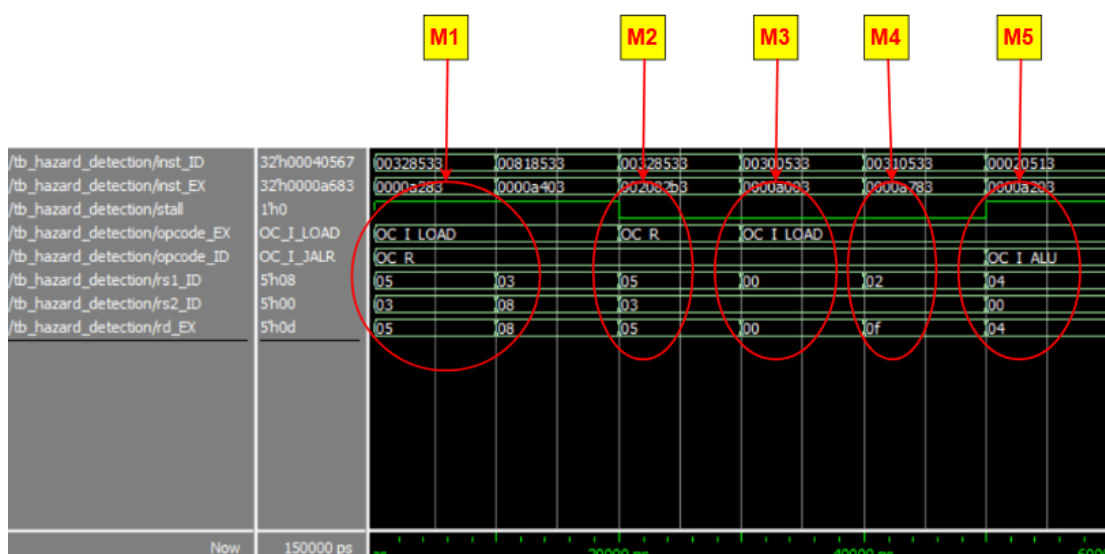
- ❖ Tại thời điểm M1 kiểm tra trường hợp không xảy ra forwarding. Khi $\text{RegWEn_MEM} = 0$ và $\text{RegWEn_WB} = 0$, kết quả $\text{forwardA} = 2'b00$ và $\text{forwardB} = 2'b00$, cho thấy ALU sử dụng trực tiếp dữ liệu từ Register File.
- ❖ Tại thời điểm M2 kiểm tra forwarding cho toán hạng từ EX/MEM. Khi $\text{rd_MEM} = \text{rs1_EX}$, $\text{RegWEn_MEM} = 1$ và $\text{WBSel_MEM} = \text{WB_ALU}$, kết quả $\text{forwardA} = 2'b10$, $\text{forwardB} = 2'b00$, xác nhận dữ liệu được chuyển tiếp từ EX/MEM đến ngõ vào A của ALU. Tương tự với toán hạng B cũng vậy khi $\text{forwardB} = 2'b10$ và $\text{forwardA} = 2'b00$. Khi rs1_EX và rs2_EX đều trùng với rd_MEM , kết quả $\text{forwardA} = 2'b10$ và $\text{forwardB} = 2'b10$, xác nhận cả hai toán hạng đều lấy từ EX/MEM.
- ❖ Tại thời điểm M3 kiểm tra forwarding cho toán hạng từ MEM/WB. Khi $\text{rd_WB} = \text{rs1_EX}$ và không tồn tại hazard với MEM, kết quả $\text{forwardA} = 2'b01$ và $\text{forwardB} = 2'b00$, cho thấy dữ liệu được lấy từ MEM/WB. Tương tự với toán hạng B cũng vậy khi $\text{forwardB} = 2'b01$ và $\text{forwardA} = 2'b00$. Khi rs1_EX và rs2_EX đều trùng với rd_WB , kết quả $\text{forwardA} = 2'b01$ và $\text{forwardB} = 2'b01$, cho thấy cả hai toán hạng đều được lấy từ MEM/WB.
- ❖ Tại thời điểm M4 kiểm tra cơ chế ưu tiên EX/MEM. Khi tồn tại hazard với MEM và WB, kết quả $\text{forwardA} = 2'b10$, xác nhận dữ liệu từ EX/MEM được ưu tiên hơn MEM/WB.

- ❖ Tại thời điểm M5 kiểm tra trường hợp $rd_MEM = 0$ hoặc $rd_WB = 0$, thì $forwardA = 2'b00$ và $forwardB = 2'b00$, cho thấy module loại bỏ đúng trường hợp ghi vào thanh ghi x0.
- ❖ Tại thời điểm M6 kiểm tra khi $RegWEn_MEM = 0$ và $RegWEn_WB = 0$. thì $forwardA = 2'b00$ và $forwardB = 2'b00$, xác nhận forwarding chỉ xảy ra khi có thao tác ghi thanh ghi.
- ❖ Tại thời điểm M7 kiểm tra trường hợp lệnh Load tại MEM, $forwardA = 2'b01$ và $forwardB = 2'b00$ và $WBSel_MEM = WB_MEM$, cho thấy dữ liệu được lấy từ MEM/WB.
- ❖ Tại thời điểm M8 kiểm tra forwarding hỗn hợp. Khi $rs1_EX$ trùng rd_MEM và $rs2_EX$ trùng rd_WB , kết quả $forwardA = 2'b10$ và $forwardB = 2'b01$, xác nhận module xử lý đồng thời nhiều nguồn forwarding khác nhau.

=> Kết quả mô phỏng cho thấy module Forwarding_Control_Logic thực hiện đúng forwarding từ EX/MEM và MEM/WB, ưu tiên EX/MEM khi có nhiều nguồn dữ liệu, loại bỏ các trường hợp không hợp lệ, đồng thời xử lý đúng trường hợp Load-Use Hazard bằng cách không forwarding dữ liệu chưa sẵn sàng từ EX/MEM.

• Hazard Detection

Hình 4.10A và hình 4.10B mô tả dạng sóng mô phỏng của Hazard Detection.



[illegible]

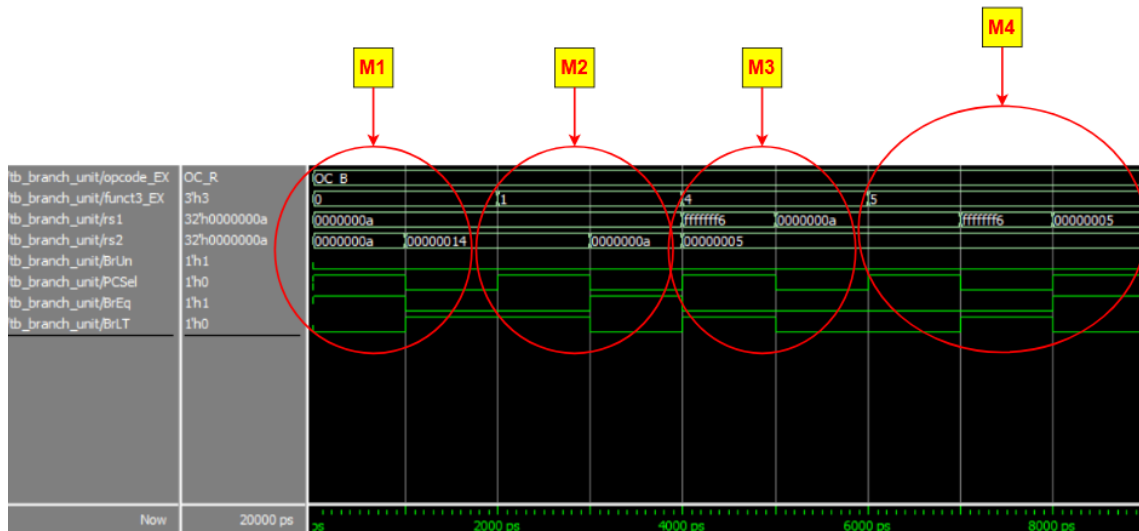
- ❖ Tại thời điểm M1 kiểm tra Load-Use Hazard. Lệnh LOAD tại EX ghi vào x5, trong khi lệnh tại ID sử dụng x5 làm rs1, mạch phát hiện hazard và đưa stall = 1. Tương tự với trường hợp sau là sử dụng x8 làm rs2, hazard được phát hiện và stall = 1.
- ❖ Tại thời điểm M2 kiểm tra trường hợp lệnh tại EX không phải LOAD. Mặc dù có trùng khớp thanh ghi giữa hai lệnh, dữ liệu có thể được xử lý bằng forwarding nên stall = 0.
- ❖ Tại thời điểm M3 kiểm tra trường hợp LOAD ghi vào x0. Do x0 luôn bằng 0 nên không được ghi, nên stall = 0.
- ❖ Tại thời điểm M4 kiểm tra trường hợp không có phụ thuộc dữ liệu giữa hai lệnh. Hazard không xảy ra và stall = 0.
- ❖ Tại thời điểm M5 kiểm tra lệnh I-type phụ thuộc dữ liệu qua rs1. Lệnh LOAD ghi vào x4, lệnh I-type sử dụng x4 làm nguồn nên stall = 1.
- ❖ Tại thời điểm M6 kiểm tra lệnh STORE phụ thuộc qua rs1 hoặc rs2. Lệnh LOAD ghi vào x6, STORE sử dụng x6 làm địa chỉ cơ sở nên stall = 1. Tương tự với rs2 khi dùng x7.

- ❖ Tại thời điểm M7 kiểm tra lệnh BRANCH phụ thuộc qua rs1 hoặc rs2. Lệnh LOAD ghi vào x9, BRANCH sử dụng x9 để so sánh điều kiện nên stall = 1. Tương tự với rs2.
- ❖ Tại thời điểm M8 kiểm tra các lệnh mặc định không stall:
 - Lệnh JAL không sử dụng rs1 hoặc rs2 nên không stall.
 - Lệnh LUI chỉ sử dụng immediate nên không stall.
 - Lệnh AUIPC không đọc rs1 hoặc rs2 nên stall = 0.
- ❖ Tại thời điểm M9 kiểm tra lệnh JALR.
 - Lệnh LOAD ghi vào x13, JALR sử dụng x13 làm địa chỉ nhảy nên stall = 1.
 - Trường hợp JALR không phụ thuộc dữ liệu. Thanh ghi nguồn của JALR khác thanh ghi đích của LOAD nên stall = 0.

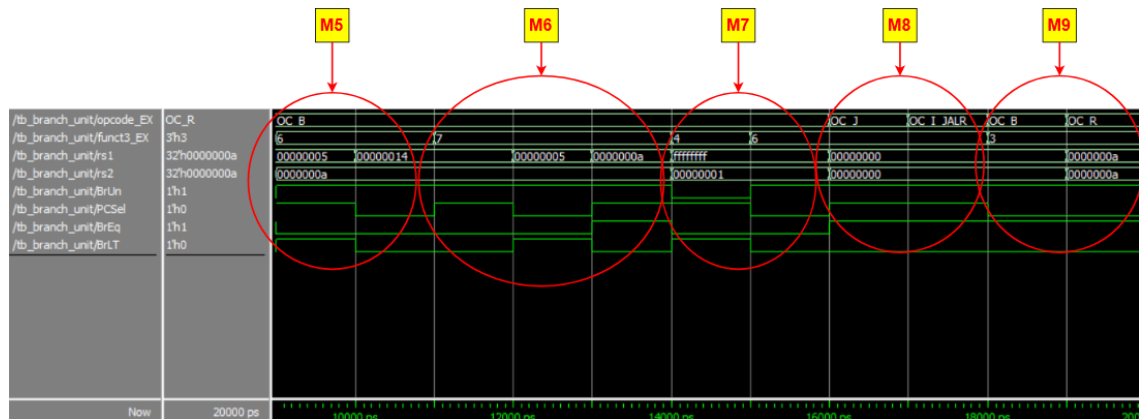
=> Kết quả mô phỏng cho thấy module Hazard Detection hoạt động đúng theo yêu cầu. Mạch phát hiện chính xác các trường hợp Load-Use Hazard và kích hoạt stall khi cần thiết, đồng thời không tạo stall đối với các trường hợp không có phụ thuộc dữ liệu.

- **Branch Unit**

Hình 4.11A và hình 4.11B mô tả dạng sóng mô phỏng của Branch Control có nhiệm vụ xử lý nhánh trong kiến trúc pipeline, thể hiện các tín hiệu so sánh BrEq và BrLT thay đổi theo giá trị hai toán hạng đầu vào để xác định điều kiện nhánh.



Hình 4.11A Dạng sóng mô tả chức năng xử lý nhánh của khối branch Unit từ 0 – 9ns



Hình 4.11B Dạng sóng mô tả chức năng xử lý nhánh của khối branch Unit từ 9 – 20ns

- ❖ Tại thời điểm M1 kiểm tra lệnh BEQ. Khi opcode_EX = OC_B và funct3_EX = 3'h0, rs1 = rs2 nên PCSel = 1 (nhảy nhánh), ngược lại PCSel = 0.
- ❖ Tại thời điểm M2 kiểm tra lệnh BNE. Với funct3_EX = 3'h1, khi rs1 ≠ rs2 thì PCSel = 1, ngược lại PCSel = 0.
- ❖ Tại thời điểm M3 kiểm tra lệnh BLT ở chế độ so sánh có dấu (BrUn = 0). Khi rs1 < rs2 thì PCSel = 1, ngược lại PCSel = 0.
- ❖ Tại thời điểm M4 kiểm tra lệnh BGE ở chế độ so sánh có dấu (BrUn = 0). Khi rs1 ≥ rs2 thì PCSel = 1, còn rs1 < rs2 thì PCSel = 0.

- ❖ Tại thời điểm M5 kiểm tra lệnh BLTU với $BrUn = 1$. Khi $rs1 < rs2$ theo dạng không dấu thì $PCSel = 1$, ngược lại $PCSel = 0$.
- ❖ Tại thời điểm M6 kiểm tra lệnh BGEU với $BrUn = 1$. Khi $rs1 \geq rs2$ theo dạng không dấu thì $PCSel = 1$, ngược lại $PCSel = 0$.
- ❖ Tại thời điểm M7 kiểm tra sự khác biệt giữa so sánh có dấu và không dấu với $rs1 = 32'hFFFFFFF$ và $rs2 = 1$. Ở chế độ signed ($BrUn = 0$), giá trị $rs1$ được hiểu là -1 nên $PCSel = 1$. Ở chế độ unsigned ($BrUn = 1$), $rs1$ là số lớn nhất 32 bit nên $PCSel = 0$.
- ❖ Tại thời điểm M8 kiểm tra lệnh JAL và JALR. Khi $opcode_EX = OC_J$ hoặc $opcode_EX = OC_I_JALR$, $PCSel$ luôn bằng 1.
- ❖ Tại thời điểm M9 kiểm tra trường hợp $funct3_EX$ không hợp lệ và tra opcode không thuộc nhóm Branch hoặc Jump. Tín hiệu $PCSel$ luôn bằng 0.

=> Kết quả mô phỏng cho thấy module Branch_Unit hoạt động chính xác với các lệnh điều khiển luồng của RV32I. Các phép so sánh signed/unsigned đều cho kết quả đúng. Đồng thời, các opcode hoặc funct3 không hợp lệ đều được xử lý an toàn bằng cách giữ $PCSel = 0$.

4.2 Kết quả mô phỏng toàn hệ thống

Sau khi hoàn thành việc thiết kế và kiểm chứng các khối chức năng độc lập, toàn bộ bộ xử lý RV32I pipeline 5 tầng được tích hợp và tiến hành mô phỏng ở mức hệ thống. Chương trình kiểm thử được sử dụng nhằm đánh giá khả năng phối hợp hoạt động của các khối chức năng trong quá trình thực thi lệnh thực tế.

Cụ thể chương trình được dùng để kiểm thử là chương trình tính tổng các số chẵn khi nạp vào 10 số bất kì (Trong chương trình này đã nạp từ số 1 đến số 10 vào bộ nhớ để thực thi). Chương trình không chỉ dùng để kiểm tra kết quả tính toán của bộ xử lý mà còn giúp kiểm thử nhiều chức năng khác nhau của kiến trúc RV32I. Trong quá trình thực hiện, chương trình sử dụng hầu hết các loại lệnh opcode có trong kiến trúc RV32I một cách thực tế nhất như R-type (dùng cho tính tổng, các lệnh ALU phụ), U-Type, I-Type, Store (khởi tạo mảng, khởi tạo biến),

Load (đọc phần tử), Branch (kiểm tra chắn lẻ, điều khiển vòng lặp) và lệnh hệ thống dùng để kết thúc chương trình, qua đó các khối chức năng như Control Unit, Register File, ALU, Data Memory... và các thanh ghi pipeline đều được tham gia vào quá trình xử lý lệnh.

```
BEGIN

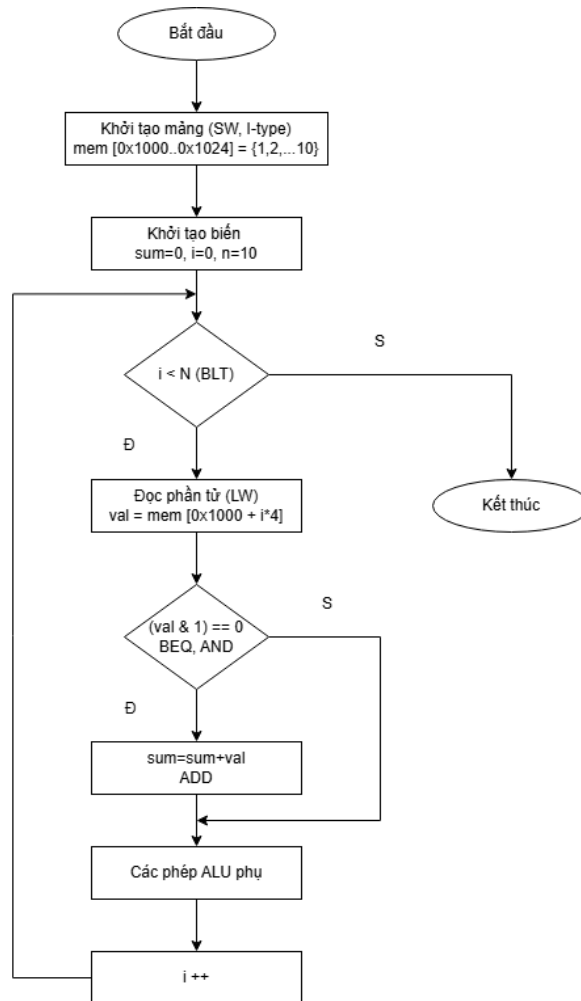
    // Khởi tạo mảng
    mem[0x1000] = 1
    mem[0x1004] = 2
    mem[0x1008] = 3
    mem[0x100C] = 4
    mem[0x1010] = 5
    mem[0x1014] = 6
    mem[0x1018] = 7
    mem[0x101C] = 8
    mem[0x1020] = 9
    mem[0x1024] = 10

    // Khởi tạo biến
    sum = 0
    i = 0
    N = 10

    WHILE (i < N) DO
        // Đọc phần tử từ bộ nhớ
        val = mem [0x1000 + i*4]
```

```
// Kiểm tra số chẵn  
  
IF ((val AND 1) == 0) THEN  
  
    sum = sum + val  
  
END IF  
  
// Các phép ALU phụ  
  
OR, XOR, AND, SUB  
  
// Tăng biến đếm địa chỉ  
  
i = i + 1  
  
END WHILE  
  
END
```

Hình 4.12 Mã giả (Pseudo-code) của chương trình kiểm thử toàn hệ thống



Hình 4.13 Lưu đồ của chương trình kiểm thử toàn hệ thống

Hình 4.12 thể hiện mã giả (Pseudo-code) và hình 4.13 thể hiện lưu đồ của chương trình, còn bảng 4.1 liệt kê chi tiết các lệnh được sử dụng để hoàn thành chương trình. Ngoài ra, việc sử dụng vòng lặp và các lệnh truy cập dữ liệu liên tiếp cũng tạo ra các trường hợp phụ thuộc dữ liệu và nhánh rẽ trong pipeline. Điều này cho phép quan sát và kiểm tra hoạt động của các cơ chế xử lý hazard như forwarding, stall và branch prediction. Do đó, chương trình có thể xem như một bài kiểm thử tổng hợp, giúp đánh giá tương đối đầy đủ khả năng hoạt động của bộ xử lý RV32I được thiết kế.

Bảng 4.1 Mã Assembly dùng mô phỏng cho chương trình kiểm thử toàn hệ thống

Địa chỉ	Mã hex	Mã Assembly	Chú thích
0x00	0x000010B7	LUI x1, 0x1	Gán giá trị cho x1=0x1 (4096)
0x04	0x00100113	ADDI x2, x0, 1	Thực hiện phép cộng x0 với 1, nhưng vì x0 luôn = 0 nên tương đương với gán x2=1
0x08	0x0020A023	SW x2, 0(x1)	Lưu giá trị x2 vừa gán vào data memory với địa chỉ x1+0=4096
0x0C	0x00200113	ADDI x2, x0, 2	Thực hiện phép cộng x0 với 2, nhưng vì x0 luôn = 0 nên tương đương với gán x2=2
0x10	0x0020A223	SW x2, 4(x1)	Lưu giá trị x2 vừa gán vào data memory với địa chỉ x1+4=4100
0x14	0x00300113	ADDI x2, x0, 3	Thực hiện phép cộng x0 với 3, nhưng vì x0 luôn = 0 nên tương đương với gán x2=3
0x18	0x0020A423	SW x2, 8(x1)	Lưu giá trị x2 vừa gán vào data memory với địa chỉ x1+8=4104
0x1C	0x00400113	ADDI x2, x0, 4	Thực hiện phép cộng x0 với 4, nhưng vì x0 luôn = 0 nên tương đương với gán x2=4
0x20	0x0020A623	SW x2, 12(x1)	Lưu giá trị x2 vừa gán vào data memory với địa chỉ x1+12=4108

0x24	0x00500113	ADDI x2, x0, 5	Thực hiện phép cộng x0 với 5, nhưng vì x0 luôn = 0 nên tương đương với gán x2=5
0x28	0x0020A823	SW x2, 16(x1)	Lưu giá trị x2 vừa gán vào data memory với địa chỉ $x1+16=4112$
0x2C	0x00600113	ADDI x2, x0, 6	Thực hiện phép cộng x0 với 6, nhưng vì x0 luôn = 0 nên tương đương với gán x2=6
0x30	0x0020AA23	SW x2, 20(x1)	Lưu giá trị x2 vừa gán vào data memory với địa chỉ $x1+20=4116$
0x34	0x00700113	ADDI x2, x0, 7	Thực hiện phép cộng x0 với 7, nhưng vì x0 luôn = 0 nên tương đương với gán x2=7
0x38	0x0020AC23	SW x2, 24(x1)	Lưu giá trị x2 vừa gán vào data memory với địa chỉ $x1+24=4120$
0x3C	0x00800113	ADDI x2, x0, 8	Thực hiện phép cộng x0 với 8, nhưng vì x0 luôn = 0 nên tương đương với gán x2=8
0x40	0x0020AE23	SW x2, 28(x1)	Lưu giá trị x2 vừa gán vào data memory với địa chỉ $x1+28=4124$

0x44	0x00900113	ADDI x2, x0, 9	Thực hiện phép cộng x0 với 9, nhưng vì x0 luôn = 0 nên tương đương với gán x2=9
0x48	0x0220A023	SW x2, 32(x1)	Lưu giá trị x2 vừa gán vào data memory với địa chỉ $x1+32=4128$
0x4C	0x00A00113	ADDI x2, x0, 10	Thực hiện phép cộng x0 với 10, nhưng vì x0 luôn = 0 nên tương đương với gán x2=10
0x50	0x0220A223	SW x2, 36(x1)	Lưu giá trị x2 vừa gán vào data memory với địa chỉ $x1+36=4132$
0x54	0x00000513	ADDI x10, x0, 0	Gán x10=0 (biến đếm)
0x58	0x00A00593	ADDI x11, x0, 10	Gán x11=10 (giới hạn của biến đếm)
0x5C	0x00000A13	ADDI x20, x0, 0	Gán x20=0 (biến lưu tổng các số chẵn)
0x60	0x00251613	SLLI x12, x10, 2	Gán x12 với phép x10 dịch trái 2 bit (biến thay đổi địa chỉ bộ nhớ)
0x64	0x00C086B3	ADD x13, x1, x12	Gán x13 với phép $x1+x12$ (biến truy cập địa chỉ)
0x68	0x0006A703	LW x14, 0(x13)	Lấy ra giá trị x14 tương đương với $mem(x13+0)$

0x6C	0x00177793	ANDI x15, x14, 1	Thực hiện phép and giữa x14 với 1 để xem có phải số chẵn không (tính chất số chia hết cho 2)
0x70	0x00079463	BNE x15, x0, 0x78	Nếu chia hết 2 thì x15 sẽ =0, so sánh với x0 nếu bằng nhau thì sẽ không nhảy mà thực hiện phép tính tiếp theo để tính tổng số chẵn, còn không bằng thì nhảy đến phép tính ở pc=0x78
0x74	0x00EA0A33	ADD x20, x20, x14	Biến x20 sẽ cộng dồn vào nếu x14 là số chẵn
0x78	0x00A74833	XOR x16, x14, x10	x16 lưu kết quả của phép xor giữa x14 và x10
0x7C	0x00F868B3	OR x17, x16, x15	x17 lưu kết quả của phép or giữa x16 và x15
0x80	0x00E8F933	AND x18, x17, x14	x18 lưu kết quả của phép xor giữa x17 và x14
0x84	0x40F909B3	SUB x19, x18, x15	x19 lưu kết quả của phép xor giữa x18 và x15
0x88	0x00150513	ADDI x10, x10, 1	Tăng biến đếm sau khi đã xong 1 vòng
0x8C	0xFCB54AE3	BLT x10, x11, 0x60	So sánh biến đếm với giới hạn đã cho, nếu còn nhỏ hơn thì còn trong vòng lặp, còn bằng nhau thì thoát chương trình

- IF: Lệnh `addi x2, x0, 2` tại địa chỉ `0x0C` được nạp.
- ID: Lệnh `sw x2, 0(x1)` được giải mã. `opcode_ID = OC_S`, `rsR1 = x1`, `rsR2 = x2`, `imm = 0x00`, `RegWEn = 0`. Tại thời điểm này `x1` chưa được ghi xong (đang ở WB), cơ chế forwarding sẽ xử lý ở giai đoạn EX.
- EX: Lệnh `addi x2, x0, 1` thực thi. `ALUSel = ALU_ADD`, `A = 0`, `B = 1`, `alu = 0x01`.
- MEM: Lệnh lui `x1, 0x1` đi qua giai đoạn MEM. `MemRW_MEM = 0`.

❖ Tại điểm M6

- IF: Lệnh `sw x2, 4(x1)` tại địa chỉ `pc = 0x10` được nạp
- ID: Lệnh `addi x2, x0, 2` được giải mã. `opcode_ID = OC_I_ALU`, `rsR1 = x0`, `rsW = x2`, `imm = 0x02`, `dataR1 = 0x00`, `RegWEn = 1`.
- EX: Lệnh `sw x2, 0(x1)` thực thi. `ALUSel = ALU_ADD`, `dataR1_fwd = 0x1000` (forward từ WB của lui), `dataR2_fwd = 0x01` (forward từ MEM của `addi x2, x0, 1`), `forwardA = 2'b01`, `forwardB = 2'b10`. `A = 0x1000`, `B = 0x00`, `alu = 0x1000`.
- MEM: Lệnh `addi x2, x0, 1` đi qua MEM. `MemRW_MEM = 0`.
- WB: Lệnh lui `x1, 0x1` ghi kết quả. `rd_WB = x1`, `WBdata = 0x1000`.

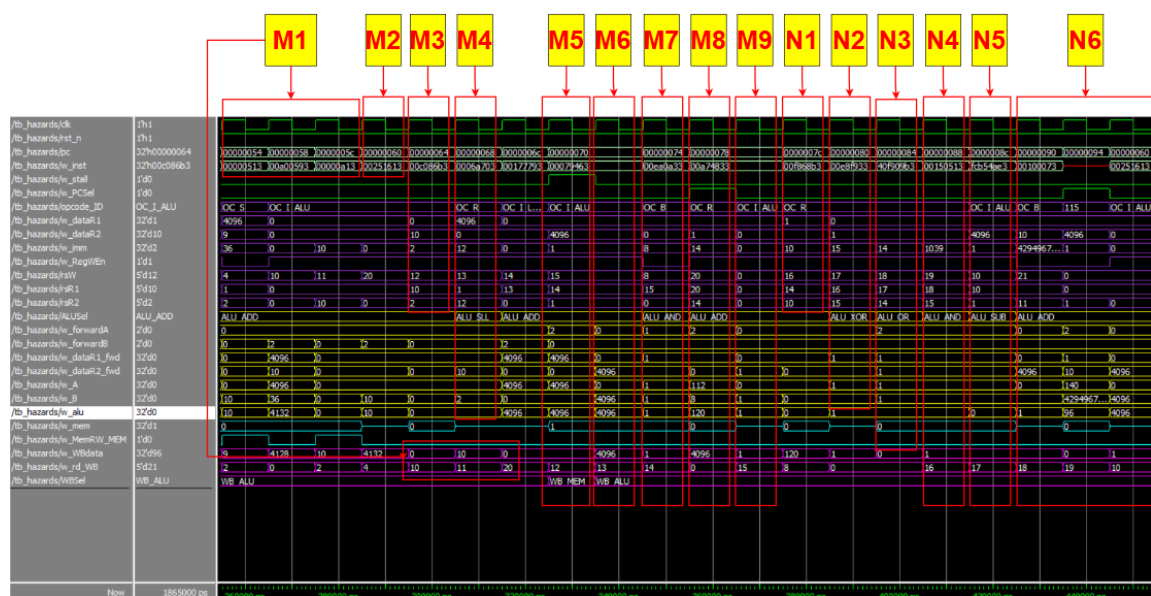
❖ Tại điểm M7

- ❖ IF: Lệnh `addi x2, x0, 3` tại địa chỉ `0x14` được nạp.
- ❖ ID: Lệnh `sw x2, 4(x1)` được giải mã. `opcode_ID = OC_S`, `rsR1 = x1`, `rsR2 = x2`, `imm = 0x04`, `RegWEn = 0`.
- EX: Lệnh `addi x2, x0, 2` thực thi. `ALUSel = ALU_ADD`, `A = 0`, `B = 2`, `alu = 0x02`.
- MEM: Lệnh `sw x2, 0(x1)` truy cập bộ nhớ. `MemRW_MEM = 1`, `addr = 0x1000` (`32'd4096`). Giá trị 1 được ghi vào `mem[0x1000]`.
- WB: Lệnh `addi x2, x0, 1` ghi kết quả. `rd_WB = x2`, `WBdata = 0x001`.

- ❖ Từ 105ns – 255ns: Tiếp tục khởi tạo mảng (lệnh 0x0C đến 0x50). Tại giai đoạn MEM của mỗi lệnh sw, các giá trị còn lại từ 2 đến 10 sẽ tương ứng được ghi vào ô nhớ có địa chỉ từ mem[0x1004] cho đến địa chỉ mem[0x1024] trong Data Memory:

- 105ns–115ns MEM: mem[0x1004] ← 2
- 115ns–125ns MEM: mem[0x1008] ← 3
- 125ns–135ns MEM: mem[0x100C] ← 4
- 135ns–145ns MEM: mem[0x1010] ← 5
- 145ns–155ns MEM: mem[0x1014] ← 6
- 155ns–165ns MEM: mem[0x1018] ← 7
- 165ns–175ns MEM: mem[0x101C] ← 8
- 175ns–185ns MEM: mem[0x1020] ← 9
- 185ns–195ns MEM: mem[0x1024] ← 10

Hình 4.14C mô tả dạng sóng hoạt động ở vòng lặp lần 1 ($x_{10} = 0$, $x_{14} = 1$ — số lẻ)



Hình 4.14C Dạng sóng hoạt động toàn hệ thống từ 255 – 445ns

❖ **Tại điểm M1, khởi tạo biến vòng lặp**

- 255ns–265ns IF: Lệnh addi x10, x0, 0 tại 0x54 được nạp. Biến đếm x10=0 tương đương rd_WB=10 và WBdata=0.
- 265ns–275ns IF: Lệnh addi x11, x0, 10 tại 0x58 được nạp. Vòng lặp x11 = 10 tương đương rd_WB=11 và WBdata=10.
- 275ns–285ns IF: Lệnh addi x20, x0, 0 tại 0x5C được nạp. Biến tính tổng số chẵn x20 tương đương rd_WB=20 và WBdata=0.

❖ **Tại điểm M2, vòng lặp lần 1 (x10 = 0, phần tử x14 = 1 — số lẻ).**

- IF: Lệnh slli x12, x10, 2 tại 0x60 được nạp, là lệnh đầu tiên của vòng lặp.

❖ **Tại điểm M3**

- IF: Lệnh add x13, x1, x12 tại 0x64 được nạp.
- ID: Lệnh slli x12, x10, 2 được giải mã. opcode_ID = OC_I_ALU, rsR1 = x10, rsW = x12, dataR1 = 0x00 (x10=0), imm = 2, RegWEn = 1.

❖ **Tại điểm M4**

- IF: Lệnh lw x14, 0(x13) tại 0x68 được nạp.
- ID: Lệnh add x13, x1, x12 được giải mã. opcode_ID = OC_R, rsR1 = x1, rsR2 = x12, rsW = x13, dataR1 = 0x1000, dataR2 = 0x00, RegWEn = 1.
- EX: Lệnh slli x12, x10, 2 thực thi. ALUSel = ALU_SLL, A = 0, B = 2, alu = 0x00

❖ **Tại điểm M5, Load-Use Hazard (Stall)**

- IF: Lệnh bne x15, x0, 0x78 tại 0x70 được nạp.
- ID: Lệnh andi x15, x14, 1 bị giữ lại (stall), chưa tiến sang EX.
- EX: Lệnh lw x14, 0(x13). ALUSel = ALU_ADD, A = 0x1000, B = 0, alu = 0x1000.
- Hazard Detection Unit phát hiện lệnh lw x14 tại EX và lệnh andi x15, x14, 1 tại ID đang đọc x14. Do đây là load-use hazard, pipeline bị treo 1 chu kỳ, thanh ghi IF/ID giữ nguyên lệnh andi x15, x14, 1, pc không tăng.

- MEM: Lệnh add x13, x1, x12 đi qua MEM. MemRW_MEM = 0, alu_MEM = 0x1000.
- WB: Lệnh slli x12, x10, 2 ghi kết quả. rd_WB = x12, WBdata = 0x00.

❖ Tại điểm M6

- IF: Lệnh bne x15, x0, 0x78 tại 0x70 bị giữ lại.
- ID: Lệnh andi x15, x14, 1 được giải mã sau khi stall kết thúc. opcode_ID = OC_I_ALU, rsR1 = x14, rsW = x15, imm = 0x01, RegWEn = 1.
- EX: Bong bóng NOP đi qua EX, không có tác động.
- MEM: Lệnh lw x14, 0(x13) truy cập bộ nhớ. MemRW_MEM = 0, addr = 0x1000, mem = 0x01 (đọc giá trị 1 từ mem[0x1000]).
- WB: Lệnh add x13, x1, x12 ghi kết quả. rd_WB = x13, WBdata = 0x00001000.

❖ Tại điểm M7

- IF: Lệnh add x20, x20, x14 tại 0x74 được nạp.
- ID: Lệnh bne x15, x0, 0x78 được giải mã. opcode_ID = OC_B, rsR1 = x15, rsR2 = x0, imm = 8 (offset +8). w_RegWEn = 0.
- EX: Lệnh andi x15, x14, 1 thực thi. ALUSel = ALU_AND, forwardA = 2'b01 (forward x14 từ WB của lw), dataR1_fwd = 0x00000001, B = 0x00000001, alu = 0x00000001 (1 AND 1 = 1, x14=1 là số lẻ).
- MEM: Bong bóng NOP đi qua MEM, không có tác động.
- WB: Lệnh lw x14, 0(x13) ghi kết quả. rd_WB = x14, WBdata = 0x00000001. Thanh ghi x14 = 0x00000001.

❖ Tại điểm M8, nhánh bne được thực hiện (Taken), flush

- EX: Lệnh bne x15, x0, 0x78 thực thi. forwardA = 2'b10 (forward x15=1 từ MEM), dataR1_fwd = 0x01, dataR2_fwd = 0x00 (x0=0). Vì x15 khác x0, điều kiện nhánh thỏa mãn: PCSel = 1, pc nhảy đến 0x78. Hai lệnh đã nạp

sau bne (add x20 tại 0x74 và xor x16 tại 0x78) bị flush khỏi giai đoạn IF và ID, thay bằng bong bóng NOP.

- MEM: Lệnh andi x15, x14, 1 đi qua MEM. MemRW_MEM = 0, alu_MEM = 0x01.
- WB: Bong bóng NOP đi qua WB, không có tác động.

❖ **Tại điểm M9, phần còn lại vòng lặp 1 (sau nhánh bne taken)**

- IF: Lệnh xor x16, x14, x10 tại 0x78 được nạp (PC đã nhảy đến 0x78).
- ID: Bong bóng NOP (flush).
- EX: Bong bóng NOP (flush).
- MEM: Lệnh bne x15, x0, 0x78 đi qua MEM. MemRW_MEM = 0.
- WB: Lệnh andi x15, x14, 1 ghi kết quả. w_rd_WB = x15, WBdata = 0x01.

❖ **Tại điểm N1**

- IF: Lệnh or x17, x16, x15 tại 0x7C được nạp.
- ID: Lệnh xor x16, x14, x10 được giải mã. opcode_ID = OC_R, rsR1 = x14, rsR2 = x10, rsW = x16, dataR1 = 0x01, dataR2 = 0x00, RegWEn = 1.

❖ **Tại điểm N2**

- IF: Lệnh and x18, x17, x14 tại 0x80 được nạp.
- ID: Lệnh or x17, x16, x15 được giải mã. opcode_ID = OC_R, rsR1 = x16, rsR2 = x15, rsW = x17, RegWEn = 1.
- EX: Lệnh xor x16, x14, x10 thực thi. ALUSel = ALU_XOR, dataR1_fwd = 0x01 (x14=1), w_dataR2_fwd = 0x00 (x10=0), alu = 0x01 (1 XOR 0 = 1).

❖ **Tại điểm N3**

- IF: Lệnh sub x19, x18, x15 tại 0x84 được nạp.
- ID: Lệnh and x18, x17, x14 được giải mã. opcode_ID = OC_R, rsR1 = x17, rsR2 = x14, rsW = x18, RegWEn = 1.

- EX: Lệnh or x17, x16, x15 thực thi. ALUSel = ALU_OR, forwardA = 2'b10 (forward x16=1 từ MEM), dataR1_fwd = 0x01, dataR2_fwd = 0x01 (x15=1), alu = 0x01.
- MEM: Lệnh xor x16, x14, x10 đi qua MEM. MemRW_MEM = 0.

❖ Tại điểm N4

- IF: Lệnh addi x10, x10, 1 tại 0x88 được nạp.
- ID: Lệnh sub x19, x18, x15 được giải mã. opcode_ID = OC_R, rsR1 = x18, rsR2 = x15, rsW = x19, RegWEn = 1.
- EX: Lệnh and x18, x17, x14 thực thi. ALUSel = ALU_AND, forwardA = 2'b10 (forward x17=1), dataR1_fwd = 0x01, dataR2_fwd = 0x01 (x14=1), alu = 0x01/
- MEM: Lệnh or x17, x16, x15 đi qua MEM. alu_MEM = 0x01.
- WB: Lệnh xor x16, x14, x10 ghi kết quả. w_rd_WB = x16, w_WBdata = 0x01.

❖ Tại điểm N5

- IF: Lệnh blt x10, x11, 0x60 tại 0x8C được nạp.
- ID: Lệnh addi x10, x10, 1 được giải mã. opcode_ID = OC_I_ALU, rsR1 = x10, rsW = x10, imm = 1, dataR1 = 0x00, RegWEn = 1.
- EX: Lệnh sub x19, x18, x15 thực thi. ALUSel = ALU_SUB, forwardA = 2'b10 (forward x18=1), dataR1_fwd = 0x01, dataR2_fwd = 0x01 (x15=1), alu = 0x00
- MEM: Lệnh and x18, x17, x14 đi qua MEM. alu_MEM = 0x01.
- WB: Lệnh or x17, x16, x15 ghi kết quả. rd_WB = x17, WBdata = 0x01.

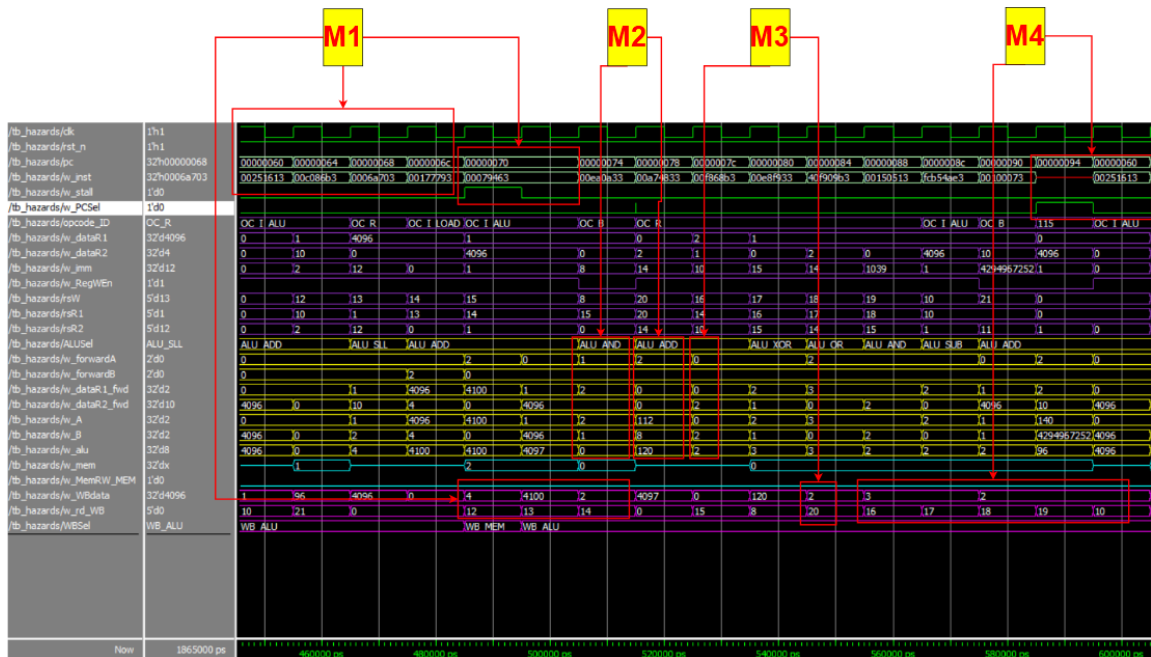
❖ Tại điểm N6, nhánh blt taken, kết thúc vòng 1

- EX: Lệnh blt x10, x11, 0x60 thực thi. forwardA = 2'b10 (forward x10=1 từ MEM của addi), dataR1_fwd = 0x01, dataR2_fwd = 0x0A (x11=10). Vì 1

< 10, điều kiện thỏa mãn: PCSel = 1, pc nhảy về 0x60. Hai lệnh vừa nạp sau blt bị flush.

- WB (cuối vòng 1): thanh ghi cập nhật sau vòng 1 $x_{20} = 0$ (không tích lũy).

Hình 4.14D mô tả dạng sóng hoạt động ở vòng lặp lần 2 ($x_{10} = 1$, $x_{14} = 2$ — số chẵn)



Hình 4.14D Dạng sóng hoạt động toàn hệ thống từ 445 - 595ns

❖ Vòng lặp lần 2 ($x_{10} = 1$, phần tử $x_{14} = 2$ — số chẵn). Từ 445ns – 505ns: $slli \rightarrow add \rightarrow lw \rightarrow (stall) \rightarrow andi$

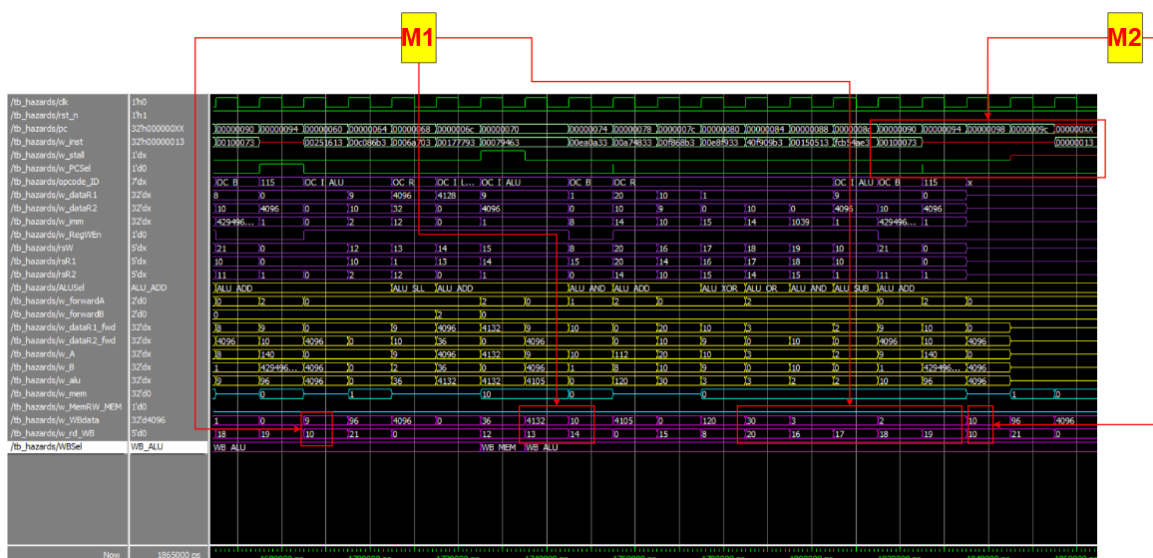
- Các lệnh $slli\ x_{12}, x_{10}, 2$; $add\ x_{13}, x_1, x_{12}$ và $lw\ x_{14}, 0(x_{13})$ lần lượt được nạp và thực thi tương tự vòng 1. Lần này $x_{10} = 1$ nên $x_{12} = 4$, $x_{13} = 0x1004$, lw đọc $mem[0x1004] = 2$. Load-use stall vẫn xảy ra tại chu kỳ tương ứng ($stall = 1$).

❖ Từ 505ns – 525ns — $andi, bne\ not\ taken$

- EX: Lệnh $andi\ x_{15}, x_{14}, 1$ thực thi. $dataR1_fwd = 0x02$ ($x_{14}=2$), $B = 0x01$, $alu = 0x00$ ($2\ AND\ 1 = 0$, số chẵn). $x_{15} = 0$.

- EX (chu kỳ tiếp): Lệnh bne x15, x0, 0x78 thực thi. dataR1_fwd = 0x00 (x15=0), dataR2_fwd = 0x00 (x0=0). Vì x15 = x0, điều kiện nhánh không thỏa mãn: PCSel = 0, pipeline tiếp tục tuần tự sang 0x74. Không có flush.
- ❖ Từ 525ns – 555ns
- EX: Lệnh add x20, x20, x14 tại 0x74 thực thi. ALUSel = ALU_ADD, dataR1_fwd = 0x00 (x20=0), dataR2_fwd = 0x02 (x14=2), alu = 0x02.
- WB (sau đó): rd_WB = x20, WBdata = 0x02.
- ❖ Phần xor → or → and → sub → addi → blt diễn ra tương tự vòng 1, với các giá trị tính toán tương ứng cho x14=2, x10=1. Nhánh blt (x10=2 < 10) vẫn taken, pc quay về 0x60.
- ❖ Các vòng lặp từ 3 đến 10 diễn ra theo hai kịch bản lặp lại:
 - Vòng lẻ (x10 = 0,2,4,6,8 → x14 = 1,3,5,7,9): andi trả về x15=1, bne taken → flush 2 chu kỳ, bỏ qua add x20.
 - Vòng chẵn (x10 = 1,3,5,7,9 → x14 = 2,4,6,8,10): andi trả về x15=0, bne not taken → thực thi add x20, x20, x14, tích lũy tổng.

Hình 4.14E mô tả dạng sóng hoạt động ở vòng lặp lần cuối (x10 = 9, x14 = 10 — số chẵn)

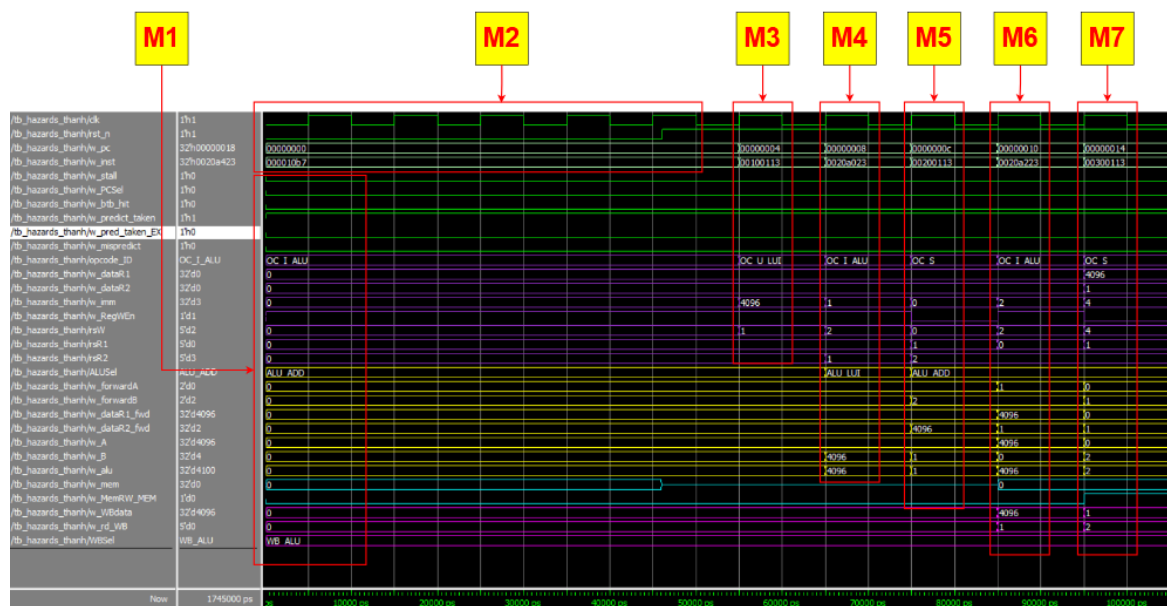


Hình 4.14E Dạng sóng hoạt động toàn hệ thống từ 1665 - 1865ns

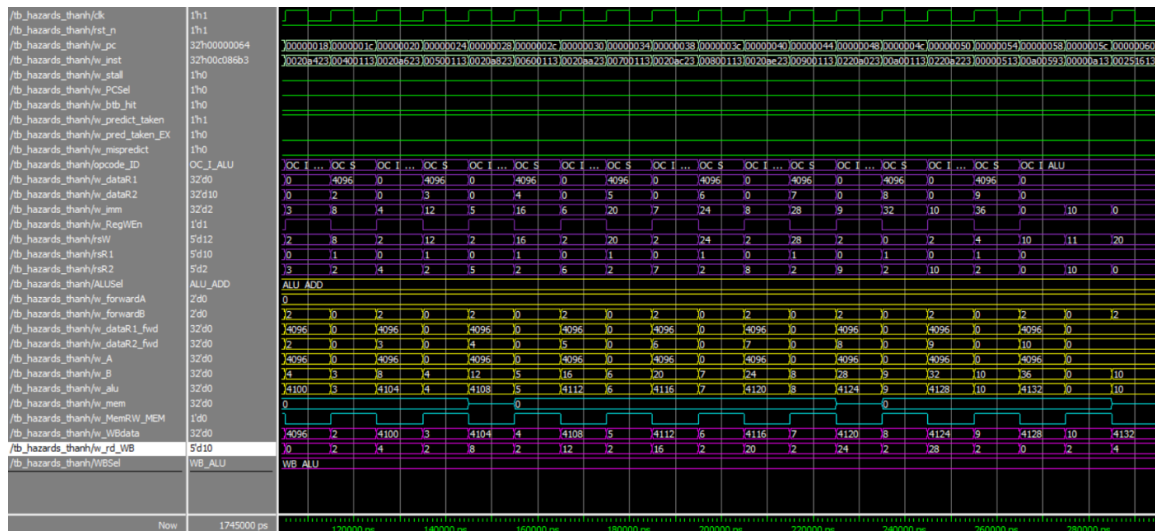
- ❖ Vòng lặp cuối ($x10 = 9$, $x14 = 10$ — số chẵn): bne not taken, add x20 thực thi ($x20 = 2+4+6+8+10 = 30 = 0x1E$). Lệnh blt x10, x11, 0x60 thực thi với $x10 = 10$, $x11 = 10$: vì $10 < 10$ là sai, PCSel = 0, pipeline không nhảy, tiếp tục tuần tự sang 0x90.
- ❖ Lệnh kết thúc - ecall tại 0x90: IF → WB: Lệnh đi qua pipeline. opcode_ID = 115 (EBREAK), w_RegWEn = 0, w_MemRW_MEM = 0. Không có ghi thanh ghi hay bộ nhớ. Chương trình kết thúc với kết quả cuối cùng: $x20 = 0x1E$ (30 thập phân), xác nhận tổng các số chẵn từ 1 đến 10 được tính đúng.

4.2.2 Pipeline 5 tầng có tích hợp cơ chế dự đoán nhánh

Hình 4.15A và hình 4.15B biểu diễn các tín hiệu hoạt động ở giai đoạn khởi tạo mảng.



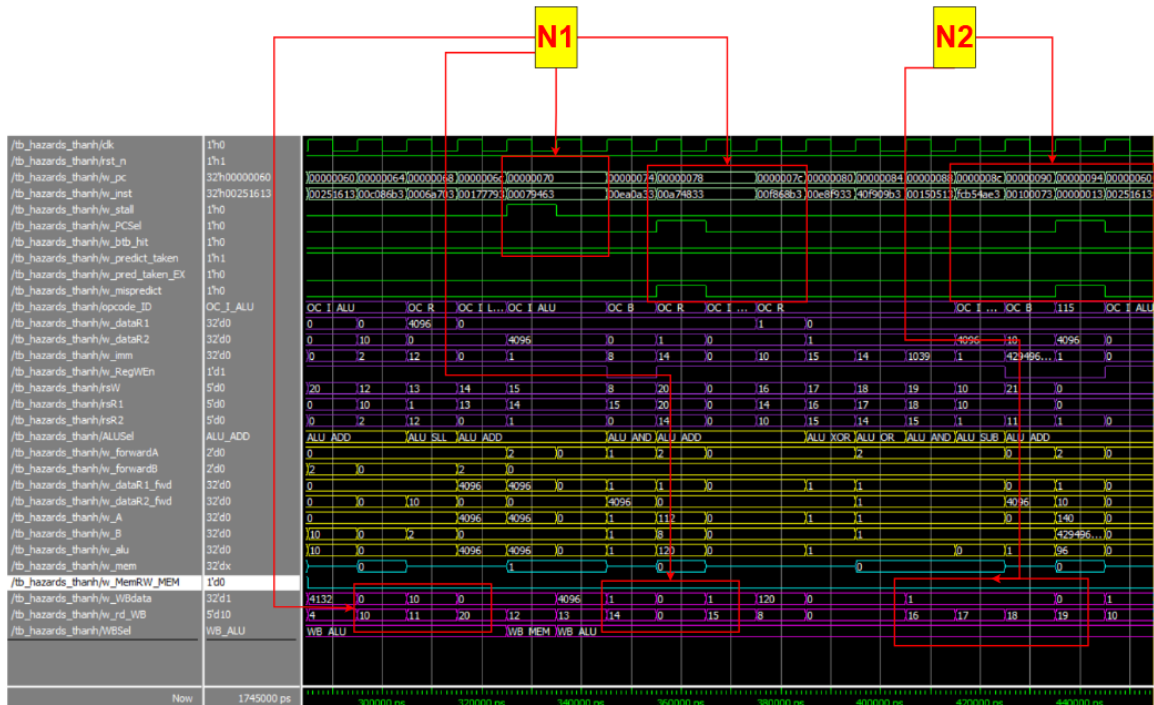
Hình 4.15A Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 0 - 105ns



Hình 4.15B Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 175 - 295ns

- ❖ Các lệnh U-Type, I-Type và S-Type hoạt động chính xác, giúp việc nạp dữ liệu vào bộ nhớ đúng với mong muốn của lệnh và giống hoàn toàn với kiến trúc RISC-V vừa được kiểm chứng. Tại các điểm từ M1 đến M7 đã cho thấy 2 lệnh là lui x1, 0x1 và addi x2, x0, 0 cũng như reset hệ thống đã hoạt động đúng theo pipeline 5 tầng tương tự kiến trúc RV32I khi chưa tích hợp bộ dự đoán nhánh. Do đó khi kiểm chứng các waveform sau thì nhóm sẽ tập trung phân tích về các tín hiệu có trong bộ dự đoán nhánh và xem chúng cho thực sự hoạt động làm giảm chu kỳ hoạt động xuống hay không. Nhóm cũng sẽ kiểm tra các giá trị được lưu trữ trong các thanh ghi tính toán để xác minh kiến trúc mới vẫn hoạt động chính xác.
- ❖ Trong giai đoạn này không có lệnh nhánh nào được thực thi. Toàn bộ tín hiệu btb_hit, predict_taken, pred_taken_EX và mispredict giữ giá trị mặc định. predict_taken = 1 do PHT ở trạng thái khởi đầu 2'b10.

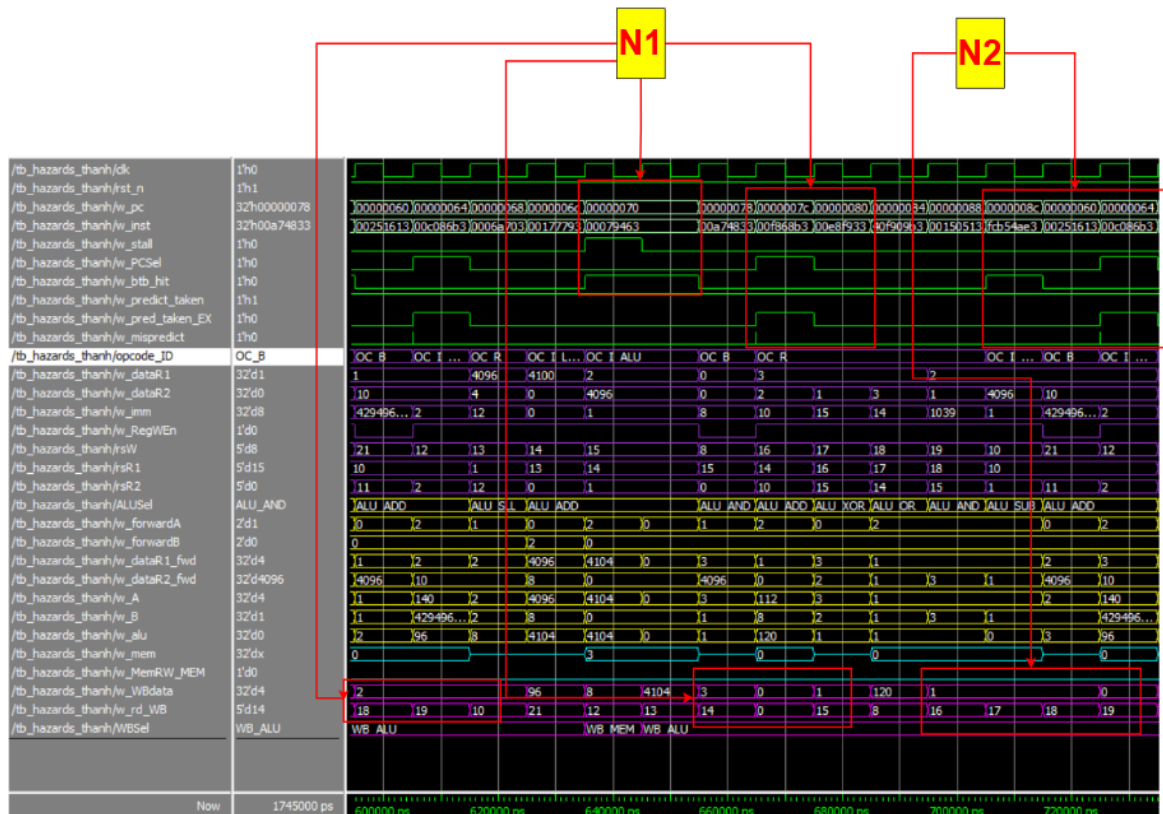
Hình 4.15C mô tả dạng sóng hoạt động ở vòng lặp lần 1 (x10 = 0, x14 = 1 — số lẻ)



Hình 4.15C Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 285 - 455ns

- ❖ Tại điểm N1, lệnh `bne x15, x0, 0x78` — Lần gặp đầu tiên
 - Lệnh `bne` đầu tiên tại địa chỉ `0x70` đi qua pipeline. Tại thời điểm lệnh này, `btb_hit` = 0 vì BTB chưa cập nhật địa chỉ `0x70` nên tín hiệu `pred_taken_EX` = 0 bất kể giá trị của `predict_taken` và không đưa ra dự đoán taken.
 - Khi lệnh `bne` đến giai đoạn EX. Vì `pred_taken_EX` = 0 trong khi thực tế taken (`x15` = 1 $\neq$ `x0`), `mispredict` = 1, pipeline phải flush 2 chu kỳ và PC nhảy về `0x78`. Sau chu kỳ này, BTB ghi nhận địa chỉ `0x70` và PHT cập nhật từ 10 lên 11 (strongly taken).
- ❖ Tại điểm N2, lệnh `blt x10, x11, 0x60` — Lần gặp đầu tiên
 - Tương tự, đây là lần đầu tiên lệnh `blt` tại `0x8C` được thực thi. Tại EX, thực tế taken (`x10` = 1 < `x11` = 10), dẫn đến `mispredict` = 1, flush 2 chu kỳ, PC nhảy về `0x60`. BTB ghi nhận `0x8C`, PHT cập nhật từ 2'b10 lên 2'b11.

Hình 4.15D mô tả dạng sóng hoạt động ở vòng lặp lần 2 (`x10` = 1, `x14` = 2 — số chặn)



Hình 4.15E Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 595 - 735ns

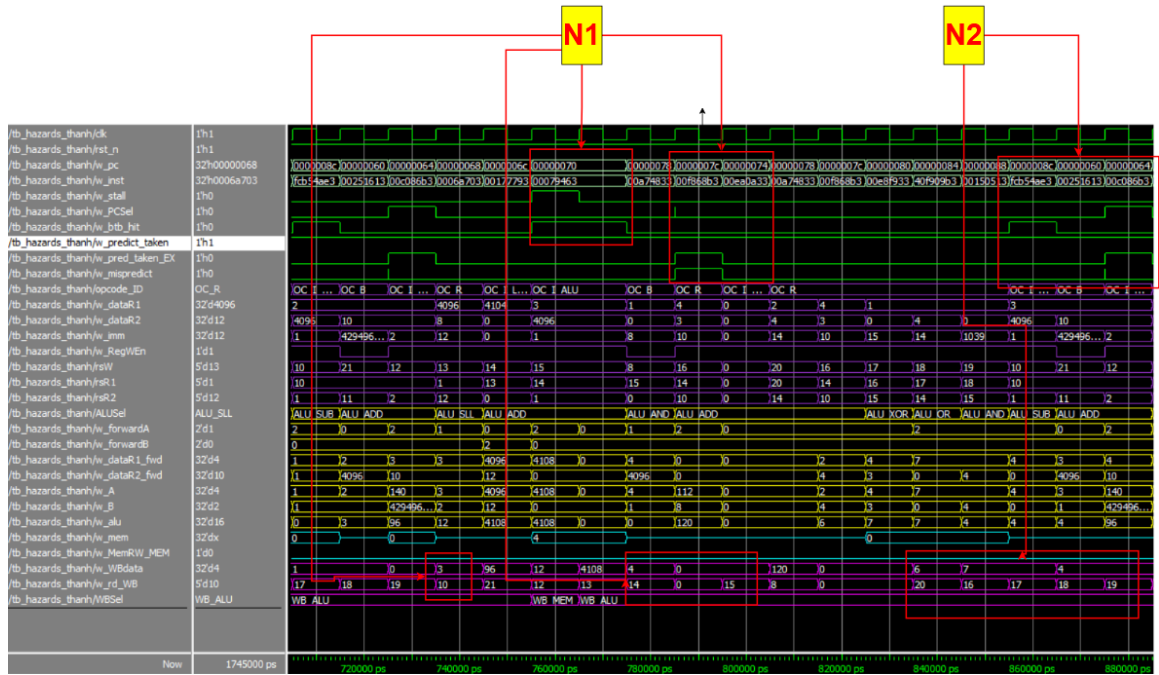
❖ Tại điểm N1, lệnh bne x15, x0, 0x78 — Lần gặp thứ 3

- btb_hit = 1, PHT ở 2'b10 (sau lần mispredict vòng 2), predict_taken = 1, pred_taken_EX = 1. Pipeline dự đoán taken, nạp lệnh từ 0x78. Thực tế: x14 = 3 (lẻ), x15 = 1, bne taken. mispredict = 0. Dự đoán đúng. PHT cập nhật từ 2'b10 lên 2'b11.

❖ Tại điểm N2, lệnh blt x10, x11, 0x60 — Lần gặp thứ 3

- btb_hit = 1, predict_taken = 1 (PHT 2'b11), pred_taken_EX = 1. Thực tế taken (x10 = 3 < 10). mispredict = 0. Dự đoán đúng. PHT giữ 2'b11.

Hình 4.15F mô tả dạng sóng hoạt động ở vòng lặp lần 4 (x10 = 3, x14 = 4 — số chẵn)



Hình 4.15F Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 705 - 875ns

❖ Tại điểm N1, lệnh bne x15, x0, 0x78 — Lần gặp thứ 4

- btb_hit = 1, PHT ở 2'b11, pred_taken_EX = 1. Pipeline dự đoán taken. Thực tế: x14 = 4 (chẵn), x15 = 0, not taken. mispredict = 1. Dự đoán sai, flush 2 chu kỳ.

❖ Tại điểm N2, lệnh blt x10, x11, 0x60 — Lần gặp thứ 4

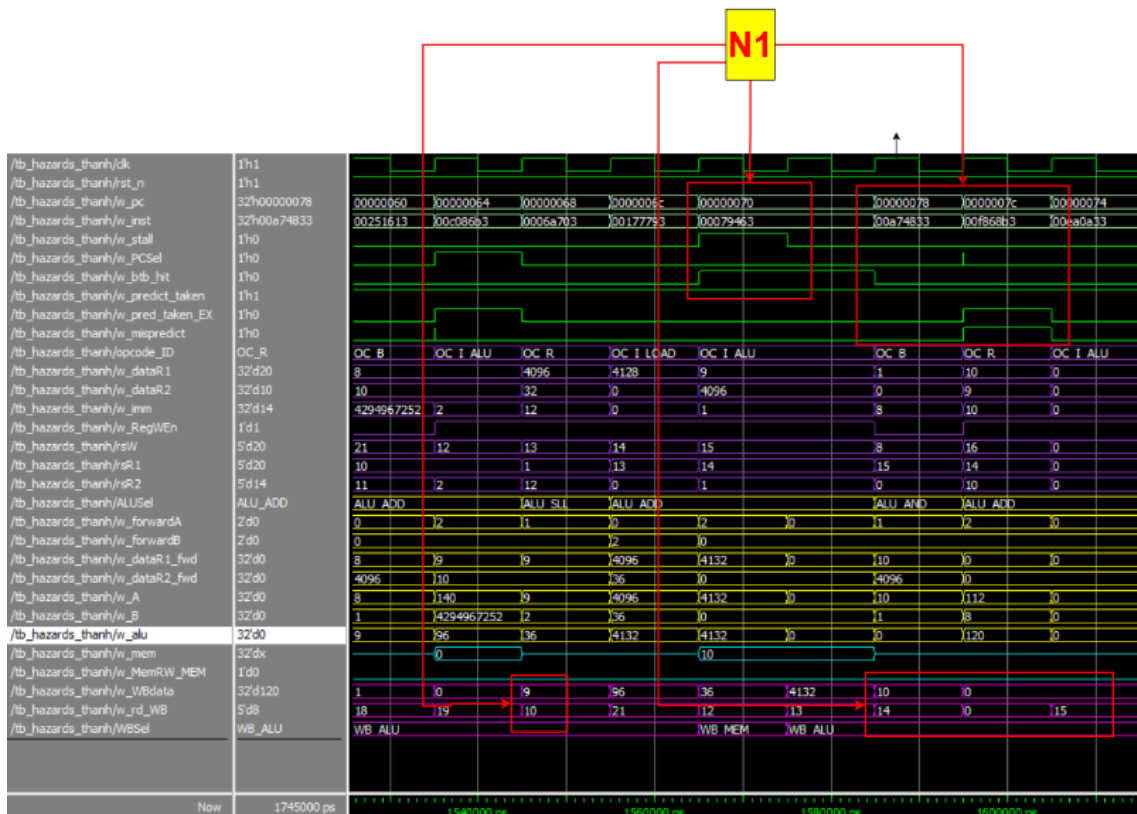
- pred_taken_EX = 1, thực tế taken (x10 = 4 < 10). mispredict = 0. Dự đoán đúng.

❖ Từ vòng 5 đến vòng 9, pattern của bne lặp lại hoàn toàn theo chu kỳ chẵn/lẻ xen kẽ:

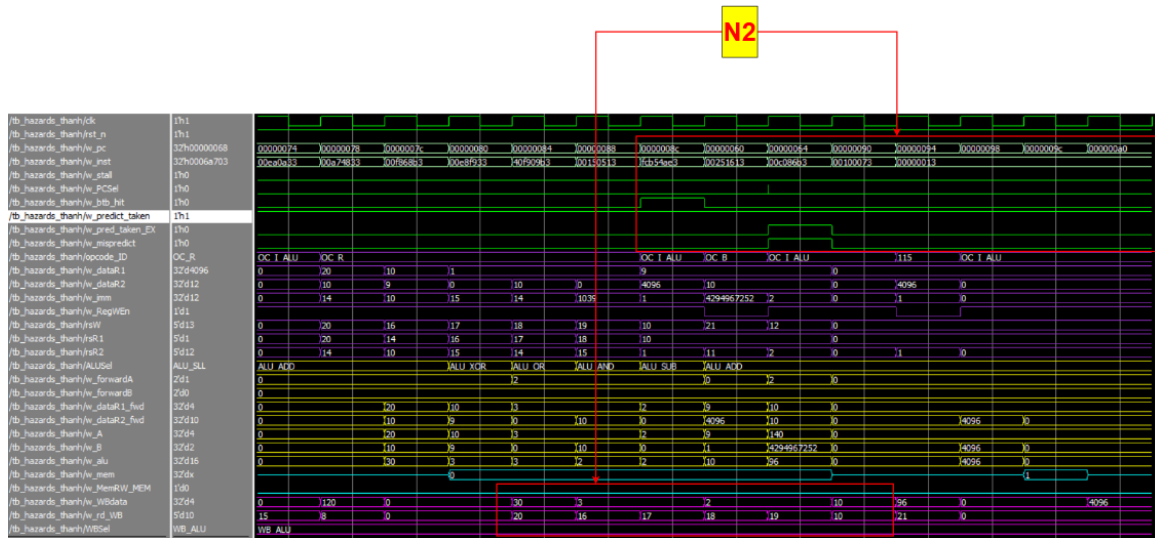
- Các vòng lẻ (x10 = 4, 6, 8 → x14 = 5, 7, 9): PHT ở trạng thái 2'b10 khi vào vòng bne, pred_taken_EX = 1, thực tế taken → mispredict = 0, dự đoán đúng, PHT lên 2'b11.
- Các vòng chẵn (x10 = 5, 7 → x14 = 6, 8): PHT ở 2'b11, pred_taken_EX = 1, thực tế not taken → mispredict = 1, dự đoán sai, PHT giảm xuống 2'b10.

- Đối với blt, từ vòng 3 đến vòng 9 đều có btb_hit = 1, pred_taken_EX = 1, thực tế taken, mispredict = 0. Bộ dự đoán nhánh hoạt động chính xác liên tục cho blt trong 8 vòng lặp này, loại bỏ flush 2 chu kỳ so với pipeline cơ bản.

Hình 4.15G và hình 4.15H mô tả dạng sóng hoạt động ở vòng lặp lần cuối (x10 = 9, x14 = 10 — số chẵn)



Hình 4.15G Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 1525 - 1615ns



Hình 4.15H Dạng sóng hoạt động toàn hệ thống có tích hợp dự đoán nhánh từ 1615 - 1745ns

- ❖ Tại điểm N1, lệnh bne x15, x0, 0x78 — Lần gặp thứ 10
- ❖ btb_hit = 1, PHT ở 2'b11, pred_taken_EX = 1. Thực tế: x14 = 10 (chẵn), x15 = 0, not taken. mispredict = 1. Dự đoán sai, flush 2 chu kỳ, add x20 được thực thi (x20 = 10).
- ❖ Tại điểm N2, lệnh blt x10, x11, 0x60 — Lần gặp thứ 10 (vòng cuối)
- ❖ btb_hit = 1, PHT ở 2'b11, pred_taken_EX = 1. Pipeline dự đoán taken, nạp lệnh từ 0x60. Thực tế: x10 = 10, x11 = 10, điều kiện 10 < 10 sai, not taken, PCSel = 0. Vì pred_taken_EX = 1 nhưng thực tế not taken, mispredict = 1. Dự đoán sai, flush 2 chu kỳ, PC tiếp tục tuần tự sang 0x90 để thực thi ecall.

=> Kết quả cho thấy kiến trúc RV32I có tích hợp bộ dự đoán nhánh đã hoạt động hoàn toàn chính xác so với kiến trúc RV32I cơ bản ban đầu, nhưng hiệu năng khi có bộ dự đoán nhánh lại cao hơn khi hoàn thành chương trình ở 1745ns trong khi kiến trúc cơ bản lại hoàn thành ở 1865ns và cả 2 đều bắt đầu hoạt động sau khi reset được tắt ở 45ns. Qua đó cho thấy ngay từ khi tỷ lệ xuất hiện lệnh rẽ nhánh còn rất ít thì đã có sự khác biệt giữa 2 kiến trúc. Để làm rõ hơn về phần hiệu năng

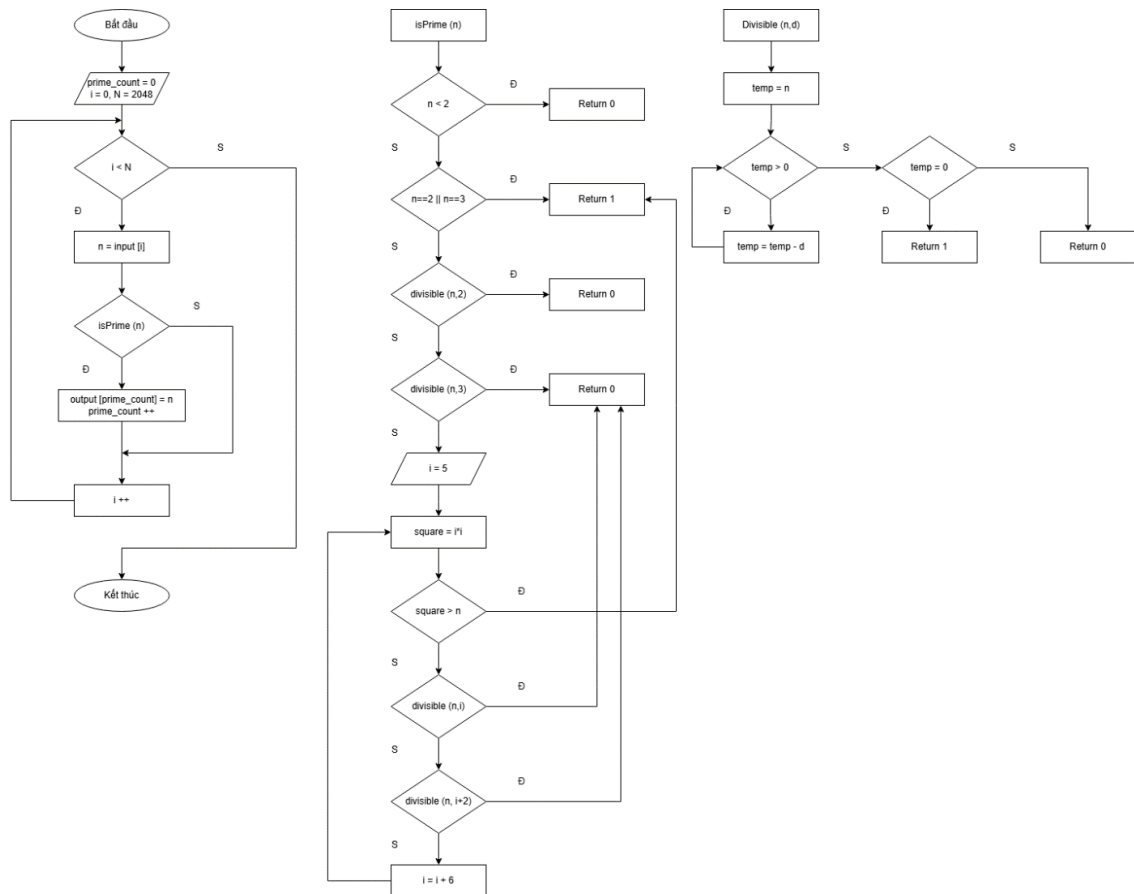
thì cần kiểm chứng qua nhiều chương trình khác nhau mà ở đó tỷ lệ nhánh được đẩy lên cao hơn nhiều so với các chương trình cơ bản thông thường.

4.3 Đánh giá hiệu năng pipeline và pipeline có tích hợp dự đoán nhánh

4.3.1 Chương trình kiểm thử

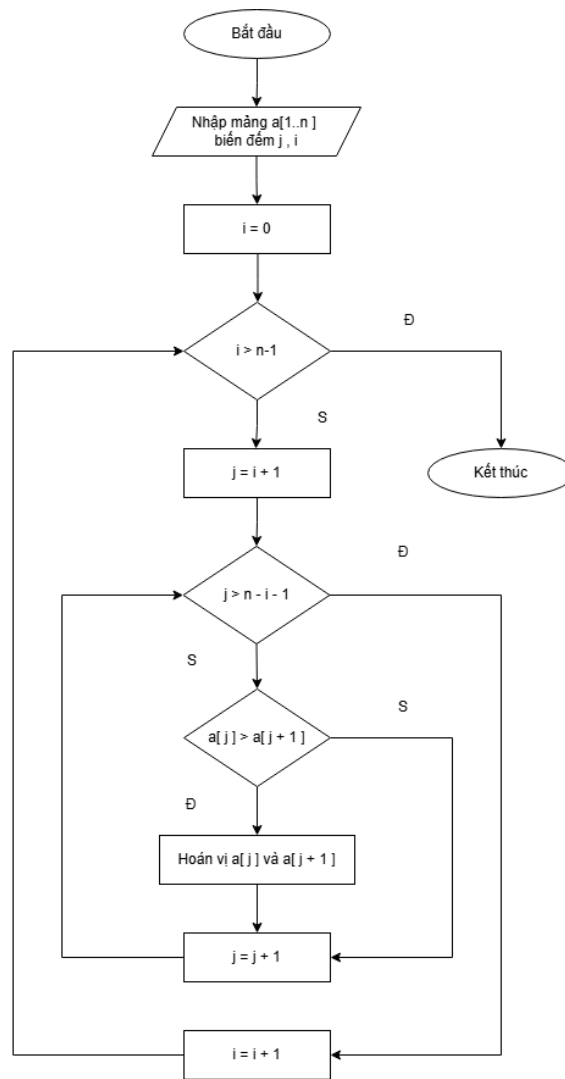
Để đánh giá hiệu quả của bộ dự đoán nhánh một cách toàn diện, ba chương trình kiểm thử được lựa chọn dựa trên tiêu chí tỷ lệ lệnh nhánh và đặc tính hành vi rẽ nhánh khác nhau, nhằm phản ánh đa dạng các tình huống thực tế mà pipeline có thể gặp phải.

Hình 4.16 thể hiện lưu đồ của chương trình tìm số nguyên tố, là chương trình có tỷ lệ lệnh nhánh cao nhất trong bộ kiểm thử, chiếm khoảng 65% tổng số lệnh. Chương trình này được lựa chọn để kiểm tra hiệu năng trong kịch bản tốt nhất cho bộ dự đoán nhánh khi pattern rẽ nhánh có tính lặp lại và định hướng cao, đồng thời được kiểm thử cả khi thuật toán chưa tối ưu và thuật toán đã tối ưu, nhằm đánh giá thêm ảnh hưởng của chất lượng mã nguồn đến hiệu năng tổng thể.



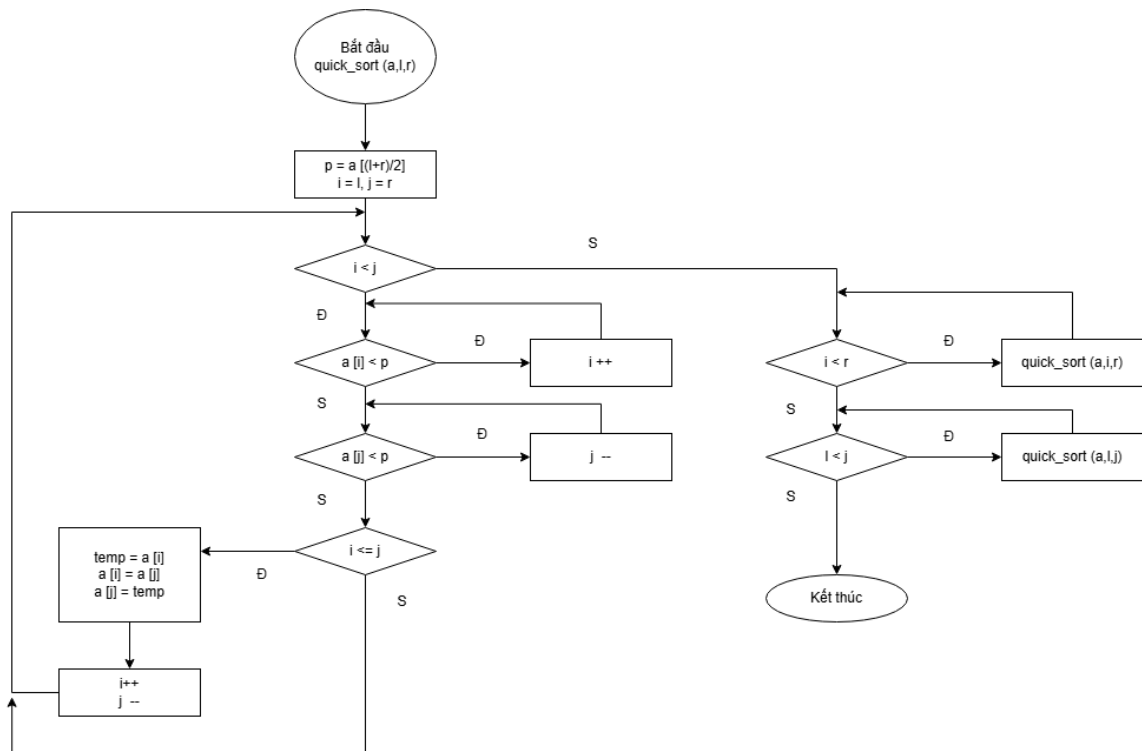
Hình 4.16 Lưu đồ của chương trình tìm số nguyên tố

Hình 4.17 thể hiện lưu đồ của chương trình thuật toán Bubble Sort được lựa chọn với tỷ lệ lệnh nhánh ở mức khá, chiếm khoảng 36% tổng số lệnh. Thuật toán này có cấu trúc vòng lặp lồng nhau đặc trưng, tạo ra số lượng lớn lệnh lw/sw xen kẽ với các lệnh nhánh, dẫn đến nhiều load-use hazard. Điều này cho phép đánh giá hiệu năng bộ dự đoán nhánh trong môi trường pipeline vốn đã có overhead từ stall, là kịch bản phản ánh sát thực các ứng dụng xử lý dữ liệu thực tế.



Hình 4.17 Lưu đồ của chương trình thuật toán Bubble Sort

Hình 4.18 thể hiện lưu đồ của chương trình thuật toán Quick Sort cũng có tỷ lệ lệnh nhánh khoảng 30%, tương đương Bubble Sort, nhưng mang đặc tính pattern rẽ nhánh ít có tính lặp lại và khó dự đoán hơn so với các vòng lặp tuyến tính. Chương trình này được lựa chọn để đánh giá hiệu năng bộ dự đoán nhánh trong kịch bản khó nhất, nơi tính cục bộ của lịch sử nhánh thấp và độ chính xác dự đoán bị giới hạn bởi tính bất định của dữ liệu.



Hình 4.18 Lưu đồ của chương trình thuật toán Quick Sort

Ba chương trình trên tạo thành một bộ kiểm thử có độ phủ rộng, từ kịch bản thuận lợi nhất (tỷ lệ nhánh cao, pattern đều đặn) đến kịch bản bất lợi nhất (nhánh phụ thuộc dữ liệu, đệ quy), cho phép đánh giá khách quan phạm vi hiệu quả và giới hạn của bộ dự đoán nhánh được tích hợp. Đặc biệt cả 2 chương trình thuật toán Bubble Sort và Quick Sort đều cùng có kích thước đầu vào giống nhau toàn để tiện kiểm chứng tính đúng đắn của thiết kế.

4.3.2 Kiểm chứng chức năng

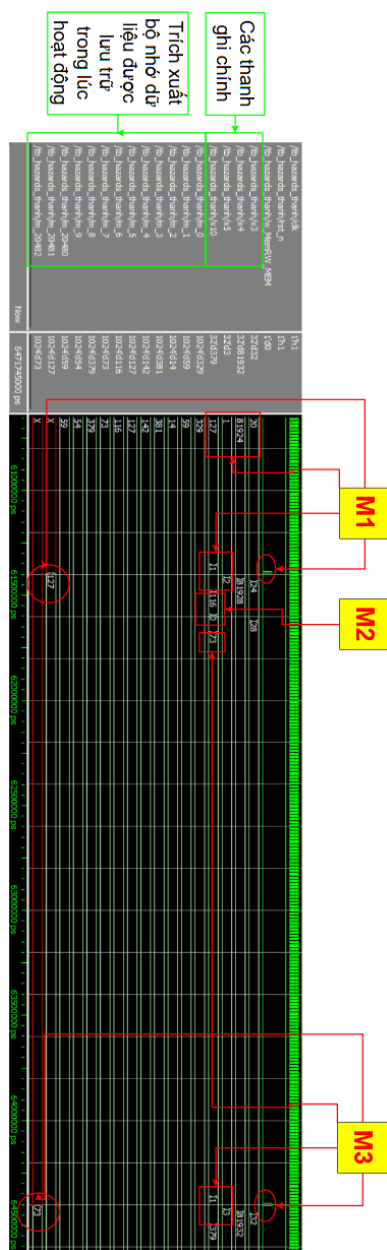
Phần này trình bày kết quả kiểm chứng chức năng của ba chương trình chạy trên CPU pipeline gồm chương trình tìm số nguyên tố, thuật toán Bubble Sort và thuật toán Quick Sort. Phương pháp kiểm chứng gồm:

- Kiểm chứng qua waveform: quan sát dạng sóng tín hiệu theo chu kỳ xung nhịp để xác nhận CPU pipeline thực thi đúng luồng điều khiển và kết quả được ghi vào đúng thanh ghi/ô nhớ đích.

- Kiểm chứng qua memory dump: truy xuất trực tiếp nội dung vùng nhớ dữ liệu trước và sau khi thực thi để đối chiếu kết quả.

4.3.2.1 Chương trình tìm số nguyên tố

a. Kiểm chứng qua waveform



lưu dữ liệu nếu là số nguyên tố, x5 là biến đếm số nguyên tố, x10 là giá trị cần xét xem có phải số nguyên tố không, nếu đúng thì ở cuối chu kì sẽ trả về giá trị 1 còn sai thì trả về 0. Tại điểm M1 ta lấy dữ liệu tại địa chỉ 20 theo byte offset thì khi Memory đọc vào sẽ chuyển thành word – mem[5] đúng với kiến trúc RV32I, ta xét giá trị x10=127 là số nguyên tố nên x10 trả về 1, x5 cũng tăng lên thành 2 và được lưu vào địa chỉ 81924 theo byte offset, tương đương với mem[20481]. Quan sát thêm tín hiệu m_20481 chính là ô nhớ mem[20481] đã lưu đúng giá trị 127 và khi dữ liệu được nạp vào bộ nhớ thì tín hiệu Mem_RW_MEM=1, đúng với thiết kế. Nhưng tại điểm M2 thì x10=116 không phải số nguyên tố nên x10 trả về giá trị 0 và không được lưu vào ô nhớ tiếp theo là mem[20482] vì chỉ có số nguyên tố mới được lưu vào ô nhớ. Tương tự ở điểm M3 ta xét x10=73 là số nguyên tố nên x10 trả về giá trị 1 và được lưu đúng vào ô nhớ tiếp theo là mem[20482].

b. Kiểm chứng qua memory dump

/tb_hazards_thanh/m_0	1024'd329	329	/tb_hazards_thanh/m_20480	1024'd59	59
/tb_hazards_thanh/m_1	1024'd59	59	/tb_hazards_thanh/m_20481	1024'd127	127
/tb_hazards_thanh/m_2	1024'd14	14	/tb_hazards_thanh/m_20482	1024'd73	73
/tb_hazards_thanh/m_3	1024'd381	381	/tb_hazards_thanh/m_20483	1024'd379	379
/tb_hazards_thanh/m_4	1024'd142	142	/tb_hazards_thanh/m_20484	1024'd281	281
/tb_hazards_thanh/m_5	1024'd127	127	/tb_hazards_thanh/m_20485	1024'd17	17
/tb_hazards_thanh/m_6	1024'd116	116	/tb_hazards_thanh/m_20486	1024'd113	113
/tb_hazards_thanh/m_7	1024'd73	73	/tb_hazards_thanh/m_20487	1024'd103	103
/tb_hazards_thanh/m_8	1024'd379	379	/tb_hazards_thanh/m_20488	1024'd281	281
/tb_hazards_thanh/m_9	1024'd54	54	/tb_hazards_thanh/m_20489	1024'd5	5
/tb_hazards_thanh/m_2038	1024'd307	307	/tb_hazards_thanh/m_20829	1024'd353	353
/tb_hazards_thanh/m_2039	1024'd3	3	/tb_hazards_thanh/m_20830	1024'd463	463
/tb_hazards_thanh/m_2040	1024'd143	143	/tb_hazards_thanh/m_20831	1024'd487	487
/tb_hazards_thanh/m_2041	1024'd441	441	/tb_hazards_thanh/m_20832	1024'd307	307
/tb_hazards_thanh/m_2042	1024'd505	505	/tb_hazards_thanh/m_20833	1024'd499	499
/tb_hazards_thanh/m_2043	1024'd76	76	/tb_hazards_thanh/m_20834	1024'd107	107
/tb_hazards_thanh/m_2044	1024'd507	507	/tb_hazards_thanh/m_20835	1024'd37	37
/tb_hazards_thanh/m_2045	1024'd68	68	/tb_hazards_thanh/m_20836	1024'd463	463
/tb_hazards_thanh/m_2046	1024'd278	278	/tb_hazards_thanh/m_20837	1024'd307	307
/tb_hazards_thanh/m_2047	1024'd130	130	/tb_hazards_thanh/m_20838	1024'd3	3
			/tb_hazards_thanh/x5	32'd359	359

Hình 4.20 Dữ liệu bộ nhớ đầu vào và đầu ra của chương trình tìm số nguyên tố

Dựa vào hình 4.20 có thể thấy chương trình tìm số nguyên tố đã hoạt động chính xác khi lọc đúng các số nguyên tố từ các số nguyên bất kì ban đầu và lưu chúng vào ô nhớ chính xác. Cụ thể phía bên trái hình 4.33 biểu thị 10 giá trị đầu từ m_0 đến m_9 và 10 giá trị cuối từ m_2038 đến m_2047 trong tổng 2048 số nguyên dương bất kì thuộc Range [2:511], còn phía bên phải hình 4.33 biểu thị 10

xếp lại, giá trị của x2 và x31 là biến đếm khởi tạo và giới hạn cho vòng lặp, x3 là địa chỉ Memory của số cần so sánh và số ngay sau nó (mem[j] và mem[j+1]), tới khi bằng x31 sẽ tìm ra được giá trị lớn nhất cho từng vòng lặp và sắp xếp chúng đúng quy luật. Ta thấy tại vị trí bắt đầu M1, giá trị x6=327 lớn hơn x7=57 tương đương mem[0] lớn hơn mem[1] (vì x3=0 nên lấy giá trị mem[0] và mem[1]) nên 2 số sẽ bị hoán đổi với nhau, lúc này vẫn sẽ lưu giá trị x6=327 là số lớn hơn. Kiểm tra giá trị m_0 và m_1 chính là giá trị ô nhớ của mem[0] và mem[1] đã được lưu chính xác 2 giá trị đã hoán đổi. Quan sát thêm 2 tín hiệu addr và Mem_RW_MEM, khi Mem_RW_MEM = 1 thì các giá trị thanh ghi đúng sẽ được lưu vào địa chỉ tương ứng, ở đây là 32'd0 và 32'd4, do CPU đọc địa chỉ theo dạng byte offset còn Memory sẽ đọc địa chỉ theo word do bỏ 2 bit cuối theo đúng kiến trúc RV32I nên địa chỉ 32'd0 và 32'd4 khi lưu vào Memory sẽ tương ứng với mem[0] và mem[1]. Tương tự tại điểm M2 cũng phát hiện được mem[1] = 327 lớn hơn mem[2] = 12 nên giá trị của 2 ô nhớ sẽ được đảo lại với nhau. Nhưng tại điểm M3 thì mem[2]=327 lại nhỏ hơn mem[3]=379 nên 2 ô nhớ không đảo lại, giá trị Mem_RW_MEM = 0 hoàn toàn chính xác, đồng thời lưu giá trị x6=379 là giá trị lớn nhất mới của cả vòng và cứ như vậy cho đến 1024 vòng sẽ tìm được số lớn nhất đưa xuống ô nhớ cuối cùng.

b. Kiểm chứng qua memory dump

[illegible]

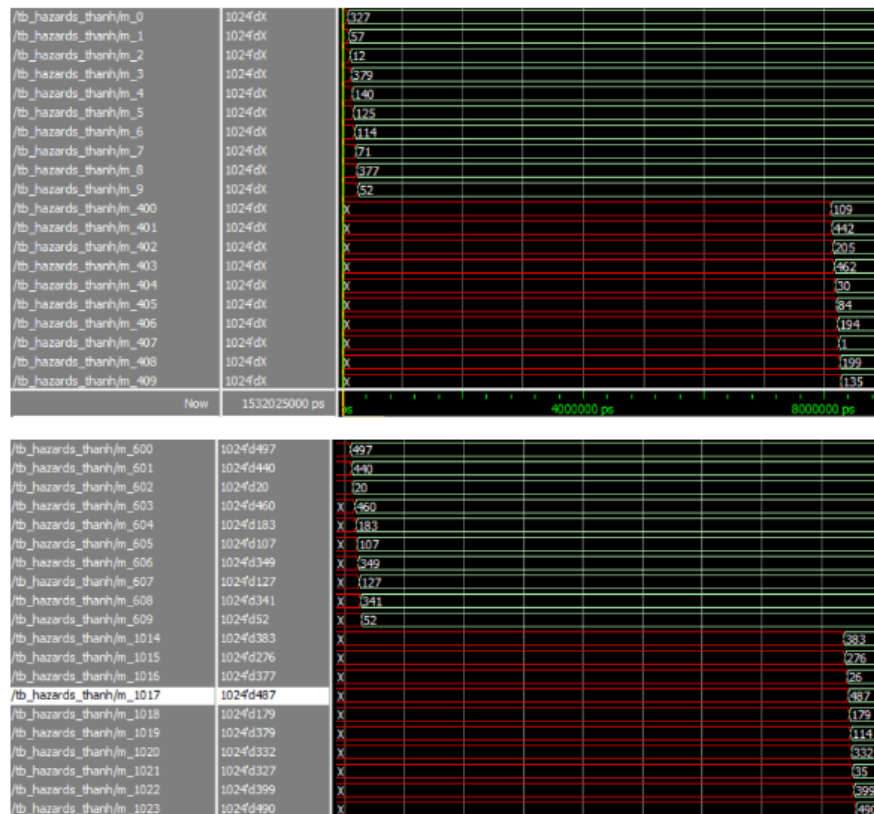
Hình 4.22 Dữ liệu bộ nhớ trước khi sắp xếp theo thuật toán Bubble Sort

fb_hazards_thanh/m_0	1024d57	0	fb_hazards_thanh/m_600	1024d295	295
fb_hazards_thanh/m_1	1024d12	0	fb_hazards_thanh/m_601	1024d295	295
fb_hazards_thanh/m_2	1024d327	0	fb_hazards_thanh/m_602	1024d295	295
fb_hazards_thanh/m_3	1024d140	1	fb_hazards_thanh/m_603	1024d296	296
fb_hazards_thanh/m_4	1024d125	1	fb_hazards_thanh/m_604	1024d296	296
fb_hazards_thanh/m_5	1024d114	1	fb_hazards_thanh/m_605	1024d296	296
fb_hazards_thanh/m_6	1024d71	2	fb_hazards_thanh/m_606	1024d297	297
fb_hazards_thanh/m_7	1024d377	3	fb_hazards_thanh/m_607	1024d298	298
fb_hazards_thanh/m_8	1024d52	3	fb_hazards_thanh/m_608	1024d298	298
fb_hazards_thanh/m_9	1024d346	3	fb_hazards_thanh/m_609	1024d298	298
fb_hazards_thanh/m_400	1024d109	196	fb_hazards_thanh/m_1014	1024d495	495
fb_hazards_thanh/m_401	1024d442	196	fb_hazards_thanh/m_1015	1024d495	495
fb_hazards_thanh/m_402	1024d205	197	fb_hazards_thanh/m_1016	1024d496	496
fb_hazards_thanh/m_403	1024d462	197	fb_hazards_thanh/m_1017	1024d496	496
fb_hazards_thanh/m_404	1024d30	199	fb_hazards_thanh/m_1018	1024d497	497
fb_hazards_thanh/m_405	1024d84	199	fb_hazards_thanh/m_1019	1024d497	497
fb_hazards_thanh/m_406	1024d194	199	fb_hazards_thanh/m_1020	1024d497	497
fb_hazards_thanh/m_407	1024d1	199	fb_hazards_thanh/m_1021	1024d498	498
fb_hazards_thanh/m_408	1024d199	201	fb_hazards_thanh/m_1022	1024d498	498
fb_hazards_thanh/m_409	1024d135	202	fb_hazards_thanh/m_1023	1024d499	499
			fb_hazards_thanh/m_1024	1024d4d	x
			fb_hazards_thanh/m_1025	1024d4d	x

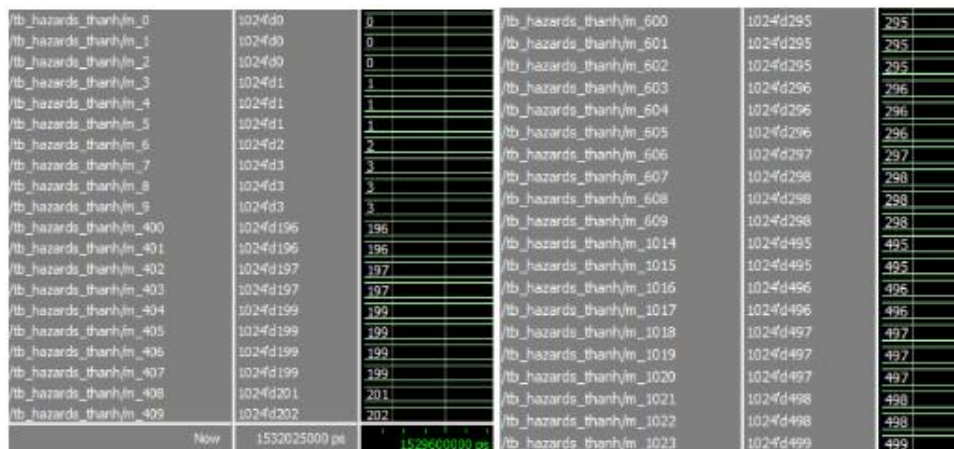
Hình 4.23 Dữ liệu bộ nhớ sau khi sắp xếp theo thuật toán Bubble Sort

Dựa vào hình 4.22 và hình 4.23 có thể thấy thuật toán đã hoạt động chính xác khi sắp xếp đúng thứ tự các giá trị từ nhỏ đến lớn vào đúng ô nhớ từ `mem[0]` tới `mem[1023]` tương ứng với từ `m - 0` tới `m - 1023`.

4.3.2.3 Thuật toán Quick Sort



Hình 4.25 Dữ liệu bộ nhớ trước khi sắp xếp theo thuật toán Quick Sort

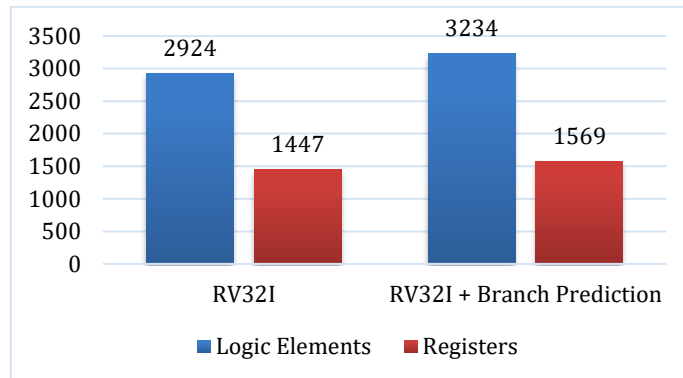


Hình 4.26 Dữ liệu bộ nhớ sau khi sắp xếp theo thuật toán Quick Sort

Dựa vào hình 4.25 và hình 4.26 có thể thấy thuật toán đã hoạt động chính xác khi sắp xếp đúng thứ tự các giá trị từ nhỏ đến lớn vào đúng ô nhớ từ mem[0] tới mem[1023] tương ứng với từ m_0 tới m_1023.

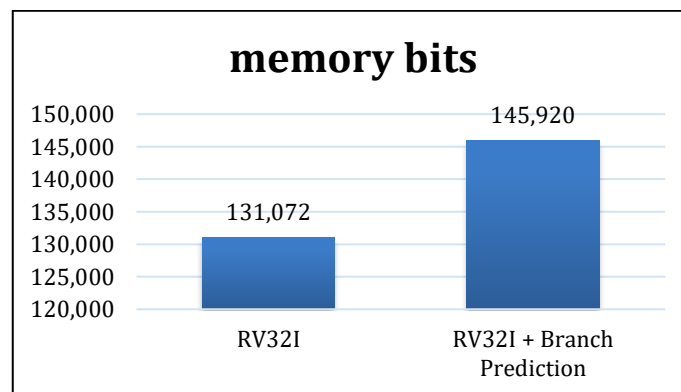
4.3.3 Đánh giá hiệu suất

4.3.3.1 Đánh giá phần cứng



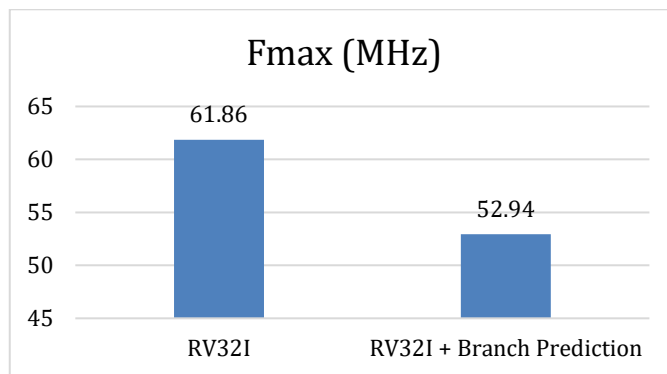
Hình 4.27 So sánh độ chiếm dụng Logic element và Register giữa hai kiến trúc

Số lượng Logic element tăng từ 2924 lên 3234, tăng khoảng 10,6%. Đồng thời số lượng Register tăng từ 1447 lên 1569, tăng khoảng 8,4%. Vì kiến trúc dự đoán nhánh cần bổ sung các khối phần cứng như Branch Target Buffer (BTB), Pattern History Table (PHT) và các thanh ghi lưu trữ trạng thái dự đoán nên đã làm tăng số lượng logic tổ hợp và phần tử tuần tự trong thiết kế.



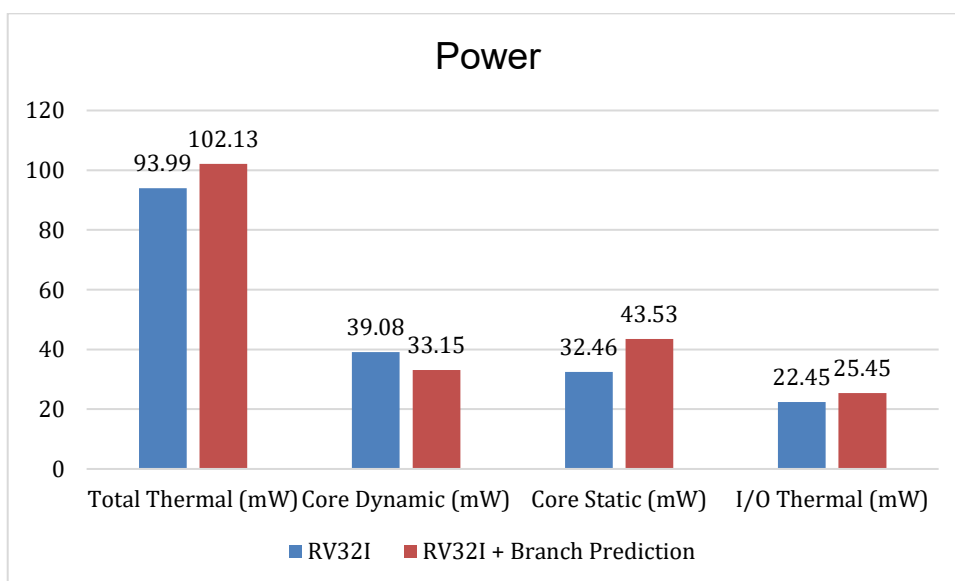
Hình 4.28 So sánh độ chiếm dụng Memory Bit giữa hai kiến trúc

Số lượng Memory bit tăng từ 131072 bit lên 145920 bit, tăng khoảng 11,3% vì bộ dự đoán nhánh cần sử dụng bộ nhớ để lưu thông tin lịch sử nhánh và địa chỉ đích của các lệnh rẽ nhánh.



Hình 4.29 So sánh tần số hoạt động cực đại giữa hai kiến trúc

Tần số hoạt động cực đại giảm từ 61,86 MHz xuống còn 52,94 MHz, giảm khoảng 14,4% vì đường dữ liệu và đường điều khiển trở nên phức tạp hơn do xuất hiện các khối BTB, PHT và logic xử lý cập nhật dự đoán. Điều này làm tăng độ dài critical path, khiến quá trình synthesis phải làm giảm tần số hoạt động tối đa để đảm bảo thời gian thiết lập và giữ tín hiệu.



Hình 4.30 So sánh công suất tiêu thụ giữa hai kiến trúc

Kết quả phân tích công suất cho thấy việc tích hợp cơ chế dự đoán nhánh làm tổng công suất tiêu thụ của bộ xử lý tăng từ 93,99 mW lên 102,13 mW, tương ứng tăng khoảng 8,7%. Nguyên nhân chủ yếu là do việc bổ sung các khối phân cứng dự đoán nhánh như BTB và PHT, làm gia tăng số lượng tài nguyên logic và

bộ nhớ được sử dụng trên FPGA. Điều này dẫn đến công suất tĩnh tăng đáng kể từ 32,46 mW lên 43,53 mW.

Ngược lại, công suất động của lõi xử lý giảm từ 39,08 mW xuống 33,15 mW. Sự cải thiện này xuất phát từ việc cơ chế dự đoán nhánh giúp giảm số lần flush pipeline do hazard điều khiển, từ đó hạn chế các hoạt động chuyển mạch không cần thiết và nâng cao hiệu quả sử dụng tài nguyên phần cứng trong quá trình thực thi.

Kết quả tổng hợp phần cứng cho thấy bộ xử lý RV32I Pipeline 5 tầng tích hợp cơ chế dự đoán nhánh yêu cầu nhiều tài nguyên hơn so với kiến trúc RV32I Pipeline 5 tầng cơ bản. Tuy nhiên, phần chi phí phần cứng và công suất tiêu thụ tăng thêm là một sự đánh đổi hợp lý để đạt được hiệu năng cao hơn. Nhờ giảm số chu kỳ stall và flush do lệnh nhánh gây ra, cơ chế dự đoán nhánh góp phần cải thiện CPI và rút ngắn thời gian thực thi chương trình, qua đó nâng cao hiệu quả hoạt động tổng thể của bộ xử lý.

4.3.3.2 Đánh giá chương trình tìm số nguyên tố

Bảng 4.2 Đo lường độ hiệu quả của chương trình tìm số nguyên tố chưa được tối ưu

	RV32I	RV32I + Dự đoán nhánh	Độ cải thiện/Đánh đổi
Cycles	1256357	803432	Giảm 36.05%
Instructions	751218	751218	
Branches	489372	489372	Chiếm 65.14%
CPI	1.6724	1.0695	Giảm 36.05%
Not Takens	237827	237827	
Takens + Jumps	251545	251545	
Stalls	0	0	
Fmax	61.86 MHz	52.94 MHz	Chậm hơn 14.4%
T _{exec} = Cycles / Fmax	20.31 ms	15.17 ms	Giảm 25.3%
Accuracy	0%	94.55%	

=> Nhận xét: Khi tích hợp bộ dự đoán nhánh, số chu kỳ giảm đáng kể từ 1.256.357 xuống còn 803.432 (giảm 36,05%), kéo theo CPI giảm tương ứng từ 1,6724 xuống 1,0695 gần với mức lý tưởng $CPI = 1$. Thời gian thực thi cải thiện 25,3% đã bao gồm sự sụt giảm Fmax (từ 61,86 MHz xuống 52,94 MHz do mạch dự đoán làm tăng độ trễ tín hiệu). Tỷ lệ nhánh rất cao (65,14% trên tổng lệnh) nên bộ dự đoán đã phát huy công dụng khi độ chính xác dự đoán 94,55% mang lại hiệu quả rõ rệt.

Bảng 4.3 Đo lường độ hiệu quả của chương trình tìm số nguyên tố được tối ưu

	RV32I	RV32I + Dự đoán nhánh	Độ cải thiện/Đánh đổi
Cycles	1028361	647170	Giảm 37.1%
Instructions	623434	623434	
Branches	394177	394177	Chiếm 63.23%
CPI	1.6495	1.0381	Giảm 37.1%
Not Takens	192738	192738	
Takens + Jumps	201439	201439	
Stalls	0	0	
Fmax	61.86 MHz	52.94 MHz	Chậm hơn 14.4%
T _{exec} = Cycles / Fmax	16.62 ms	12.22 ms	Giảm 26.5%
Accuracy	0%	96.03%	

=> Nhận xét : Sau khi tối ưu thuật toán, số lệnh giảm từ 751.218 xuống 623.434 (giảm ~17%), dự đoán nhánh cho thấy hiệu quả cao hơn so với khi chưa tối ưu, chu kỳ giảm 37,1%, CPI giảm từ 1,6495 xuống 1,0381, thời gian thực thi giảm 26,5%. Đặc biệt, độ chính xác dự đoán tăng lên 96,03% (so với 94,55% ở bảng 4.3), cho thấy thuật toán tối ưu tạo ra các mẫu nhánh có tính quy luật cao hơn, dễ dự đoán hơn. Tổng hợp lại cho thấy việc tối ưu mã nguồn cũng sẽ góp phần giúp bộ dự đoán nhánh phát huy hiệu quả tốt nhất.

4.3.3.3 Đánh giá thuật toán Bubble Sort

Bảng 4.4 Đo lường độ hiệu quả của chương trình thuật toán Bubble Sort

	RV32I	RV32I + Dự đoán nhánh	Độ cải thiện/Đánh đổi
--	-------	-----------------------	-----------------------

Cycles	3186847	2641780	Giảm 17.1%
Instructions	2223352	2223352	
Branches	796351	796351	Chiếm 35.82%
CPI	1.418	1.188	Giảm 17.1%
Not Takens	447176	447176	
Takens + Jumps	349175	349175	
Stalls	265146	265146	
Fmax	61.86 MHz	52.94 MHz	Chậm hơn 14.4%
T_exec = Cycles / Fmax	51.52 ms	49.9 ms	Giảm 3.14%
Accuracy	0%	85.08%	

=> Nhận xét : Bubble Sort là benchmark có quy mô lớn nhất (hơn 2,2 triệu lệnh, chu kỳ lên đến gần 3,19 triệu) do bản chất của thuật toán. Tỷ lệ lệnh nhánh ở mức khá (35,82%) và độ chính xác dự đoán đạt 85,08% cho thấy thuật toán đã trở nên khó dự đoán hơn so với chương trình tìm số nguyên tố. Dự đoán nhánh giúp giảm 17,1% chu kỳ và CPI giảm từ 1,418 xuống 1,188, tuy nhiên mức cải thiện thời gian thực thi chỉ đạt 3,14% – rất thấp so với mức giảm chu kỳ. Nguyên nhân chủ yếu đến từ hai yếu tố:

- Stalls = 265.146 – đây là benchmark có stall khác 0, cho thấy tồn tại các hazard dữ liệu hoặc cấu trúc không thể che giấu hoàn toàn bằng dự đoán nhánh, làm hạn chế hiệu quả tổng thể. Nhóm em đã tính toán nếu không có lượng stall này thì mức cải thiện thời gian sẽ được nâng lên khoảng 5% so với 3.14% ban đầu, tuy vẫn không được cải thiện đáng kể nhưng cũng cho thấy được sự ảnh hưởng của hazard dữ liệu đối với độ hiệu quả của bộ dự đoán nhánh.
- Fmax giảm 14,4% làm triệt tiêu phần lớn lợi ích từ việc giảm chu kỳ.

4.3.3.4 Đánh giá thuật toán Quick Sort

Bảng 4.5 Đo lường độ hiệu quả của chương trình thuật toán Quick Sort

	RV32I	RV32I + Dự đoán nhánh	Độ cải thiện/Đánh đổi
Cycles	180470	153191	Giảm 15.11%
Instructions	120055	120055	

Branches	36219	36219	Chiếm 30.17%
CPI	1.503	1.276	Giảm 15.11%
Not Takens	14576	14576	
Takens + Jumps	21643	21643	
Stalls	0	0	
Fmax	61.86 MHz	52.94 MHz	Chậm hơn 14.4%
T_exec = Cycles / Fmax	2.917 ms	2.894 ms	Giảm 0.8%
Accuracy	0%	68.66%	

=> Nhận xét : Quick Sort có quy mô nhỏ nhất chỉ khoảng 120.000 lệnh và cũng cho kết quả tệ nhất. Mặc dù chu kỳ giảm 15,11% và CPI giảm từ 1,503 xuống 1,276, thời gian thực thi thực tế hầu như không thay đổi (chỉ giảm 0,8%, từ 2,917 ms xuống 2,894 ms). Điều này xảy ra vì mức giảm chu kỳ (15,11%) không đủ để bù đắp sự sụt giảm Fmax (14,4%). Tỷ lệ nhánh thấp nhất trong tất cả benchmark (30,17%) và độ chính xác dự đoán chỉ 68,66% gần với xác suất đoán ngẫu nhiên cho thấy các nhánh trong Quick Sort có tính ngẫu nhiên cao do phụ thuộc vào phần tử chốt và phân bố dữ liệu đầu vào. Đây là trường hợp mà chi phí phần cứng của bộ dự đoán nhánh (giảm Fmax) không được bù đắp xứng đáng bởi lợi ích mang lại.

4.3.4 Nhận xét và đánh giá

Kết quả từ ba chương trình kiểm thử cho phép rút ra một số nhận xét quan trọng về hiệu quả và giới hạn của bộ dự đoán nhánh được tích hợp.

- Thứ nhất, tỷ lệ lệnh nhánh và độ chính xác dự đoán (Accuracy) là hai yếu tố quyết định mức độ cải thiện hiệu năng thực tế. Chương trình tìm số nguyên tố với tỷ lệ nhánh 65% và độ chính xác 94–96% mang lại mức cải thiện T_exec lên đến 25–26%, trong khi Quick Sort với tỷ lệ nhánh 30% và độ chính xác 68% chỉ cải thiện 0,8%.
- Thứ hai, chi phí phần cứng Fmax là hằng số cố định (~14,4%) không phụ thuộc vào chương trình chạy. Bộ dự đoán nhánh chỉ thực sự có lợi khi mức giảm chu kỳ vượt quá chi phí phần cứng 14,4% này. Với các chương trình

có tỷ lệ nhánh thấp hoặc pattern nhánh khó dự đoán, xen kẽ thì đây là ngưỡng khó vượt qua.

- Thứ ba, stall do load-use hazard không bị ảnh hưởng bởi bộ dự đoán nhánh. Trong Bubble Sort, 265.146 stall tạo ra một overhead cố định làm giảm hiệu quả tương đối của việc cải thiện CPI từ bộ dự đoán nhánh.
- Thứ tư, việc tối ưu thuật toán hỗ trợ tốt cho bộ dự đoán nhánh. Kết quả từ hai phiên bản của chương trình tìm số nguyên tố cho thấy tối ưu mã nguồn không chỉ giảm số lệnh mà còn làm cho pattern nhánh đều đặn hơn, giúp độ chính xác dự đoán tăng từ 94,55% lên 96,03% và T_{exec} giảm thêm.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Trong phạm vi đề tài, nhóm đã nghiên cứu, thiết kế và xây dựng thành công bộ xử lý RISC-V RV32I pipeline 5 tầng theo kiến trúc IF–ID–EX–MEM–WB. Hệ thống được xây dựng ở mức RTL bằng ngôn ngữ SystemVerilog, đồng thời tích hợp các cơ chế xử lý hazard và dự đoán nhánh động nhằm nâng cao hiệu năng thực thi.

Đề tài đã triển khai thành công cơ chế dự đoán nhánh dựa trên sự kết hợp giữa Branch Target Buffer và Pattern History Table sử dụng bộ đếm bão hòa 2-bit. Kết quả mô phỏng cho thấy bộ xử lý hoạt động đúng với tập lệnh RV32I và thực hiện chính xác các chương trình kiểm thử.

Thông qua quá trình đánh giá, kiến trúc tích hợp dự đoán nhánh đã giúp giảm số chu kỳ thực thi và cải thiện chỉ số CPI đối với các chương trình có nhiều lệnh rẽ nhánh. Mặc dù tài nguyên phần cứng và công suất tiêu thụ tăng nhẹ, hiệu năng tổng thể của hệ thống vẫn được cải thiện đáng kể so với kiến trúc pipeline truyền thống.

Kết quả đạt được đã chứng minh tính khả thi và hiệu quả của việc tích hợp cơ chế dự đoán nhánh vào bộ xử lý RV32I pipeline 5 tầng, đồng thời tạo nền tảng cho các nghiên cứu và phát triển nâng cao trong tương lai.

5.2 Hướng phát triển

Trong tương lai, đề tài có thể được phát triển theo các hướng sau:

- Nghiên cứu và triển khai các bộ dự đoán nhánh có độ chính xác cao hơn như Gshare hoặc TAGE.
- Mở rộng tập lệnh từ RV32I sang các phần mở rộng như RV32M hoặc RV32C.
- Tích hợp bộ nhớ đệm nhằm giảm thời gian truy cập bộ nhớ và nâng cao hiệu năng.

TÀI LIỆU THAM KHẢO

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface (RISC-V Edition)*. Cambridge, MA, USA: Morgan Kaufmann (Elsevier), 2018.
- [2] T.-D. Ta, T.-P. Nguyen, and Q.-T. Tran, "Implementation of an Efficient RISC-V Processor Featuring a Novel Gshare Branch Prediction Technique," *Journal of VLSI Circuits and Systems*, vol. 7, no. 2, pp. 68–76, Dec. 2025.
- [3] D. G. Sutar and N. B. Guinde, "Towards Memory-Efficient and High-Performance Branch Prediction: The LXOR Architecture for Control Flow Optimization in Embedded and General-Purpose RISC-V Processors," *Journal of Low Power Electronics and Applications*, vol. 15, no. 4, art. no. 64, Oct. 2025.
- [4] D. A. Jiménez, "Multiperspective Perceptron Predictor," in *Proceedings of the 5th Championship on Branch Prediction (CBP-5)*, 2016, pp. 1–11.
- [5] A. Seznec, "TAGE-SC-L Branch Predictors Again," *Technical Report, INRIA/IRISA*, pp. 1–11, 2016.
- [6] Y. Tang, Z. Chen, and W. Fang, "Study of Register Value Branch Predictor Based on CNN," *Applied Sciences*, vol. 15, no. 5, art. no. 2725, Mar. 2025.
- [7] Z. Jin, H. Di, T. Hu, and P. Wang, "Real-Time Optimization of RISC-V Processors Based on Branch Prediction and Division Data Dependency," *Applied Sciences*, vol. 15, no. 2, art. no. 632, Jan. 2025.
- [8] M. Yan, J. Choi, D. Skarlatos, A. Morrison, C. W. Fletcher, and J. Torrellas, "InvisiSpec: Making Speculative Execution Invisible in the Cache Hierarchy," in *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018, pp. 428–441.
- [9] R. Singh, "Microarchitectural Advancements in Branch Prediction: Evaluating Modern Techniques for Out-of-Order RISC-V Processors," *University of South Dakota & Elevance Health Research Report*, Oct. 2025.
- [10] Z. Zhou, F. Tong, H. Wang, X. Cheng, F. Jiang, Z. Zhang, and Y. Mao, "CIBPU: A Conflict-Invisible Secure Branch Prediction Unit," arXiv preprint arXiv:2501.10983v1, Jan. 2025.

- [11] H. Li, C. Jing, and J. Liu, "Performance-Optimised Design of the RISC-V Five-Stage Pipelined Processor NRP," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 15, no. 2, pp. 276–282, 2024.
- [12] C. A. Rathi, G. Rajakumar, T. A. Kumar, and T. S. A. Samuel, "Design and Development of an Efficient Branch Predictor for an In-order RISC-V Processor," *Journal of Nano- and Electronic Physics*, vol. 12, no. 5, pp. 05021(1–4), Oct. 2020.
- [13] Z. Jin, T. Hu, Z. Jie, and P. Wang, "Research and Implementation of Performance Optimization Methods for RISC-V Level-5 Processors," *Applied Sciences*, vol. 15, no. 21, art. no. 11634, Oct. 2025.
- [14] A. Aakash, R. Aparna, A. B. Abraham, and S. R. Malathi, "Design of 5 Stage Pipelined RISC-V Processor," *International Journal of Innovative Research in Technology (IJIRT)*, vol. 10, no. 12, pp. 2778–2781, May 2024.